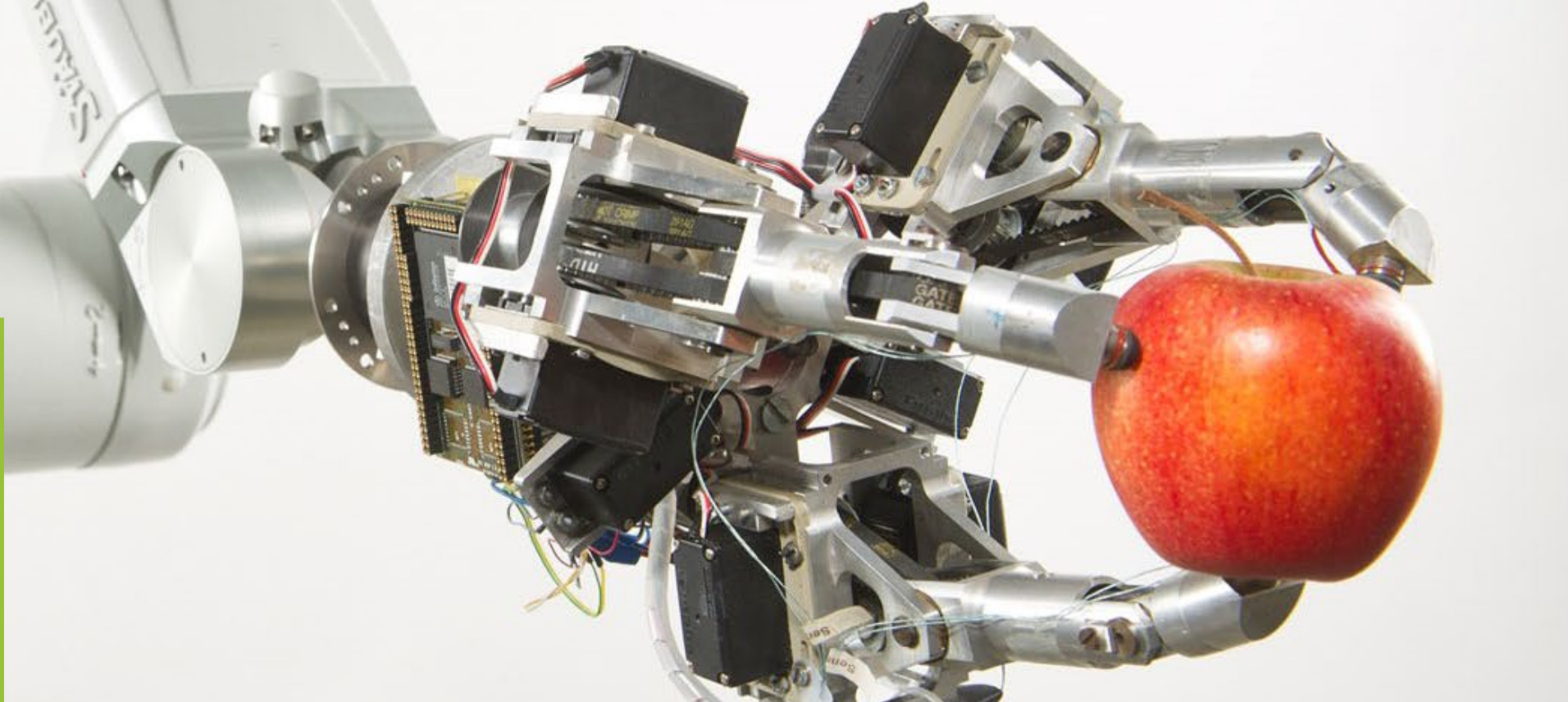




Prof. Dr.-Ing. Jörg Franke

**Lehrstuhl für Fertigungsautomatisierung
und Produktionssystematik**

Friedrich-Alexander-Universität Erlangen-Nürnberg



**Friedrich-Alexander-Universität
Technische Fakultät**

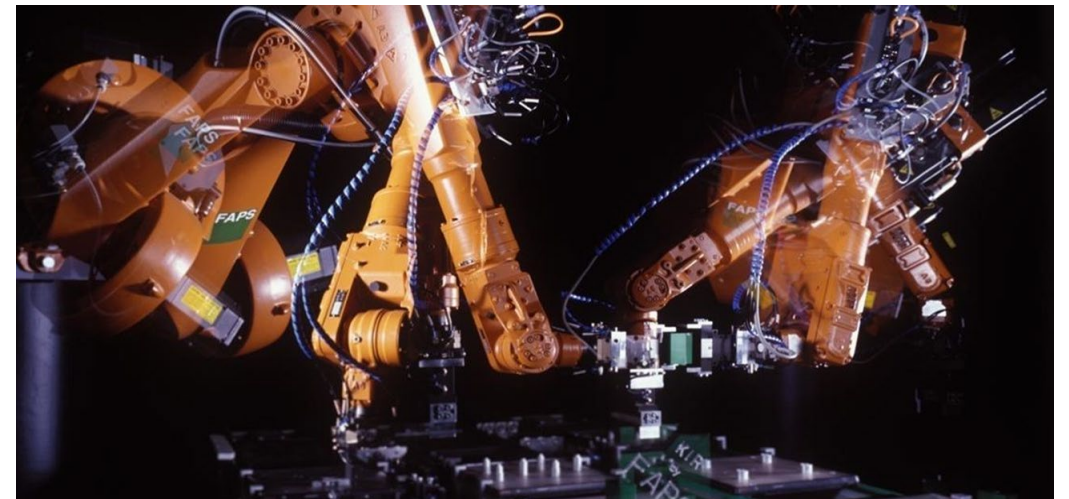
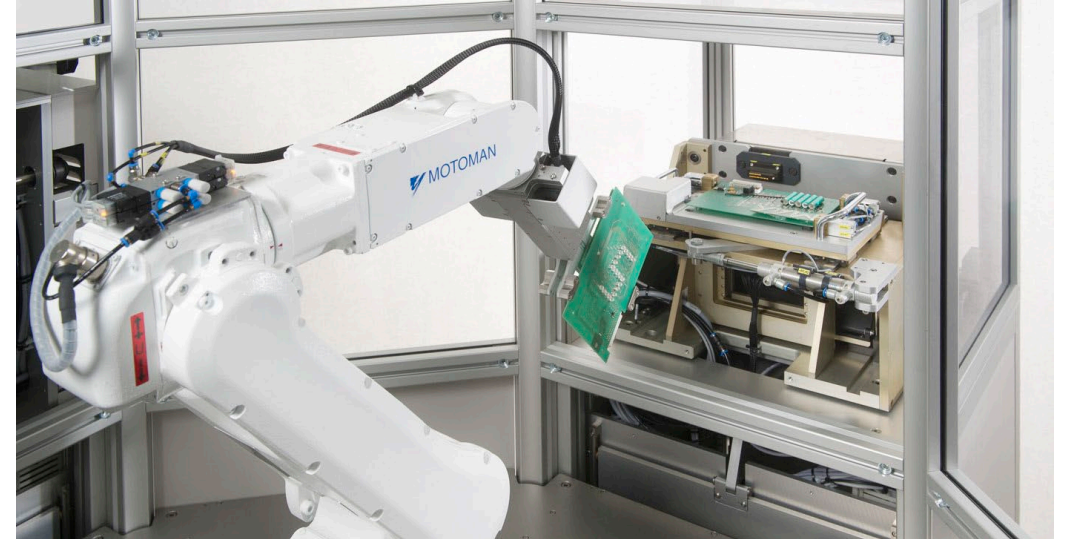
Grundlagen der Robotik

VE05 - Bahnplanung und Regelung

Im Fach Grundlagen der Robotik werden die fachspezifischen Grundlagen zur Konzeption, Implementierung und Realisierung von Robotersystemen vermittelt.

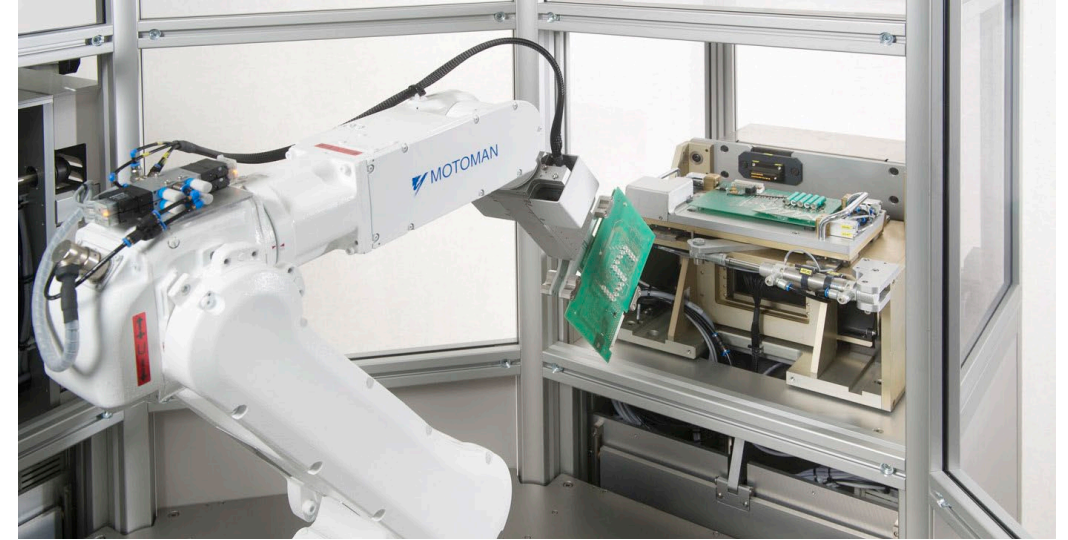
VE	Thema
----	-------

- | | |
|----|--|
| 1 | --- entfällt --- |
| 2 | Einführung und Grundlagen der Industrierobotik & Neuartige Konzepte und Kinematiken in der Robotik |
| 3 | Steuerung - Sensorik – Aktorik |
| 4 | Roboterkinematik |
| 5 | Roboterdynamik |
| 6 | Bahnplanung und Regelung |
| 7 | Grundlagen des Robot Operating System (ROS) |
| 8 | ROS - Erweiterte Konzepte und Tools |
| 9 | Simulation und Planung |
| 10 | Rechnersehen |
| 11 | Maschinelles Lernen |
| 12 | Seilrobotik |
| 14 | Service-, Pflege- und Medizinrobotik |



Die zugehörige Übung setzt sich aus Tafel- und Hausübungen zusammen und fokussiert unter anderem die praktische Arbeit mit dem Robot Operating System.

UE	Thema
1	--- entfällt ---
2	--- entfällt ---
3	Hallenführung in Kleingruppen
4	Steuerung - Sensorik – Aktorik
5	Roboterkinematik
6	Roboterdynamik
7	Bahnplanung und Regelung
8	Hallenführung in Kleingruppen
9	Grundlagen des Robot Operating System (ROS)
10	ROS - Erweiterte Konzepte und Tools
11	Simulation und Planung
12	Rechnersehen
13	Maschinelles Lernen



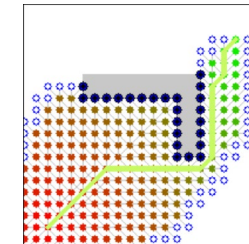
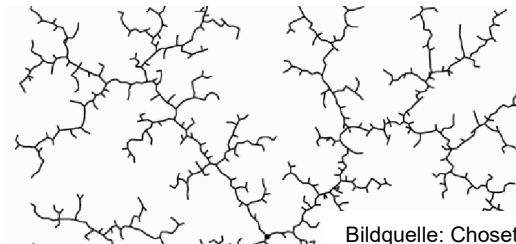
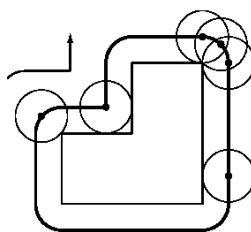
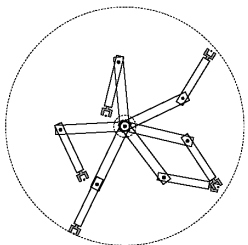
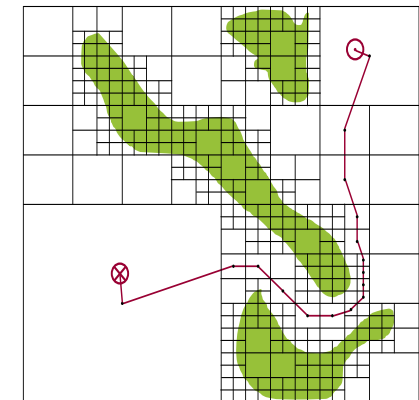
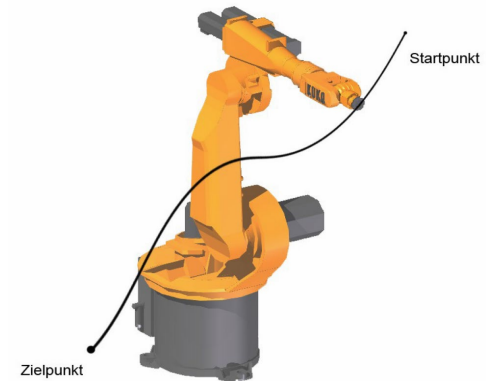
Im Rahmen der Tafelübungen und der zugehörigen Hausübungen werden einfache Programmierbeispiele mit Python sowie die praktische Anwendung des Robot Operating System (ROS) behandelt.

Die Vorlesungseinheit „Bahnplanung und Regelung“ soll eine Übersicht zu den Methoden der Bahnplanung geben sowie Grundlagen der Regelungstechnik vermitteln.

Die Aufgabe der **Bahnplanung** ist es, einen **kollisionsfreien Pfad** zwischen einer Start- und einer Zielpose zu finden.
Durch die **Regelung** soll die Durchführung von definierten Bewegungen hinreichend genau ermöglicht werden.

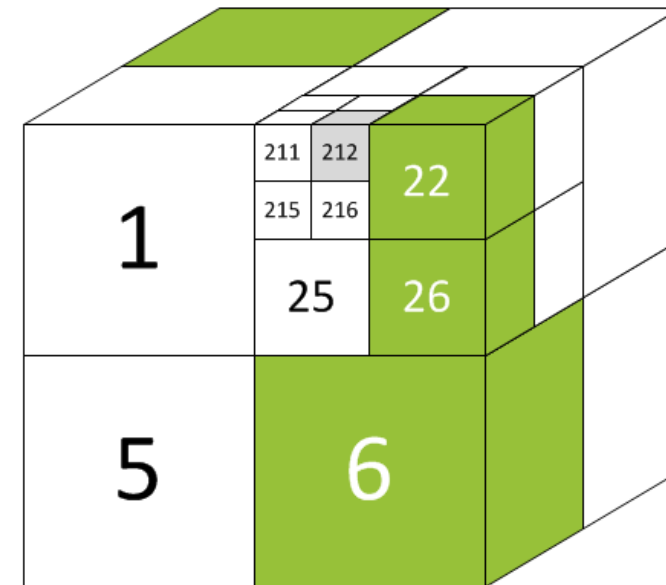
Nach dem Besuch dieser Vorlesungseinheit sollten Sie in der Lage sein:

- die Aufgaben der **Roboterbewegungsplanung** zu beschreiben,
- wichtige **Begriffe der Bahnplanung** zu erläutern,
- **Bahnplanungsverfahren** zu unterscheiden und verschiedene Methoden anzuwenden,
- die Arten der **Bewegungssteuerung** zu unterscheiden,
- den Aufbau und die Wirkungsweise eines **Regelkreises** zu verstehen,
- die Aufgabe der **Regelung in der Robotik** einzuordnen.



**Die Bahnplanung ist ein Teilbereich der Roboterbewegungsplanung;
Steuerung und Regelung ermöglichen die Ausführung von Bewegungen.**

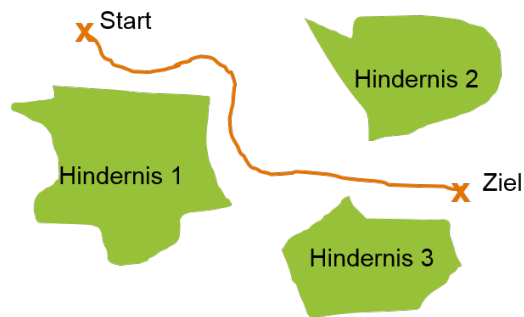
- **Aufgaben der Bewegungsplanung**
- Bahnplanungsproblem und Begrifflichkeiten der Bahnplanung
- Bahnplanungsverfahren
- Bewegungssteuerung
- Grundlagen der Regelungstechnik



Die Roboterbewegungsplanung kann in die vier Teilaufgaben Navigation bzw. Bahnplanung, Abdeckung, Lokalisierung und Kartierung untergliedert werden.

Roboterbewegungsplanung

**(Navigation/)
Bahnplanung**
(engl. „navigation“)



- Planen einer kollisionsfreien Roboterbahn von einem Start- zu einem Zielpunkt
- Beispiele: mobile Roboter, Industrieroboter

Abdeckung
(engl. „coverage“)



- Abdeckung eines kompletten Raums durch eine Roboterbewegung
- Beispiele: Staubsaug-, Rasenmäh- oder Lackierroboter

Lokalisierung
(engl. „localization“)



- Bestimmung einer unbekannten Roboterpose in einer bekannten Umgebungskarte

Kartierung
(engl. „mapping“)

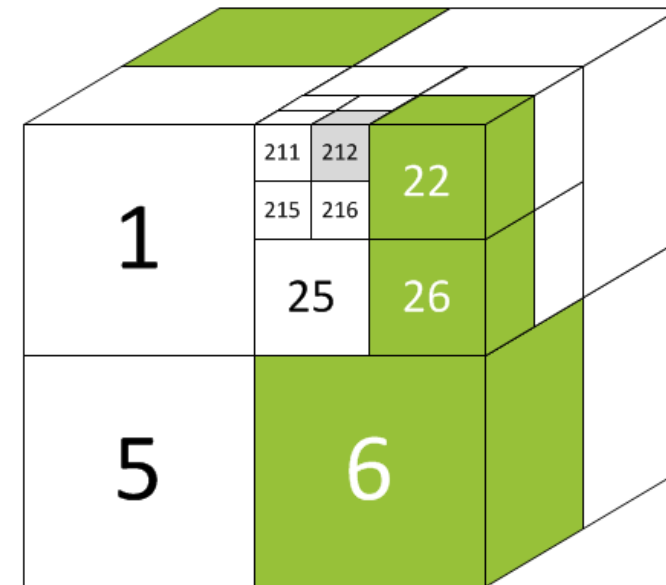


- Generierung einer Karte der unbekannten Umgebung auf Basis von Sensoren (z. B. Laserscanner)

SLAM (Simultaneous Localization and Mapping)

Die Bahnplanung ist ein Teilbereich der Roboterbewegungsplanung; Steuerung und Regelung ermöglichen die Ausführung von Bewegungen.

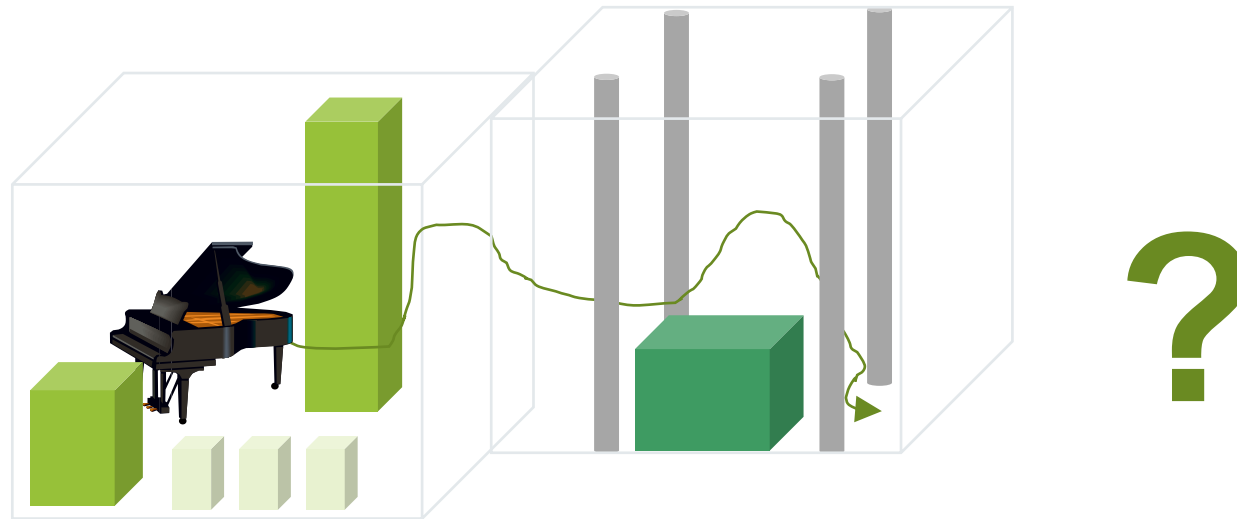
- Aufgaben der Bewegungsplanung
- **Bahnplanungsproblem und Begrifflichkeiten der Bahnplanung**
- Bahnplanungsverfahren
- Bewegungssteuerung
- Grundlagen der Regelungstechnik



Die Bahnplanung basiert auf dem ursprünglichen „Piano mover’s“ und dem verallgemeinerten „Generalized mover’s problem“.

Piano mover’s problem

Wie muss ein Roboter (z. B. Klavier) im Raum bewegt werden, um kollisionsfrei an ein vorgegebenes Ziel zu gelangen? Der Roboter und die Hindernisse bestehen aus starren polyedrischen Körpern.



Generalized mover’s problem

Bei der gleichen Problemstellung besteht der Roboter (z. B. Sechssachs-Knickarmroboter) aus mehreren Polyedern, die über Gelenke frei miteinander verbunden sind.

Bei der Bahnplanung wird ein kollisionsfreier Pfad von der Start- zur Zielkonfiguration des Roboters gesucht.

Problemstellung

Gegeben

- Konfigurationsraum $\mathcal{C}$
- Startkonfiguration $q_{start} \in \mathcal{C}$
- Zielkonfiguration $q_{ziel} \in \mathcal{C}$

Gesucht

- Stetige Trajektorie mit $\tau: [0,1] \rightarrow \mathcal{C}$ mit

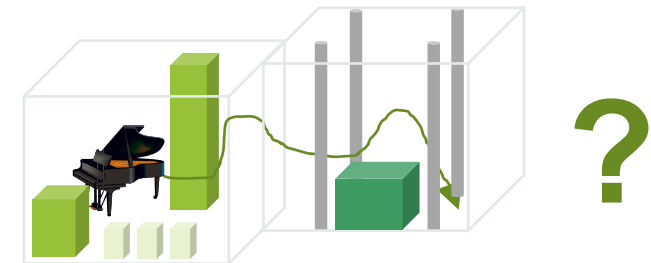
$$\tau(0) = q_{start}$$

$$\tau(1) = q_{ziel}$$

- Unter Berücksichtigung von Gütekriterien, Neben- und Randbedingungen sowie Zwangsbedingungen

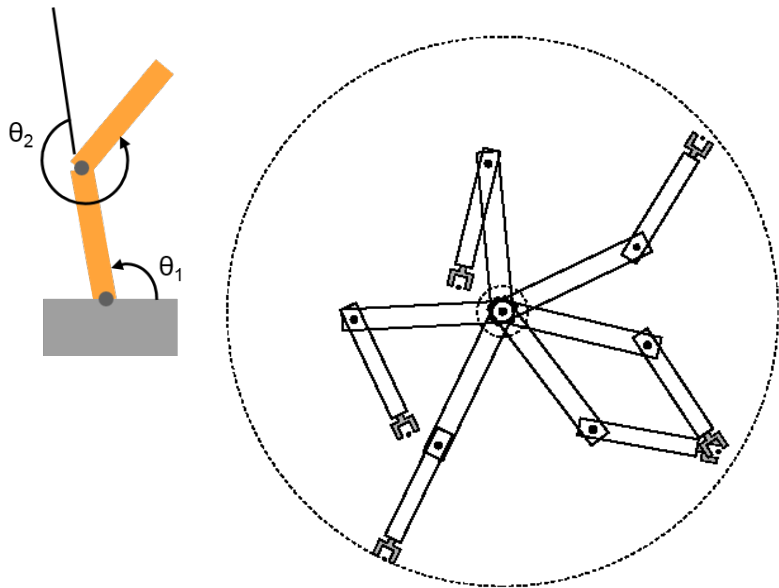
Ziel

Erzeugen einer kollisionsfreien Trajektorie unter Berücksichtigung verschiedener Ziele und Einschränkungen

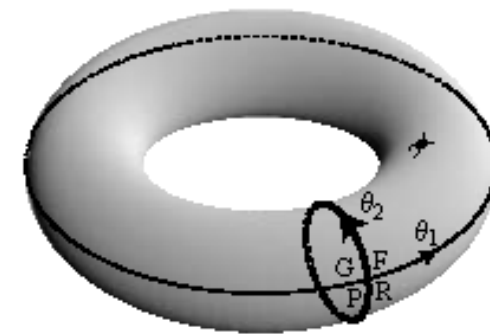
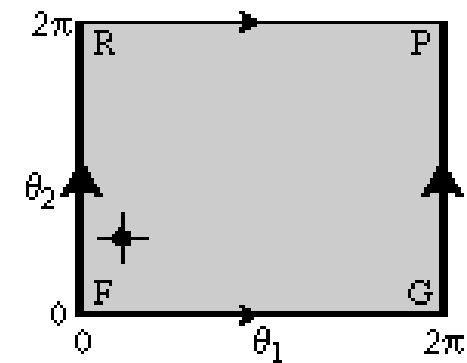


Der Konfigurationsraum eines Roboters in der Ebene mit zwei rotatorischen Gelenken ohne Achsbegrenzungen entspricht einem Torus.

Arbeitsraum eines beispielhaften
zweiachsigen Roboters in der Ebene



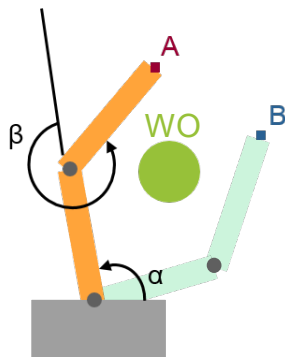
Konfigurationsraum
(zweidimensionaler Torus T2)
ohne Hindernis



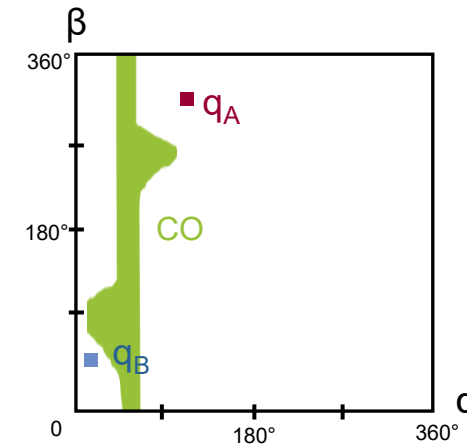
Quelle: Choset

Ein kollisionsfreier Roboterpfad enthält keine Konfiguration, bei der der Roboter mit einem Hindernis kollidiert.

Arbeitsraum eines beispielhaften
zweiachsigen Roboters
mit Hindernis in $\mathbb{R}^2$



Konfigurationsraum
mit Hindernis



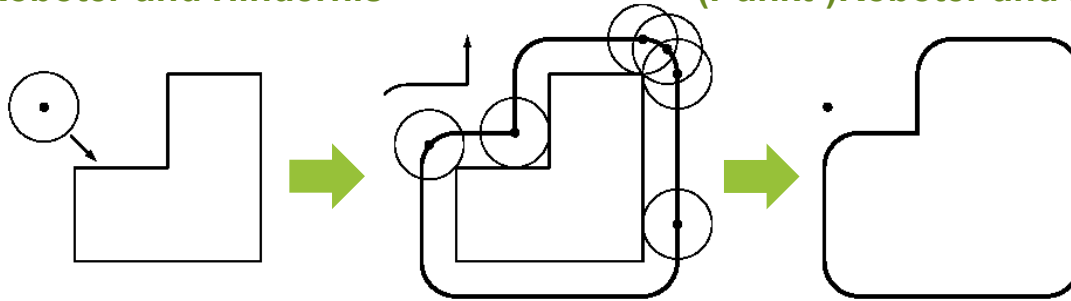
WO: Hindernis im
Arbeitsraum
CO: Hindernis im
Konfigurationsraum

- Ein **kollisionsfreier Pfad** („free path“)
ist eine stetige Abbildung c mit $c(0) = q_{Start}$ und $c(1) = q_{Ziel}$, bei der der Pfad **keine** Konfiguration enthält, bei der es zu einer **Kollision** zwischen Roboter und einem Hindernis kommt.
- Ein **halbfreier Pfad** („semifree path“)
ist eine stetige Abbildung c mit $c(0) = q_{Start}$ und $c(1) = q_{Ziel}$, bei der im Gegensatz zu kollisionsfreien Pfaden eine **Berührung** zwischen dem Hindernis und dem Roboter erlaubt ist.

Die Arbeits- und Konfigurationsräume unterscheiden sich für punkt- und kreisförmige mobile Roboter.

Arbeitsraum: Kreisförmiger Roboter und Hindernis

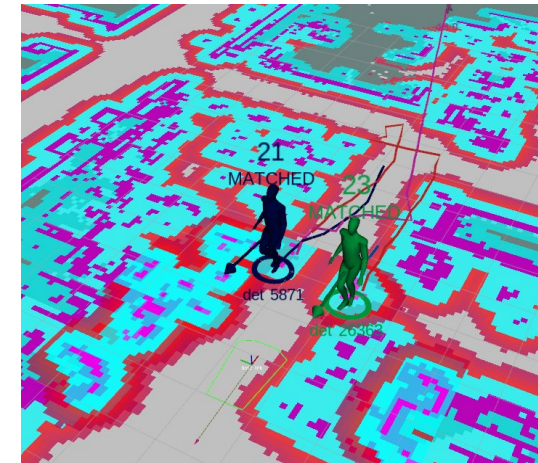
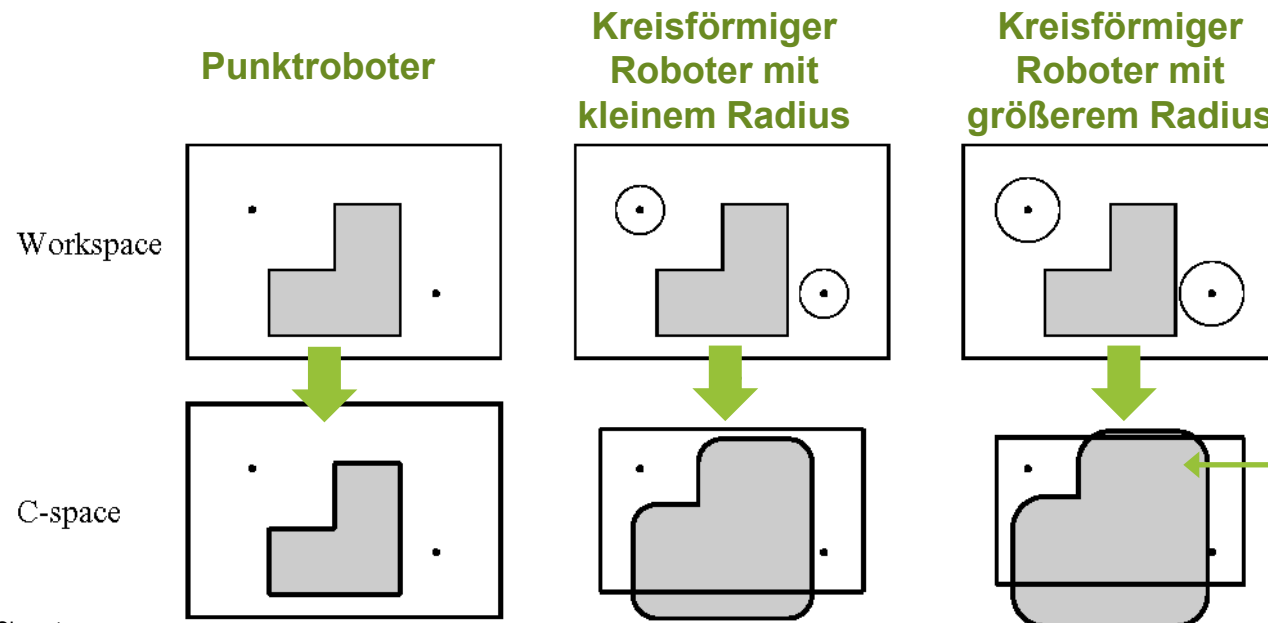
Konfigurationsraum: (Punkt-)Roboter und Hindernis



=> Überführung des Arbeitsraumhindernisses in ein Konfigurationsraumhindernis durch „Vergrößern“ des Polygons

ABER:

Die Transformation ist nicht immer so einfach!



Falls der freie Konfigurationsraum C_{free} nicht mehr verbunden ist, kann keine Lösung für das Bahnplanungsproblem gefunden werden

Quelle: Choset

Wichtige grundlegende Begriffe im Bereich der Bahnplanung sind der Arbeitsraum W , der Konfigurationsraum C und die Konfiguration q .

Arbeitsraum W („workspace“)

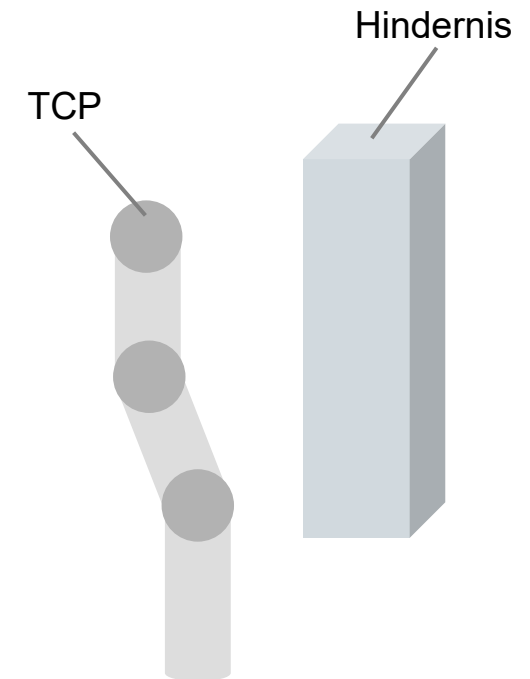
- Entspricht dem euklidischen Umgebungsraum des Roboters in $\mathbb{R}^2$ oder $\mathbb{R}^3$, der durch den Endeffektor (Tool Center Point (TCP)) erreichbar ist
- WO_i : i-tes Hindernis im Arbeitsraum
- $W_{free} = W \setminus (\cup_i WO_i)$: freier Arbeitsraum

Konfigurationsraum C („c-space“, „configuration space“)

- n-dimensionaler Konfigurationsraum
- Entspricht dem Raum aller möglichen Konfigurationen eines Roboters
- Darstellung einer Konfiguration durch einen Punkt im Konfigurationsraum
- C_{free} : alle kollisionsfreien Konfigurationen
- C_{obs} : alle Konfigurationen, die zu einer Kollision führen
- $C = C_{free} \cup C_{obs}$

Konfiguration q

- beschreibt den Zustand eines Roboters als Lage und Orientierung im euklidischen Raum oder als Gelenkwinkelvektor im Gelenkwinkelraum



Weitere Begriffsdefinitionen sind für die Grundlagen der Bahnplanung wichtig.

Arbeitsraumhindernis H

- Raum, welcher von einem Objekt im Arbeitsraum eingenommen wird

Konfigurationsraumhindernis C_H

- Menge aller Punkte des Konfigurationsraumes, welche zu einer Kollision mit dem Hindernis H führen

Hindernisraum C_{obs}

- Menge aller Konfigurationsraumhindernisse
- $C_{obs} = \bigcup_i C_{H_i}$

Freiraum C_{free}

- Menge aller Punkte des Konfigurationsraumes, welche nicht im Hindernisraum liegen
- $C_{free} = \{q \in C \mid q \notin C_{obs}\} = C \setminus C_{obs}$

Starres Objekt (z.B. mobiler Roboter)

- 2D Problem (Position: x,y)
- 3D Problem (Position: x,y ; Rotation: α)
→ Piano Mover's Problem

Mehrkörpersysteme (z.B. Roboterarme, Systeme mit mehreren Robotern)

- Hochdimensionale Problemstellungen

Rand- / Zwangsbedingungen

- Global: limitieren den gültigen Konfigurationsraum
- Lokal: schränken Übergänge zwischen Konfigurationen ein

Umweltmodellierung

- exakt: in Form einer algebraischen Beschreibung
- approximiert: Beschreibung der Umwelt durch Näherung (Boxen, verallgemeinerte Zylinder, Polyeder etc.)

Die Bahnplanung unterscheidet sich im Wesentlichen von der Trajektorienplanung dahingehend, dass die Zeit nicht berücksichtigt wird.

Bahnplanung

Keine Berücksichtigung
der Ausführungszeit

Roboterkinematik als Grundlage

(meist) Planung im Konfigurationsraum

Trajektorienplanung (=vollständige Bewegungsplanung)

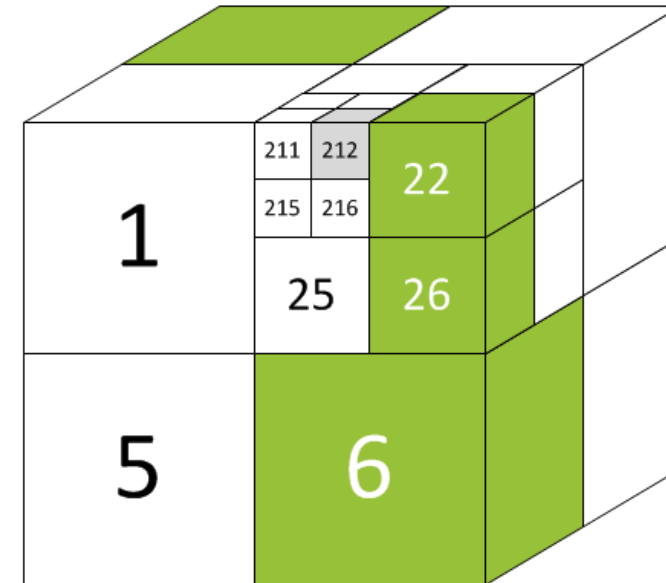
Berücksichtigung der Ausführungszeit,
um eine kollisionsfreie Trajektorie
des Roboters zu erhalten

Roboterdynamik als Grundlage

Planung im Zustandsraum
(engl. „state space“)

Die Bahnplanung ist ein Teilbereich der Roboterbewegungsplanung; Steuerung und Regelung ermöglichen die Ausführung von Bewegungen.

- Aufgaben der Bewegungsplanung
- Bahnplanungsproblem und Begrifflichkeiten der Bahnplanung
- **Bahnplanungsverfahren**
- Bewegungssteuerung
- Grundlagen der Regelungstechnik



Bahnplanungsverfahren können anhand verschiedener Aspekte eingeteilt werden.

Mögliche Einteilungen von Bahnplanungsverfahren

Einteilung nach Betrachtungsraum

- Arbeitsraumbezogene Planungsverfahren
- Konfigurationsraumbezogene Planungsverfahren

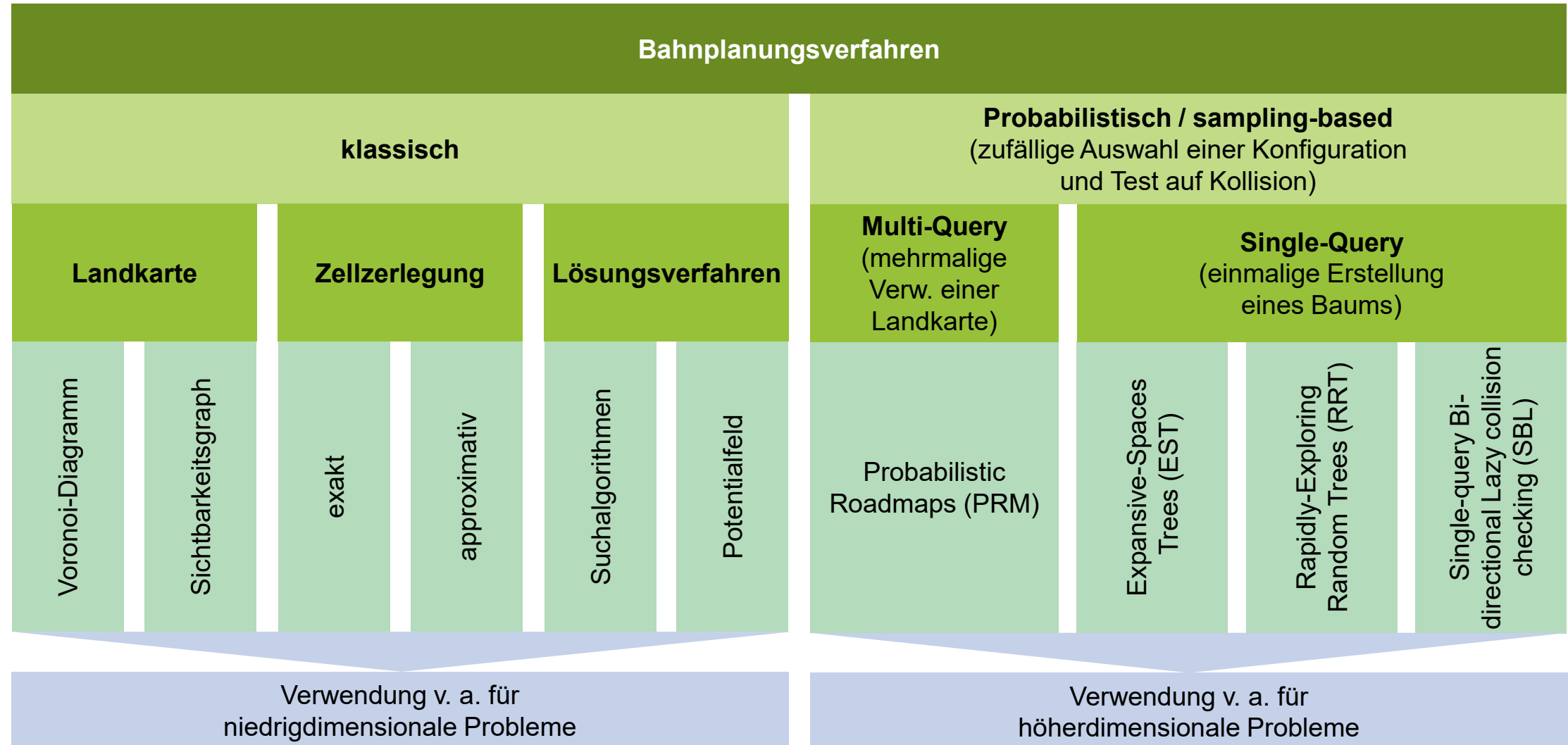
Einteilung bzgl. der Eigenschaften der eingesetzten Algorithmen

- Komplexität (Laufzeit, Speicherverbrauch)
- Vollständigkeit: vollständig, auflösungsvollständig, probabilistisch vollständig

Weitere Einteilungen

- basierend auf dem Zeitpunkt der Durchführung
 - offline: Die Bahn wird im Voraus geplant
 - online: Die Bahn wird schrittweise während der Roboterbewegung erstellt (sensorbasiert)
- Vollständigkeit der Umgebungsabbildung: global $\Leftrightarrow$ lokal
- sensorbasiert $\Leftrightarrow$ Weltmodell basierend

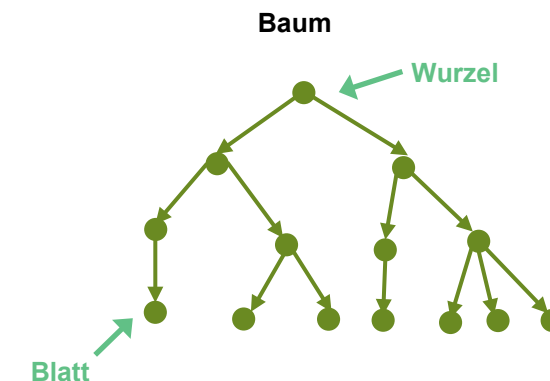
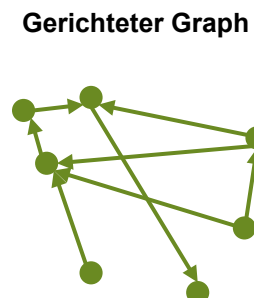
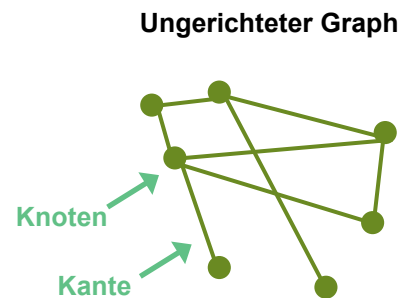
Bahnplanungsverfahren für Roboter gliedern sich in klassische und probabilistische (bzw. sampling-based) Verfahren.



Viele Bahnplanungsverfahren basieren auf der Erstellung eines Graphen, der mittels bekannter Suchalgorithmen zu durchsuchen ist.

Graph

- Ein **Graph** ist eine Sammlung aus Knoten (engl. „node“, „vertex“) und Kanten (engl. „edge“).
- **Gerichtete** Graphen erlauben im Gegensatz zu **ungerichteten** Graphen nur eine der beiden Möglichkeiten der Durchquerung.
- Ein **zyklischer Graph** besitzt mindestens eine Schleife (einen Zyklus).
- Ein **Baum** ist ein spezieller gerichteter Graph, der einen Ausgangsknoten (Wurzel) und keine Schleifen (d. h. es existiert kein Pfad mit dem selben Knoten als Start- und Endknoten) aufweist.
- Ein Knoten, der keine Kinderknoten besitzt, heißt **Blatt**.
- Der **Verzweigungsfaktor b** (engl. „branching factor“) gibt die Anzahl der Nachfolgeknoten eines Knotens an.



Bei der Pfadplanung für mobile Roboter wird ein Weg zwischen der Start- und der Zielkonfiguration gesucht.

Pfadplanung für mobile Roboter

Gegeben

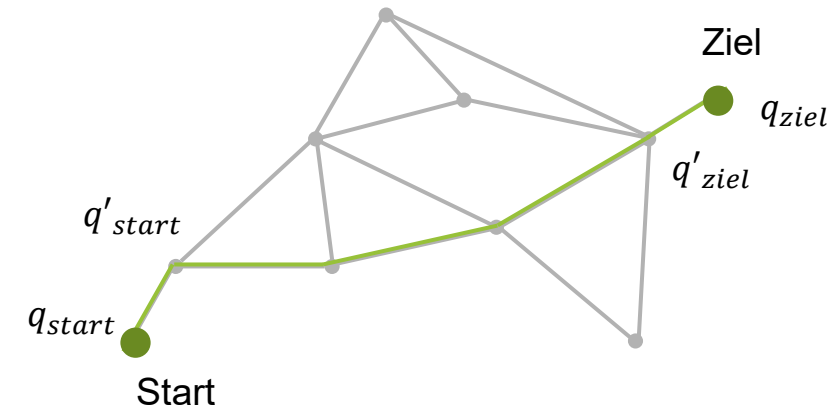
- 2D Weltmodell (z.B. Straßenkarte)
- Start- und Zielpositionen q_{start} und q_{ziel}

Gesucht

- Günstigste Verbindung von q_{start} nach q_{ziel}

Ansatz

- Konstruiere ein Netz W in C_{free}
- Bilde q_{start} und q_{ziel} auf die nächsten Knoten q'_{start} und q'_{ziel} in W ab
- Suche in W einen Weg von q'_{start} nach q'_{ziel}
- Finde einen Weg zwischen q_{start} und q'_{start} sowie zwischen q'_{ziel} und q_{ziel}



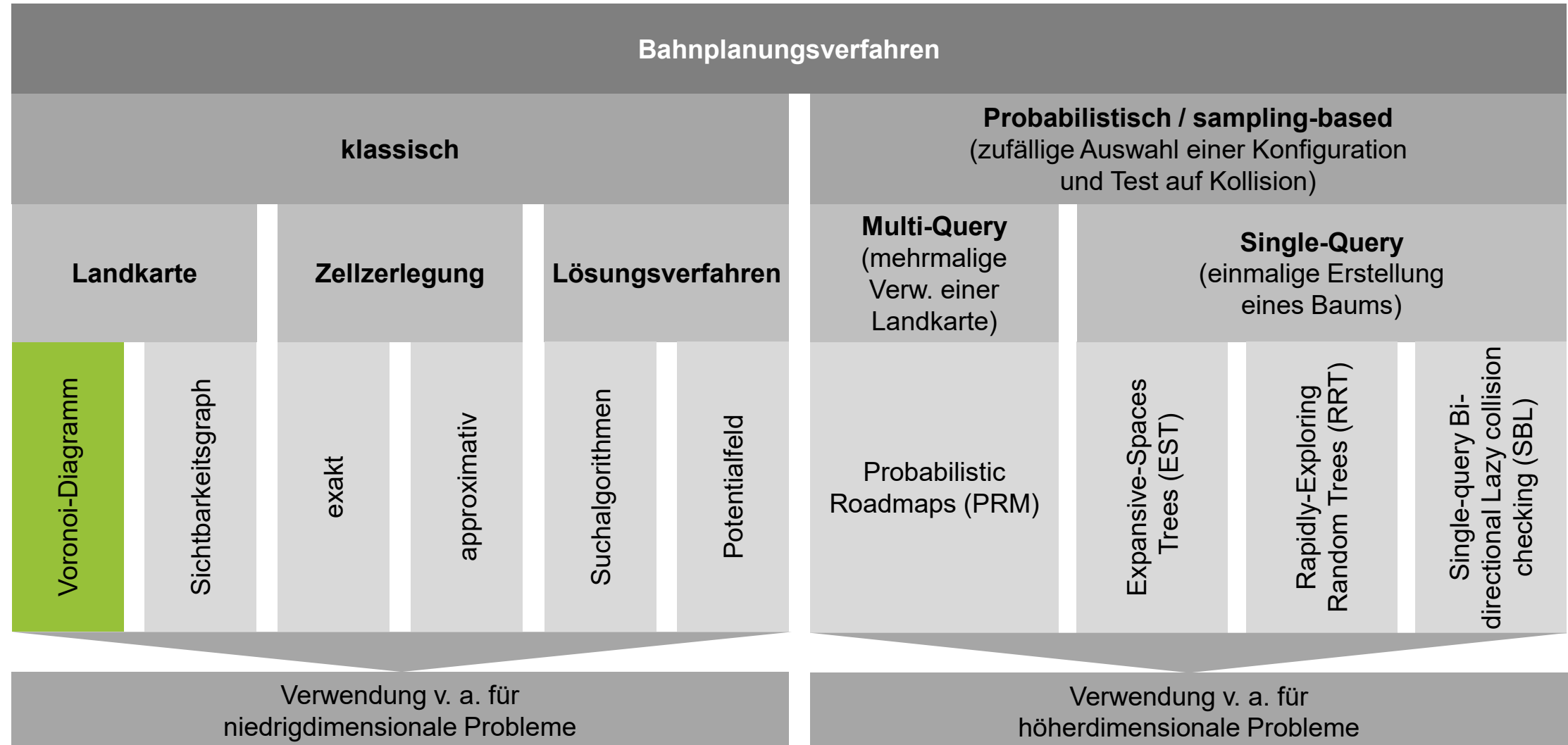
Konstruktion des Wegnetzes

(z.B. Voronoi-Diagramm, Sichtbarkeitsgraph, Zellzerlegung)

Suche im Wegnetz

(z.B. Baumsuche, A*)

Bahnplanungsverfahren für Roboter gliedern sich in klassische und probabilistische (bzw. sampling-based) Verfahren.



Mit dem Voronoi-Diagramm erhält man Landkarten mit maximalem Abstand zu Hindernissen.

Voronoi-Diagramme

- Zerlegung eines Raumes in Regionen basierend auf vorgegebenen Punkten
- Graph bestehend aus Voronoi-Knoten und Voronoi-Kanten

Voronoi-Region

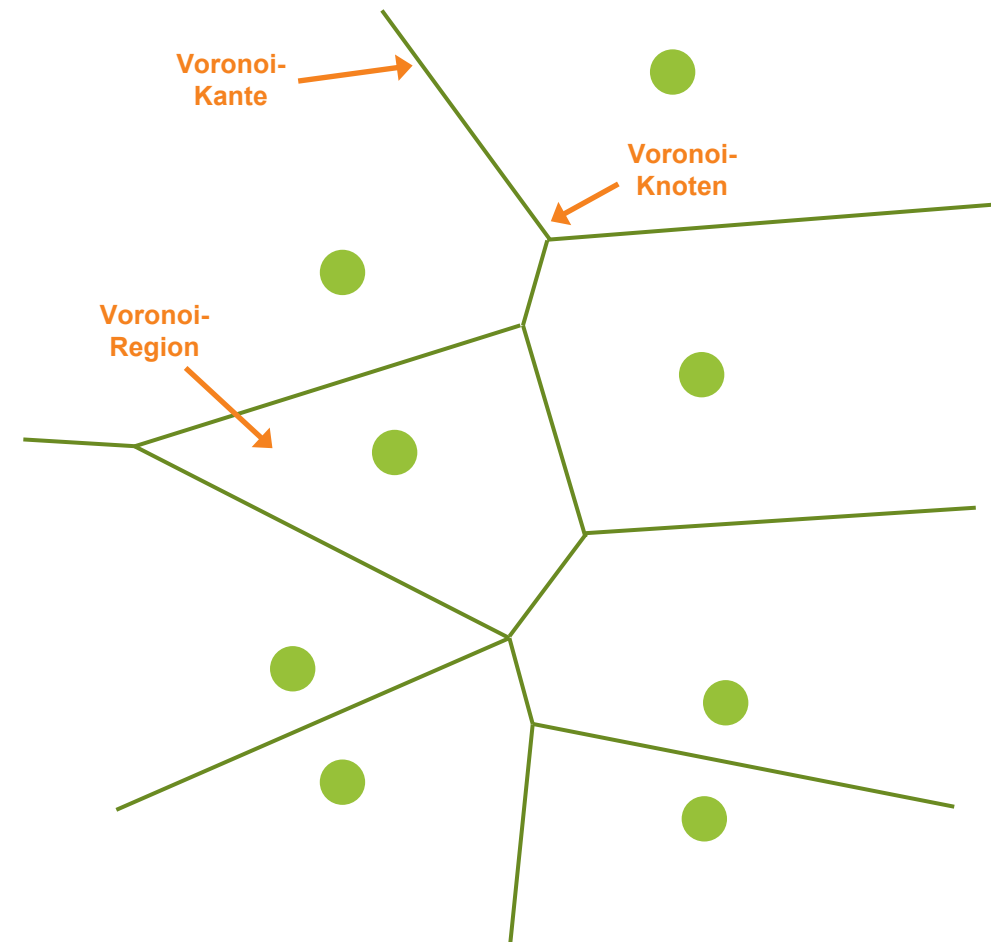
- Menge aller Punkte, deren Abstand zum Zentrum geringer ist als zu allen anderen Zentren

Voronoi-Kante

- Gemeinsamer Rand zweier Voronoi-Regionen
- Alle Punkte auf der Grenze zwischen zwei Regionen besitzen den gleichen Abstand zum eigenen und zum benachbarten Zentrum

Voronoi-Knoten

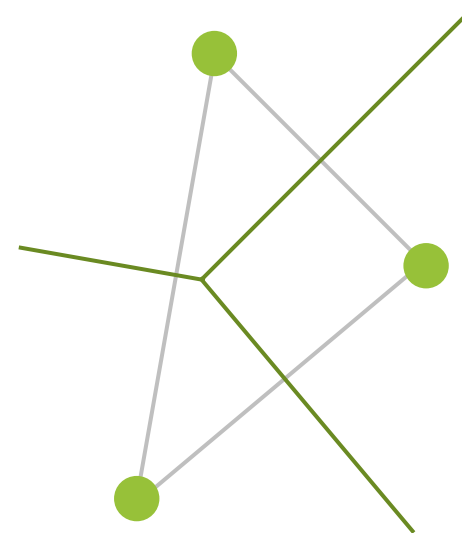
- Drei oder mehr Voronoi-Regionen stoßen zusammen
- Ein Knoten besitzt zu mehr als zwei Zentren minimalen Abstand



Die Konstruktion eines Voronoi-Diagramms erfolgt durch Bilden von Mittelsenkrechten auf den Verbindungslinien der Zentren.

- Gegeben: Punktmenge P
- Rekursive Unterteilung der Punktmenge
→ Reduzierung auf zwei einfache Fälle
- 2 Punkte: Die Mittelsenkrechte bildet das Voronoi-Diagramm
- 3 Punkte: Die Mittelsenkrechten aller Punktpaare werden am gemeinsamen Schnittpunkt abgeschnitten

Fall 2: 3 Punkte

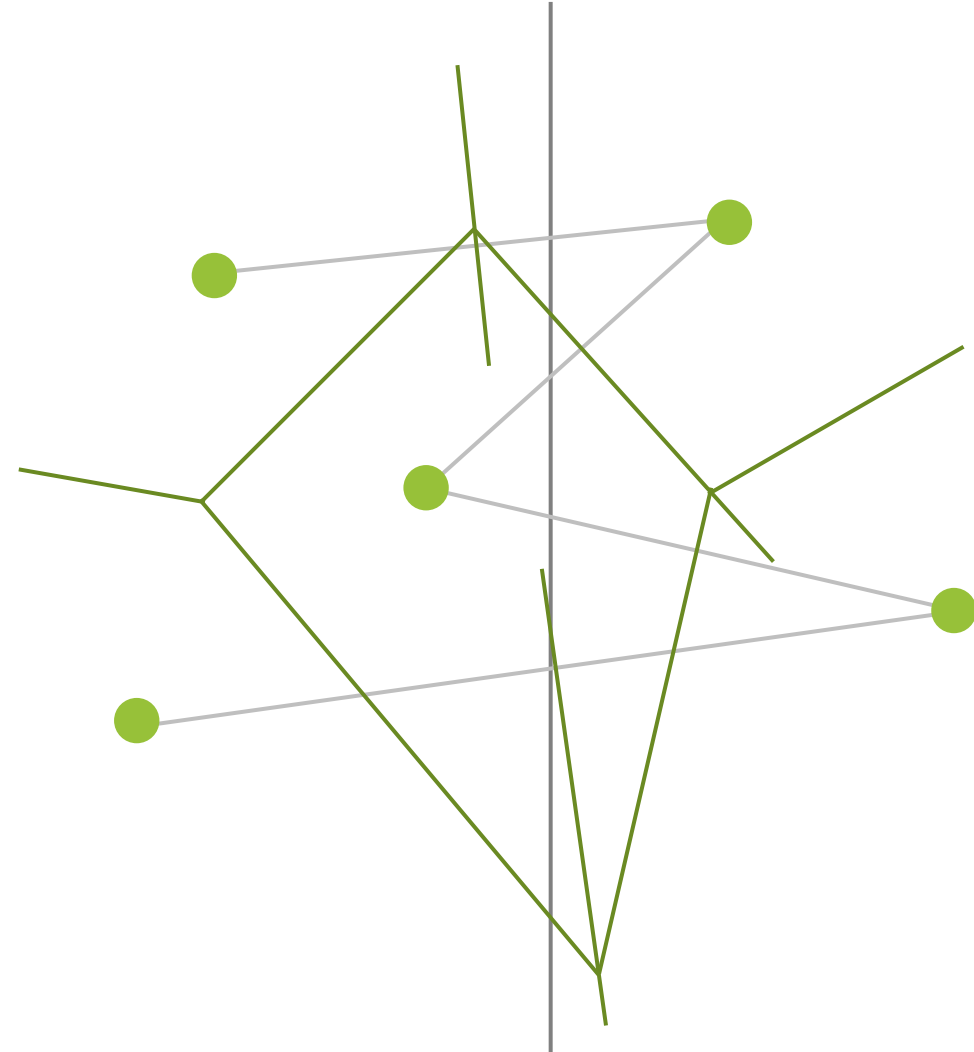


Fall 1: 2 Punkte



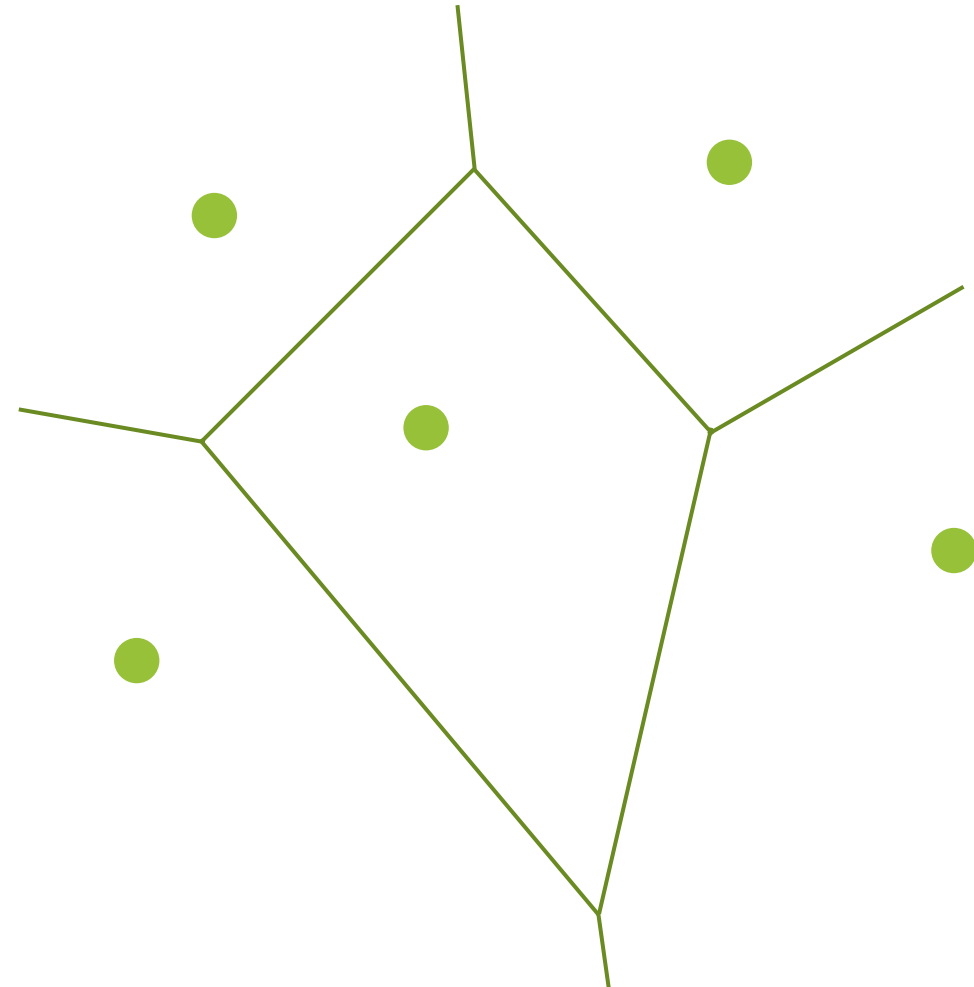
Die Konstruktion eines Voronoi-Diagramms erfolgt durch Bilden von Mittelsenkrechten auf den Verbindungslinien der Zentren.

- Gegeben: Punktmenge P
- Rekursive Unterteilung der Punktmenge
→ Reduzierung auf zwei einfache Fälle
- 2 Punkte: Die Mittelsenkrechte bildet das Voronoi-Diagramm
- 3 Punkte: Die Mittelsenkrechten aller Punktpaare werden am gemeinsamen Schnittpunkt abgeschnitten
- Verschmelzen der beiden Teilmengen durch Verbinden der nächsten Nachbarn entlang der Trennungslinie und Einzeichnen und Abschneiden der neuen Mittelsenkrechten

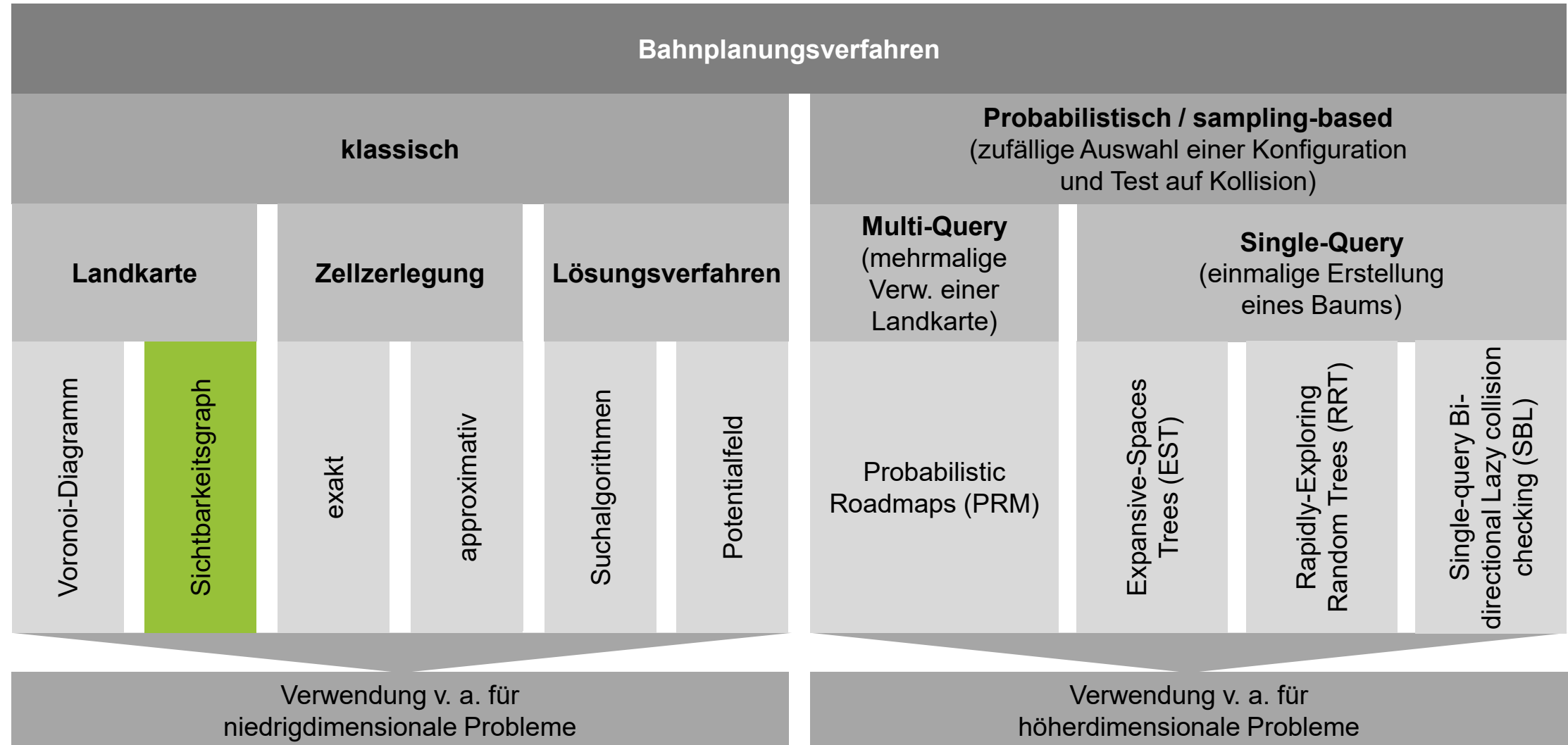


Die Konstruktion eines Voronoi-Diagramms erfolgt durch Bilden von Mittelsenkrechten auf den Verbindungslinien der Zentren.

- Gegeben: Punktmenge P
- Rekursive Unterteilung der Punktmenge
→ Reduzierung auf zwei einfache Fälle
- 2 Punkte: Die Mittelsenkrechte bildet das Voronoi-Diagramm
- 3 Punkte: Die Mittelsenkrechten aller Punktpaare werden am gemeinsamen Schnittpunkt abgeschnitten
- Verschmelzen der beiden Teilmengen durch Verbinden der nächsten Nachbarn entlang der Trennungslinie und Einzeichnen und Abschneiden der neuen Mittelsenkrechten



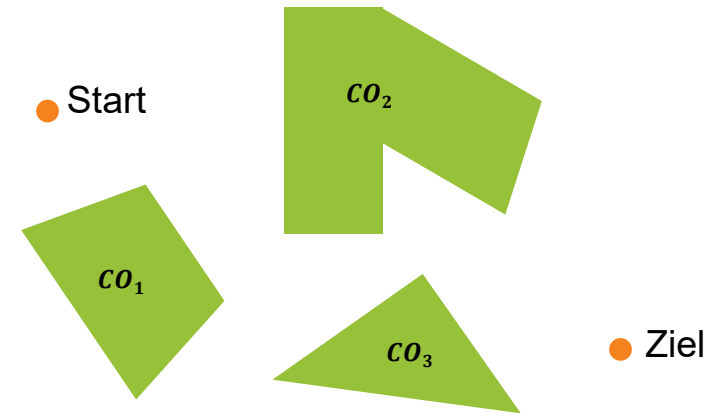
Bahnplanungsverfahren für Roboter gliedern sich in klassische und probabilistische (bzw. sampling-based) Verfahren.



Sichtbarkeitsgraphen werden i. d. R. nur für zweidimensionale Planungsprobleme eingesetzt, wie im Beispiel für einen Punktroboter.

Beispielhafte Erstellung eines Sichtbarkeitsgraphen

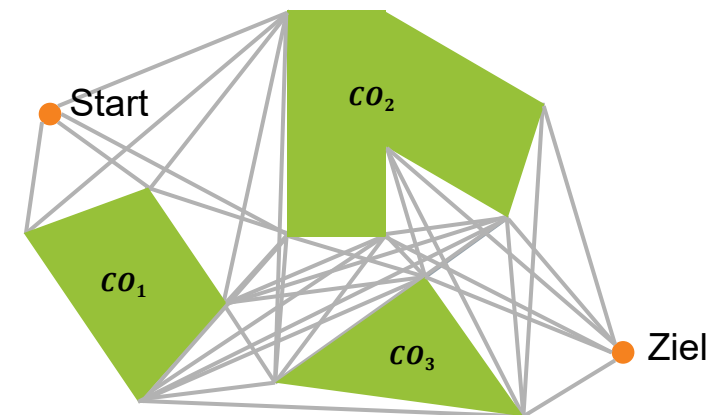
- 2D-Planungsraum
- Punktroboter
- Polygonale Hindernisse



Sichtbarkeitsgraphen werden i. d. R. nur für zweidimensionale Planungsprobleme eingesetzt, wie im Beispiel für einen Punktroboter.

Vorgehen

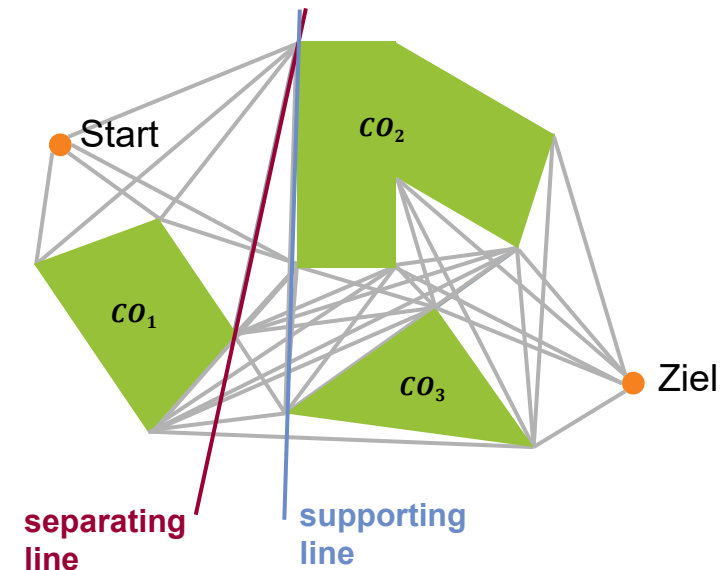
- Sichtbarkeitsgraph G im Konfigurationsraum erstellen mit Knoten, die entweder ein Start- oder Zielpunkt oder ein Eckpunkt des Hindernisses O_i sind
- Verbinden der Knoten, falls die Verbindungsstrecke vollständig in C_{free} liegt
- Wenn der Roboter kein Punkt ist, dann entsprechende Vergrößerung der Hindernisse zu Konfigurationsraumhindernissen CO_i



Sichtbarkeitsgraphen werden i. d. R. nur für zweidimensionale Planungsprobleme eingesetzt, wie im Beispiel für einen Punktroboter.

Vorgehen

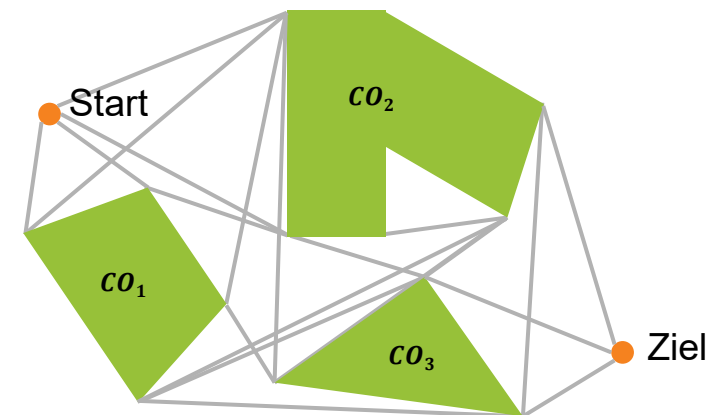
- Sichtbarkeitsgraph G im Konfigurationsraum erstellen mit Knoten, die entweder ein Start- oder Zielpunkt oder ein Eckpunkt des Hindernisses O_i sind
- Verbinden der Knoten, falls die Verbindungsstrecke vollständig in C_{free} liegt
- Wenn der Roboter kein Punkt ist, dann entsprechende Vergrößerung der Hindernisse zu sog. Konfigurationsraumhindernissen CO_i
- (Vereinfachung des Sichtbarkeitsgraphen durch Entfernung aller Kanten, die keine „**supporting lines**“ (Tangente an zwei Hindernissen, sodass beide Hindernisse auf der selben Seite der Tangente liegen) oder „**separating lines**“ (Tangente an zwei Hindernissen, sodass beide Hindernisse auf der jeweils anderen Seite der Tangente liegen) darstellen)



Sichtbarkeitsgraphen werden i. d. R. nur für zweidimensionale Planungsprobleme eingesetzt, wie im Beispiel für einen Punktroboter.

Vorgehen

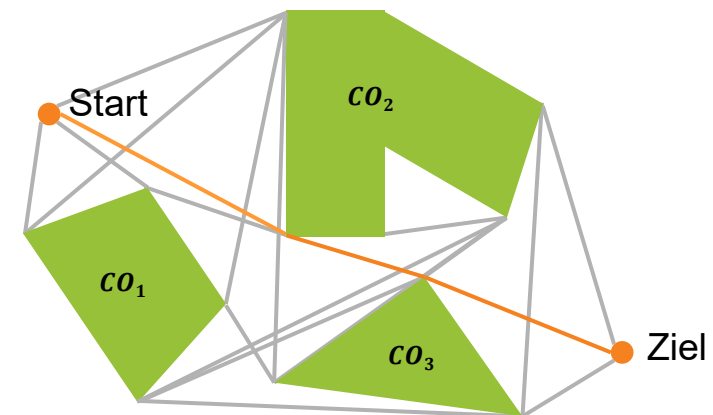
- Sichtbarkeitsgraph G im Konfigurationsraum erstellen mit Knoten, die entweder ein Start- oder Zielpunkt oder ein Eckpunkt des Hindernisses O_i sind
- Verbinden der Knoten, falls die Verbindungsstrecke vollständig in C_{free} liegt
- Wenn der Roboter kein Punkt ist, dann entsprechende Vergrößerung der Hindernisse zu sog. Konfigurationsraumhindernissen CO_i
- (Vereinfachung des Sichtbarkeitsgraphen durch Entfernung aller Kanten, die keine „**supporting lines**“ (Tangente an zwei Hindernissen, sodass beide Hindernisse auf der selben Seite der Tangente liegen) oder „**separating lines**“ (Tangente an zwei Hindernissen, sodass beide Hindernisse auf der jeweils anderen Seite der Tangente liegen) darstellen)



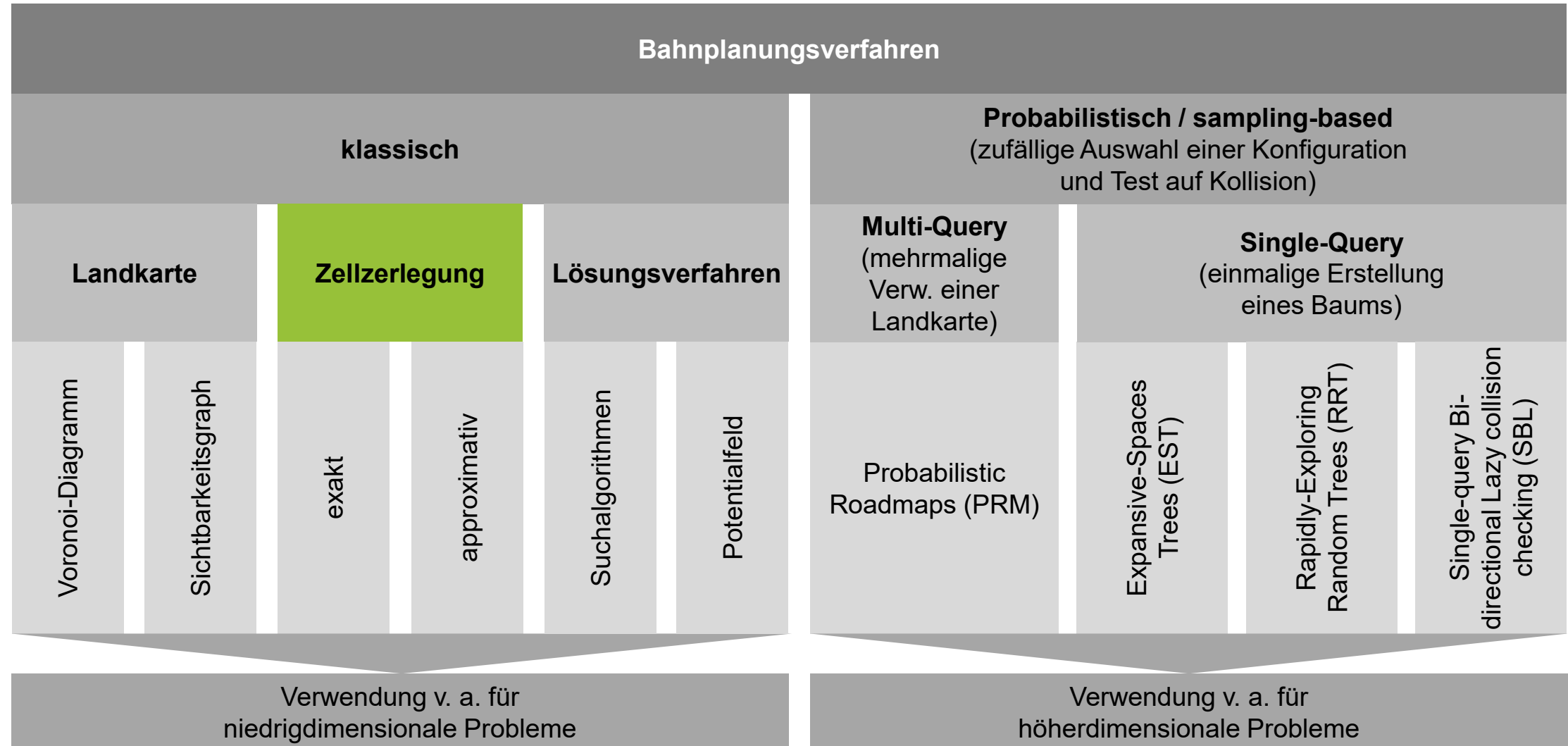
Sichtbarkeitsgraphen werden i. d. R. nur für zweidimensionale Planungsprobleme eingesetzt, wie im Beispiel für einen Punktroboter.

Vorgehen

- Sichtbarkeitsgraph G im Konfigurationsraum erstellen mit Knoten, die entweder ein Start- oder Zielpunkt oder ein Eckpunkt des Hindernisses O_i sind
- Verbinden der Knoten, falls die Verbindungsstrecke vollständig in C_{free} liegt
- Wenn der Roboter kein Punkt ist, dann entsprechende Vergrößerung der Hindernisse zu sog. Konfigurationsraumhindernissen CO_i
- (Vereinfachung des Sichtbarkeitsgraphen durch Entfernung aller Kanten, die keine „**supporting lines**“ (Tangente an zwei Hindernissen, sodass beide Hindernisse auf der selben Seite der Tangente liegen) oder „**separating lines**“ (Tangente an zwei Hindernissen, sodass beide Hindernisse auf der jeweils anderen Seite der Tangente liegen) darstellen)
- Sichtbarkeitsgraph wird nach dem kürzesten Weg zwischen Start- und Zielknoten mit einem Graphsuchverfahren (z. B. A*-Algorithmus) durchsucht



Bahnplanungsverfahren für Roboter gliedern sich in klassische und probabilistische (bzw. sampling-based) Verfahren.



Weitere klassische Bahnplanungsverfahren stellen die exakte und die approximative Zellzerlegung dar.

Vorgehensweise Zellzerlegung

- Aufteilung des freien Raums (meist des Konfigurationsraums) in nicht überlappende Regionen, sog. **Zellen**
- Aufstellen des **Zusammenhangsgraphen**, der die Nachbarschaft der Zellen repräsentiert

Knoten: Zellen des freien Raums
Kanten: Verbindungen zwischen benachbarten Zellen
- Suche des optimalen Wegs in dem Graphen

Arten der Zellzerlegung

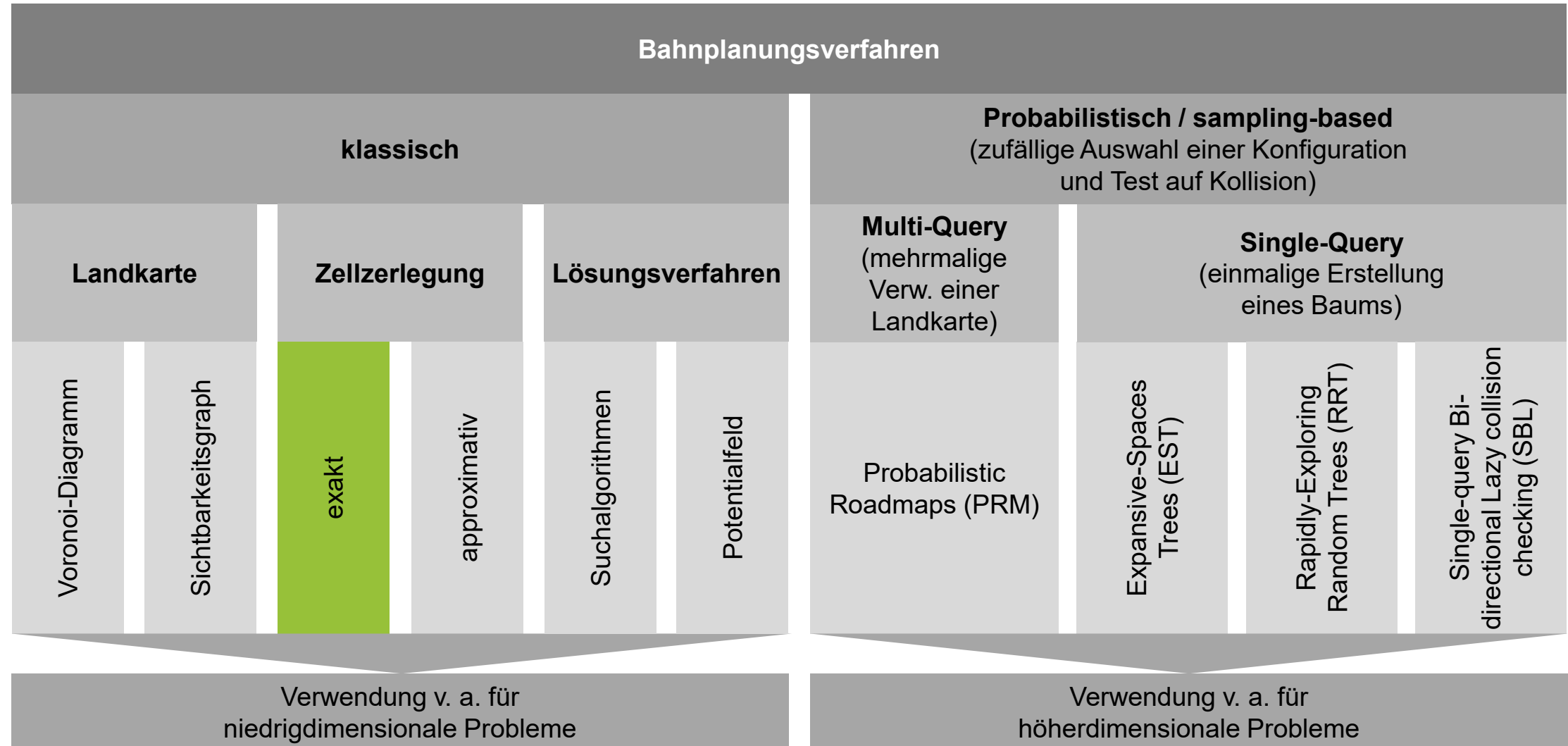
Exakte Zellzerlegung

- Zerlegung des freien Raums in Zellen, sodass dieser komplett abgedeckt wird
- Die Vereinigungsmenge aller Zellen ist der freie Konfigurationsraum

Approximative Zellzerlegung

- Verwendung von einfachen, vordefinierten Zellformen z. B. (unterschiedlich große) Rechtecke
- Vollständige Abdeckung nicht zwingend
- Approximationsfehler abhängig von der Zellengröße

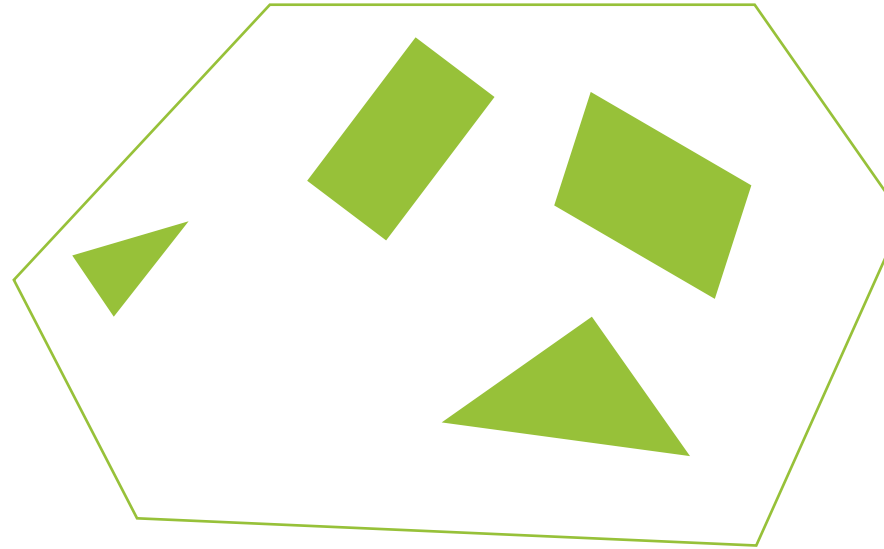
Bahnplanungsverfahren für Roboter gliedern sich in klassische und probabilistische (bzw. sampling-based) Verfahren.



Das bekannteste Verfahren der exakten Zellzerlegung ist die trapezförmige Zerlegung, bei der die Zellen aus Trapezen (oder Dreiecken) bestehen.

Beispielhafte Trapezzerlegung

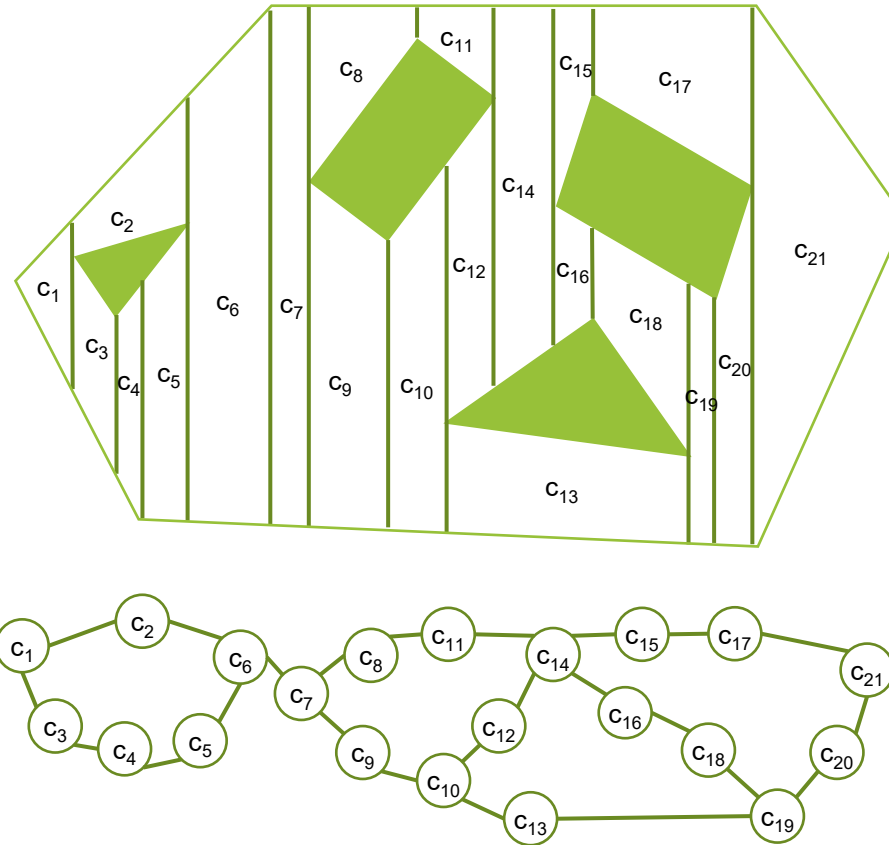
- Zweidimensionaler (x, y) Konfigurationsraum
- Polygonale Hindernisse
- Der freie Raum wird durch ein Polygon begrenzt



Das bekannteste Verfahren der exakten Zellzerlegung ist die trapezförmige Zerlegung, bei der die Zellen aus Trapezen (oder Dreiecken) bestehen.

Vorgehen

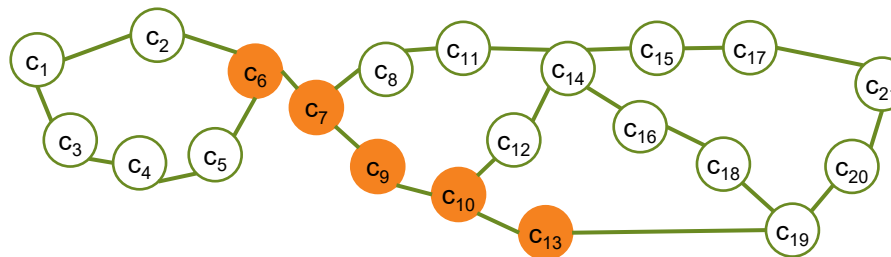
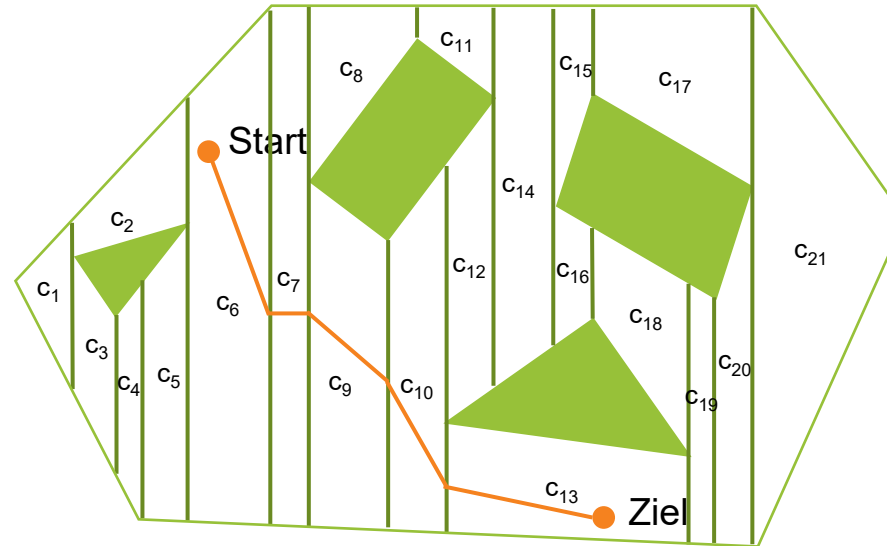
- Einzeichnen vertikaler Linien an allen Eckpunkten der Hindernisse bis zu einem anderen Polygon. Dadurch ergeben sich trapezförmige bzw. zu Dreiecken degenerierte Zellen.
- Aufstellen des Zusammenhangsgraphen



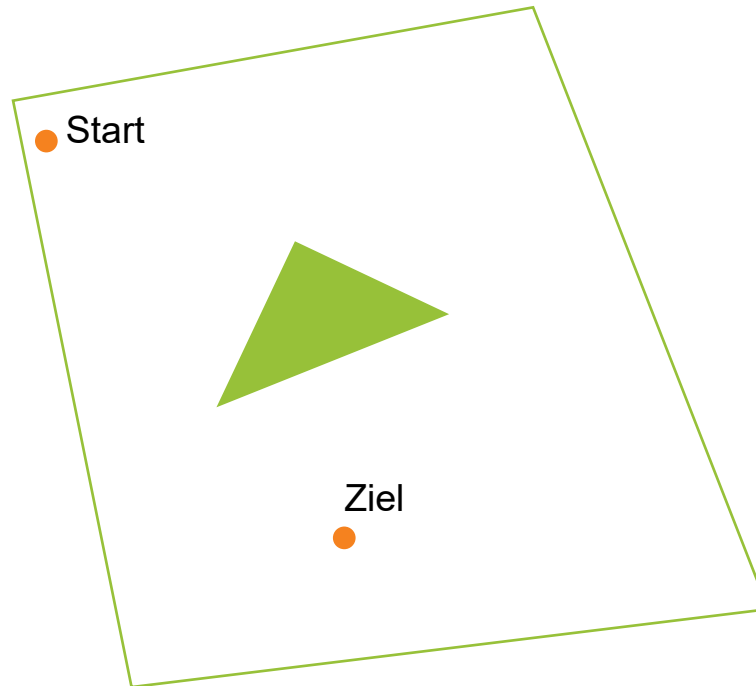
Das bekannteste Verfahren der exakten Zellzerlegung ist die trapezförmige Zerlegung, bei der die Zellen aus Trapezen (oder Dreiecken) bestehen.

Vorgehen

- Einzeichnen vertikaler Linien an allen Eckpunkten der Hindernisse bis zu einem anderen Polygon. Dadurch ergeben sich trapezförmige bzw. zu Dreiecken degenerierte Zellen.
- Aufstellen des Zusammenhangsgraphen
- Suche nach dem kürzesten Weg vom Start- zum Zielknoten im Zusammenhangsgraph



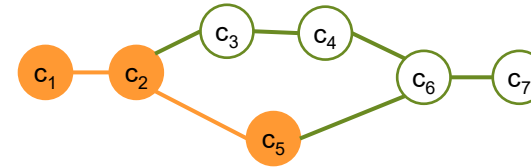
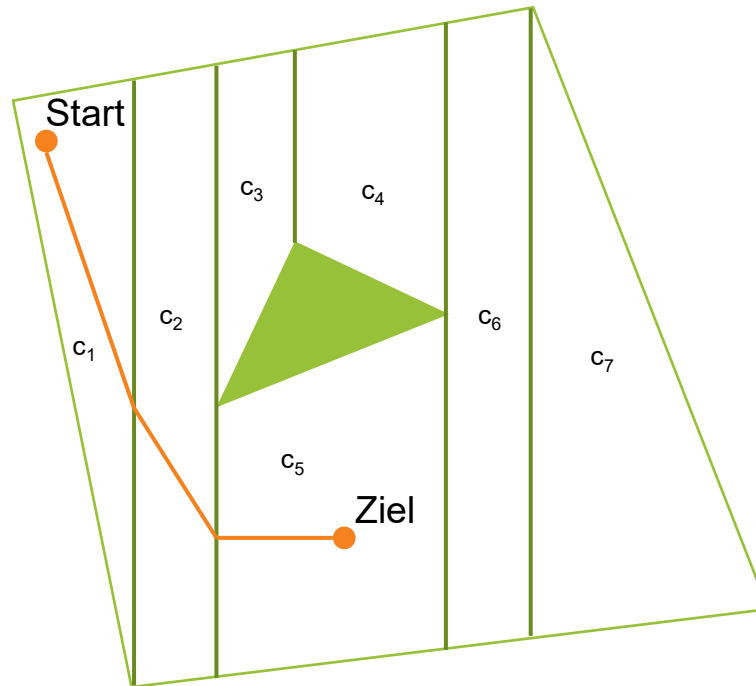
Übung: Das bekannteste Verfahren der exakten Zellzerlegung ist die trapezförmige Zerlegung, bei der die Zellen aus Trapezen (oder Dreiecken) bestehen.



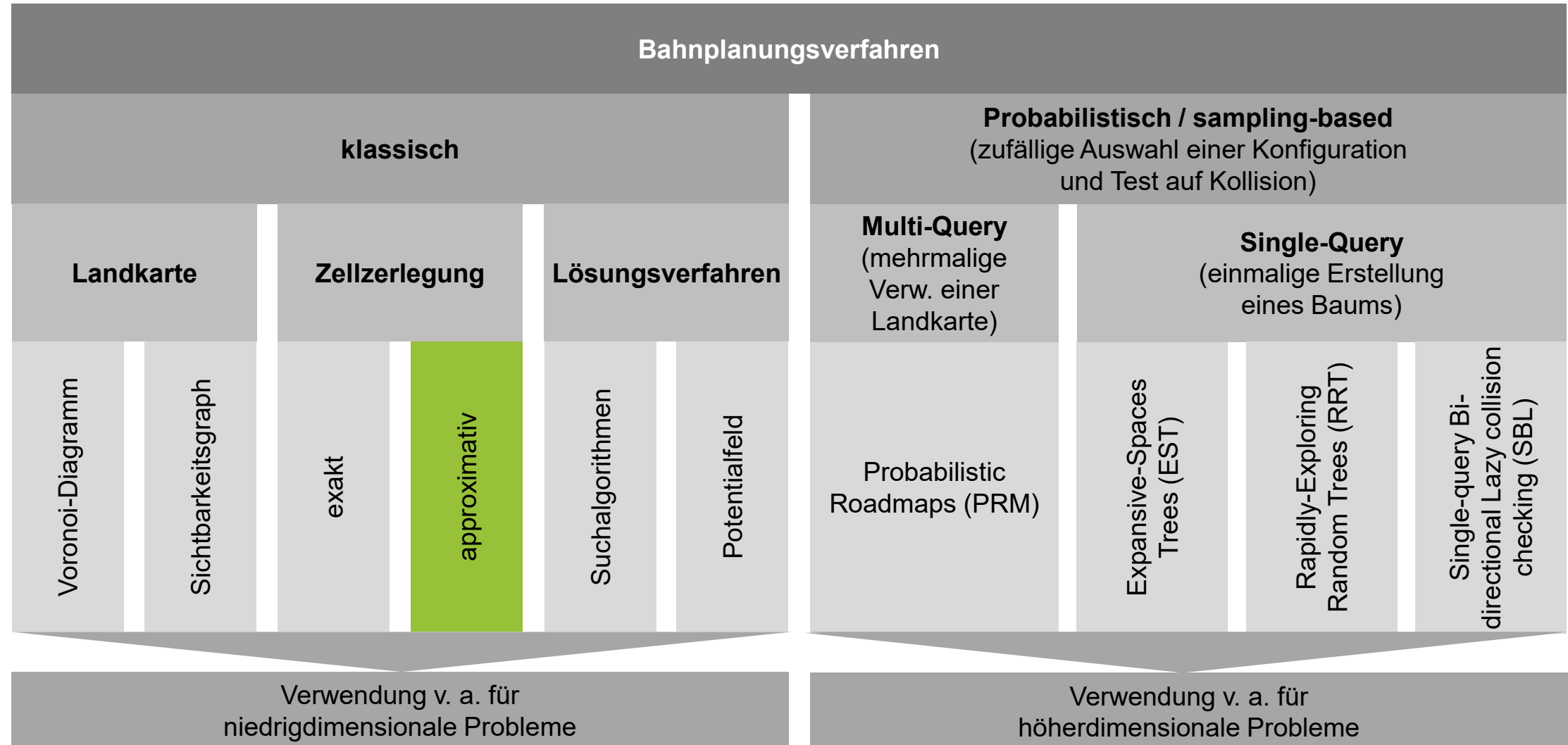
Übungsaufgabe 1)

Führen Sie eine **Trapezzerlegung** für einen Punktroboter im zweidimensionalen Konfigurationsraum durch und stellen Sie anschließend den **Zusammenhangsgraphen** auf.

Übung: Das bekannteste Verfahren der exakten Zellzerlegung ist die trapezförmige Zerlegung, bei der die Zellen aus Trapezen (oder Dreiecken) bestehen.



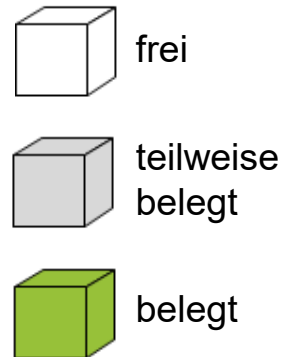
Bahnplanungsverfahren für Roboter gliedern sich in klassische und probabilistische (bzw. sampling-based) Verfahren.



Bei der approximativen Zellzerlegung bieten sich neben Gitterverfahren vor allem hierarchische Zerlegungen, wie beim Quadtree- oder Oktree-Verfahren, an.

Zustände der Zellen bei einer hierarchischen Zerlegung

- frei von Hindernissen
- teilweise mit einem Hindernis belegt
- komplett mit einem Hindernis belegt

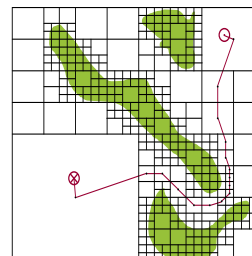


Vorgehensweise

- Zerlegung des Raums ausgehend von großen Zellen in immer kleinere Zellen
 - Bis nur noch freie oder komplett belegte Zellen übrig sind oder
 - Ein vordefinierter Zellzerlegungsgrad erreicht wird
- Es entsteht ein 2^m -Baum (m =Dimension)
- Aufstellen des Zusammenhangsgraphen und Durchsuchen nach Pfad zwischen Start- und Zielknoten

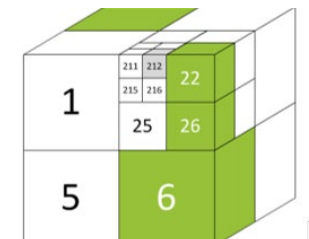
Quadtree-Verfahren

- Zerlegung eines zweidimensionalen Raums in **Quadrate**
- Entstehung eines 2^2 -Baums
- Gemischte Zellen, die nicht ein Blatt des Baums darstellen, besitzen 2^2 gleichförmige (quadratische) Kinderzellen

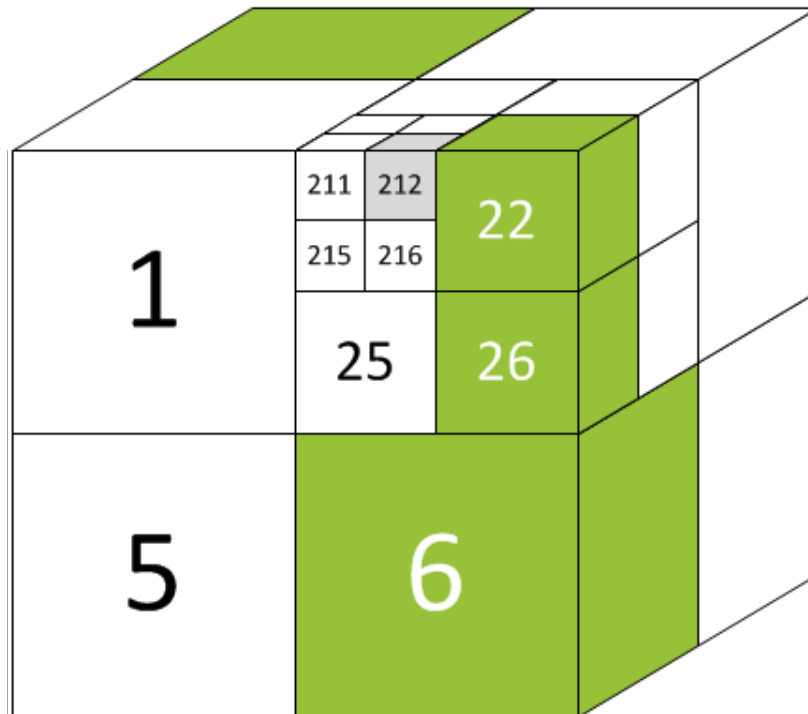
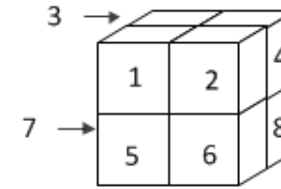
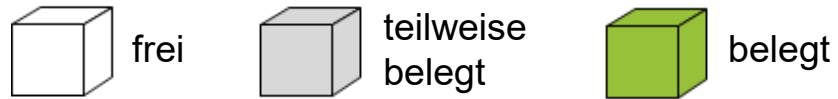


Oktree-Verfahren

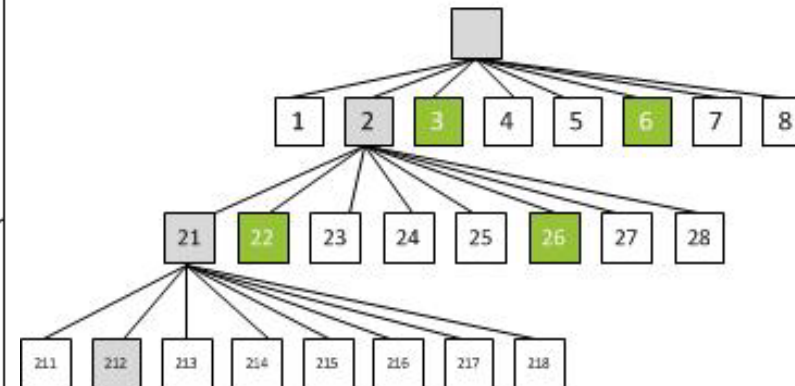
- Zerlegung eines dreidimensionalen Raums in **Würfel**
- Entstehung eines 2^3 -Baums
- Gemischte Zellen, die nicht ein Blatt des Baums darstellen, besitzen 2^3 gleichförmige (kubische) Kinderzellen



Bei der Oktree-Zellzerlegung werden teilweise belegte Zellen in acht gleichförmige, kubische Kinderzellen aufgeteilt.



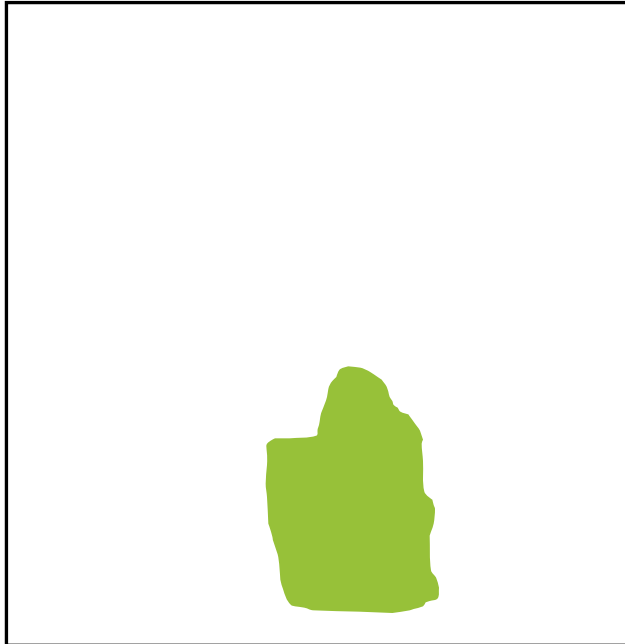
Beispielhafte Erstellung eines Oktrees



Übung: Beim Quadtree-Verfahren wird durch hierarchische Zerlegung ein zweidimensionaler Raum in Quadrate unterteilt.

Nummerierung

1	2
3	4



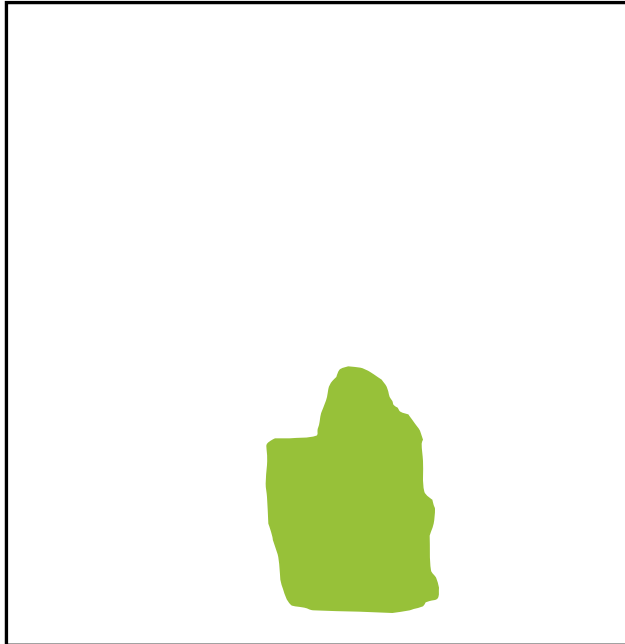
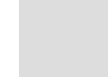
Übungsaufgabe 2)

Führen Sie eine **Quadtree-Zerlegung** mit einem **Zerlegungsgrad von drei** für die gegebene Hindernissituation im zweidimensionalen Raum durch.

Übung: Beim Quadtree-Verfahren wird durch hierarchische Zerlegung ein zweidimensionaler Raum in Quadrate unterteilt.

Nummerierung

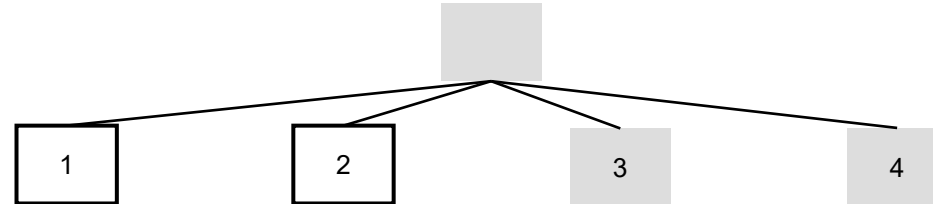
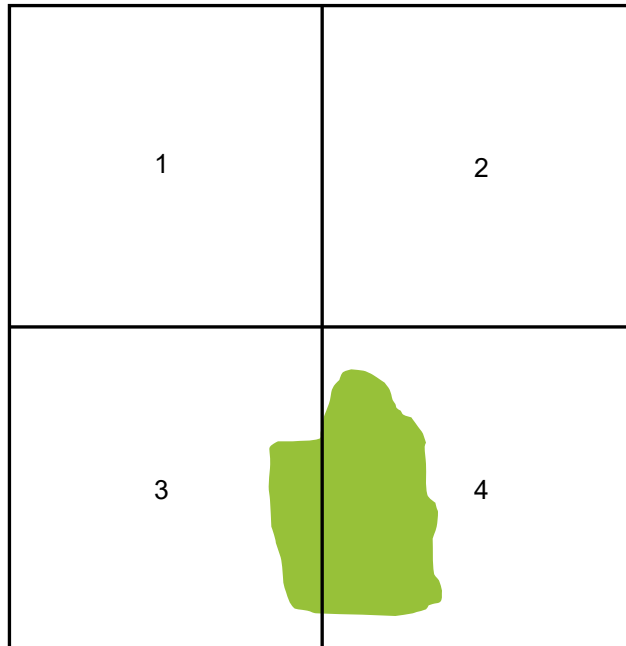
1	2
3	4



Übung: Beim Quadtree-Verfahren wird durch hierarchische Zerlegung ein zweidimensionaler Raum in Quadrate unterteilt.

Nummerierung

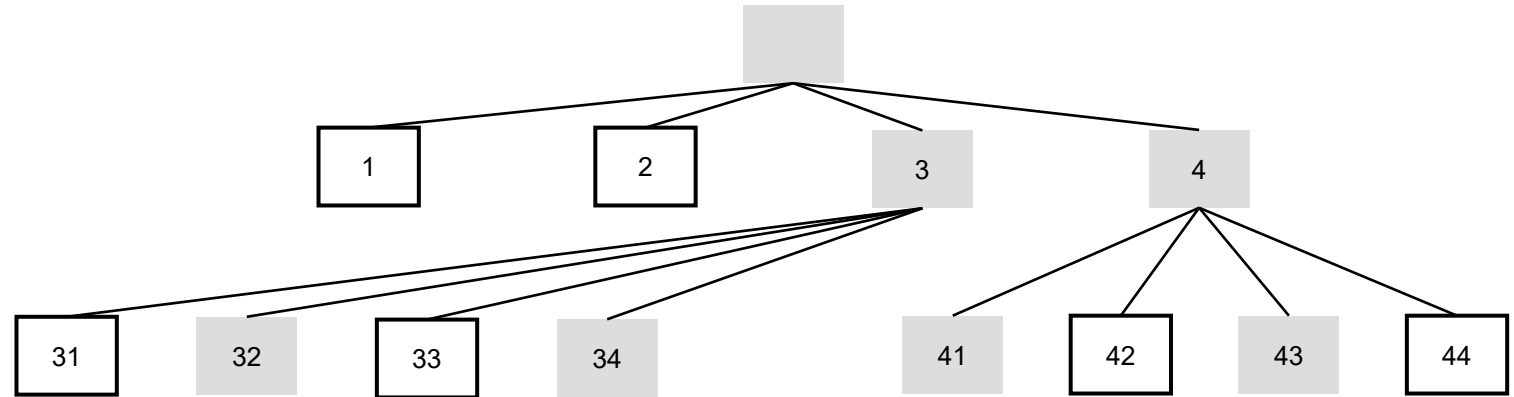
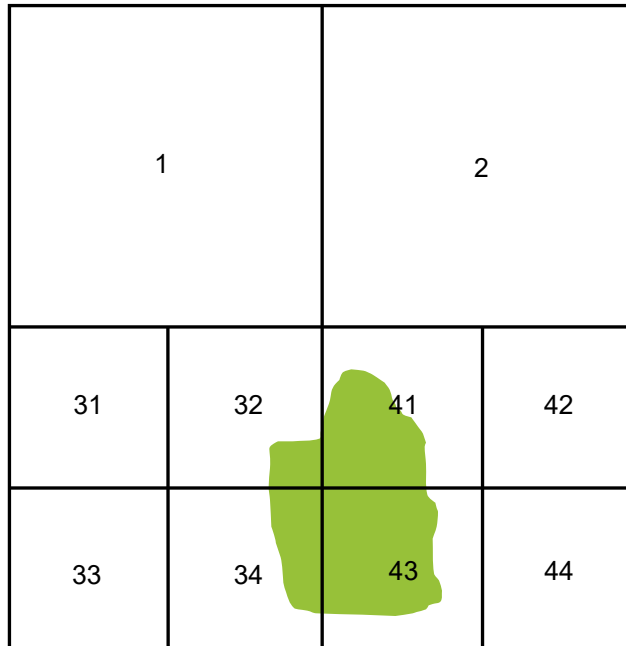
1	2
3	4



Übung: Beim Quadtree-Verfahren wird durch hierarchische Zerlegung ein zweidimensionaler Raum in Quadrate unterteilt.

Nummerierung

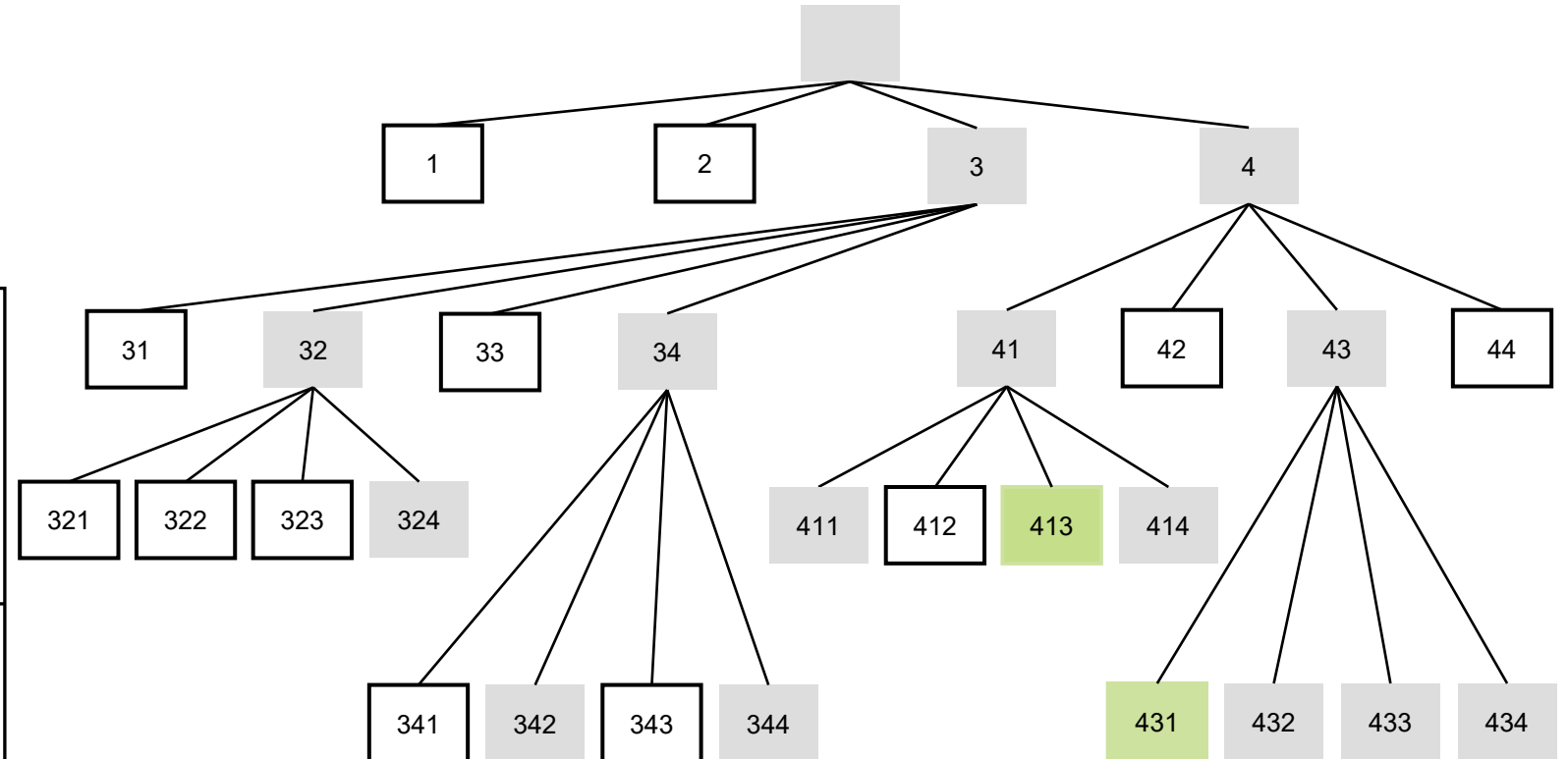
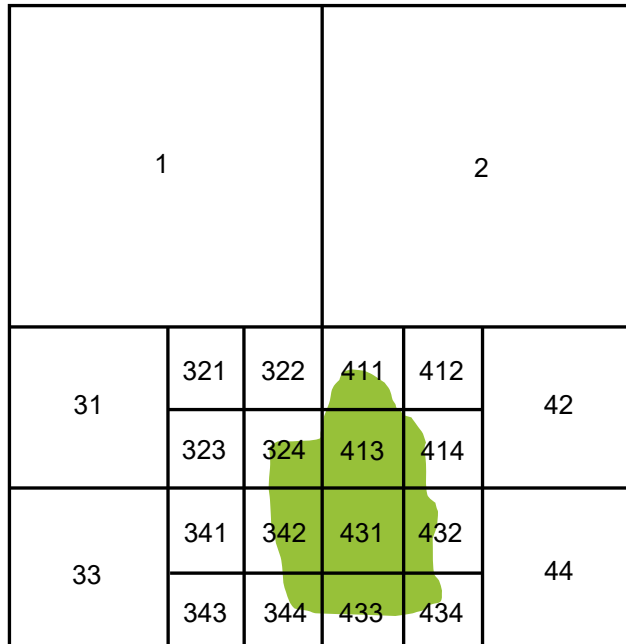
1	2
3	4



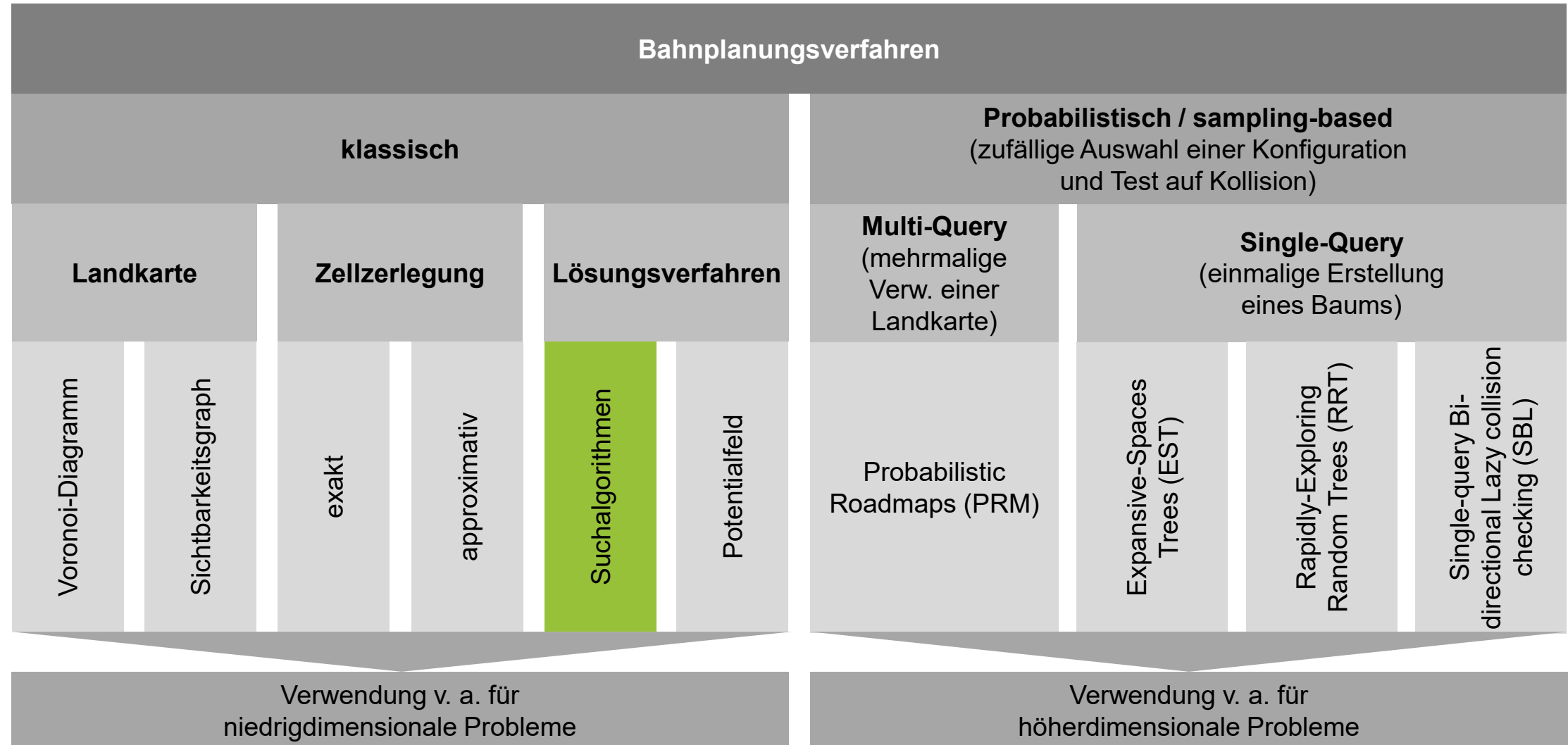
Übung: Beim Quadtree-Verfahren wird durch hierarchische Zerlegung ein zweidimensionaler Raum in Quadrate unterteilt.

Nummerierung

1	2
3	4



Bahnplanungsverfahren für Roboter gliedern sich in klassische und probabilistische (bzw. sampling-based) Verfahren.



Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Ziel

- Finde kürzesten Pfad von Erlangen nach Erfurt

Grundidee

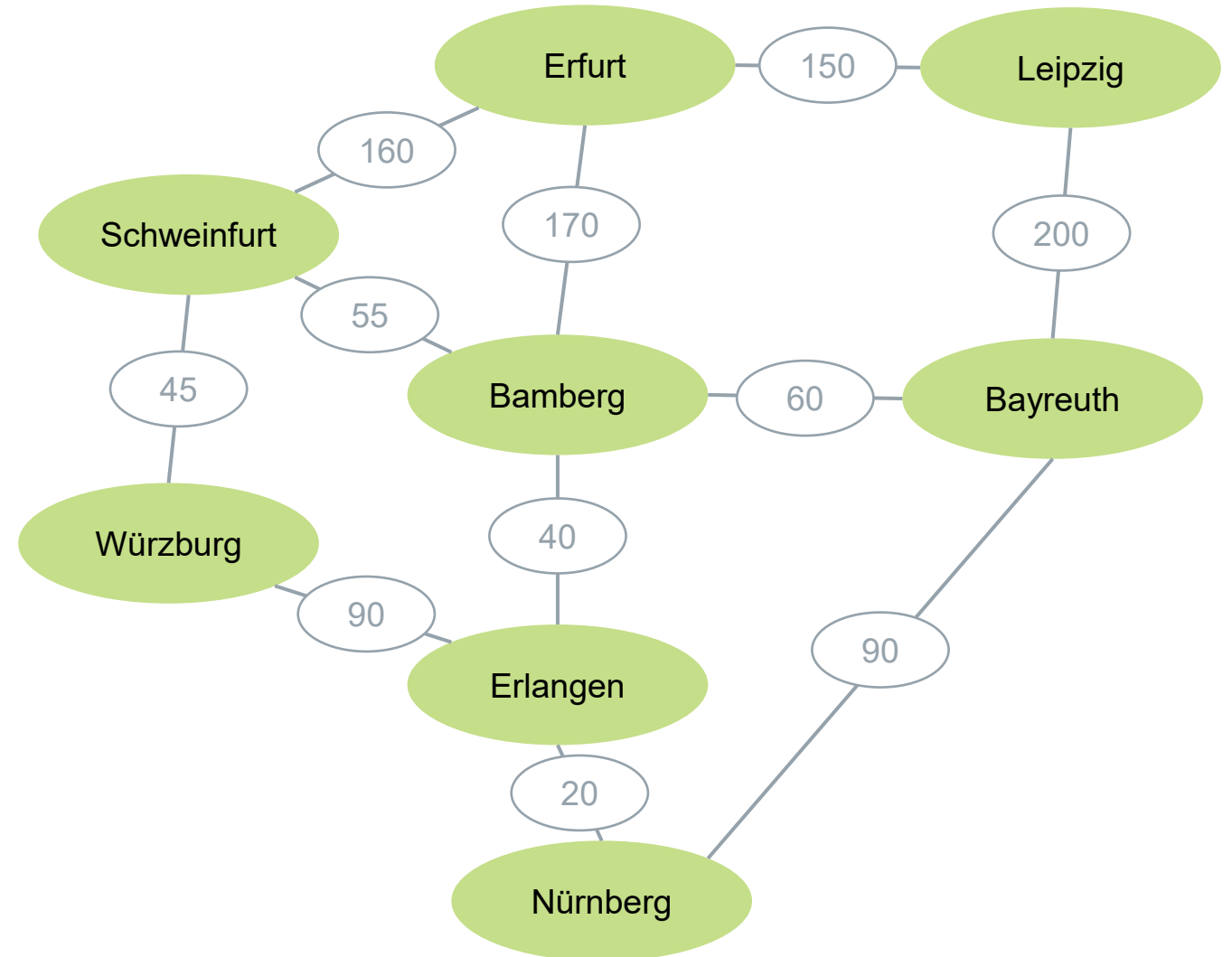
- Verfolgen der günstigsten Kante zum Startknoten führt zum günstigsten Weg

Vorgehen

- Wähle günstigsten Knoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

Eigenschaften

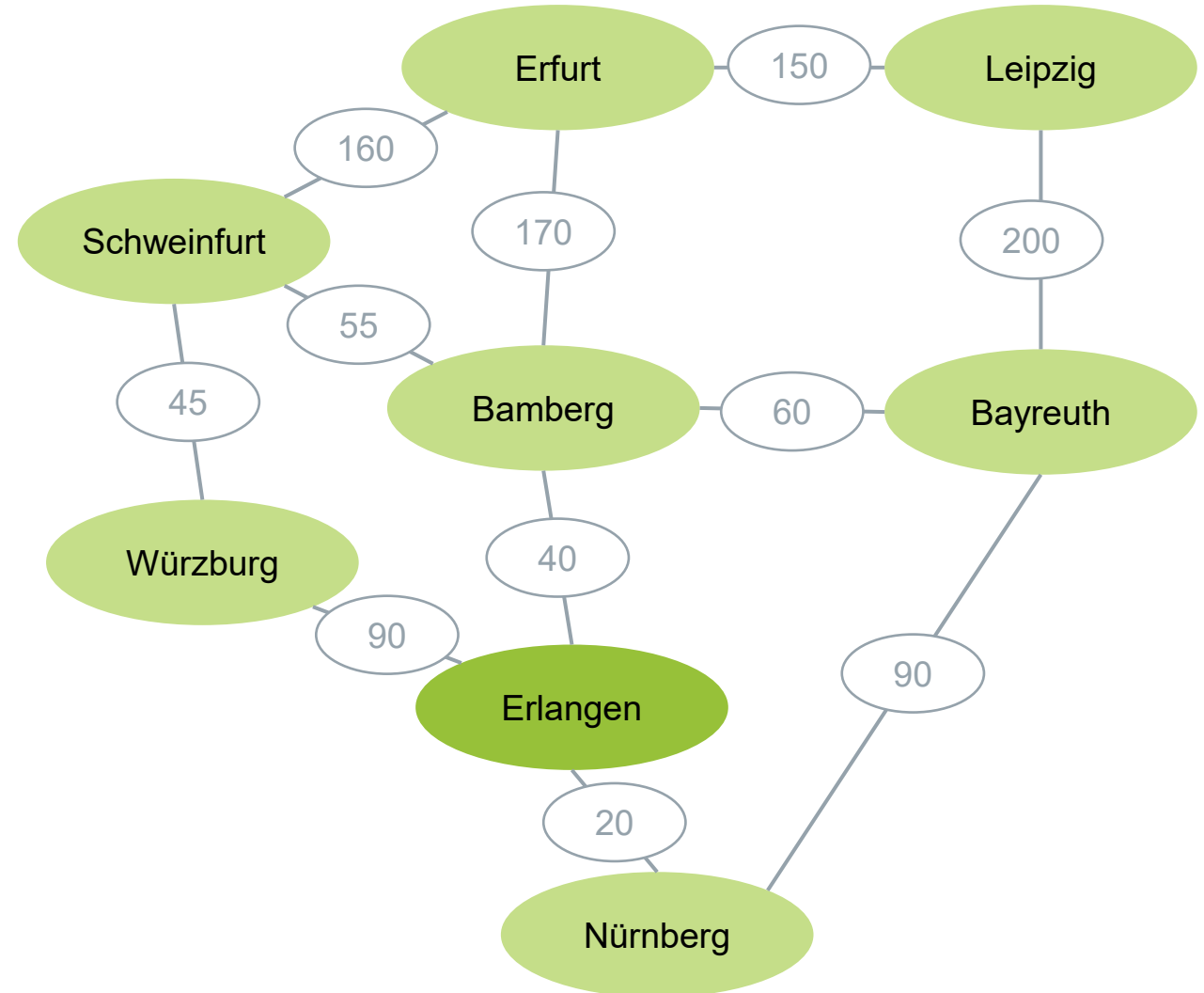
- Vollständig
- Optimal
- Expandiert in alle Richtungen Greedy-Algorithmus (gierig)



Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Übungsaufgabe 3)

Berechnen Sie den kürzesten Pfad von Erlangen nach Erfurt unter Verwendung des **Dijkstra-Algorithmus**. Geben sie dabei ausführlich ihren Rechenweg in der gegebenen Tabelle mit den jeweiligen Kosten $g(x)$ an. Geben sie zu jeder Iteration den aktuell günstigsten Knoten an. Geben Sie den kürzesten Pfad und seine Kosten explizit an.

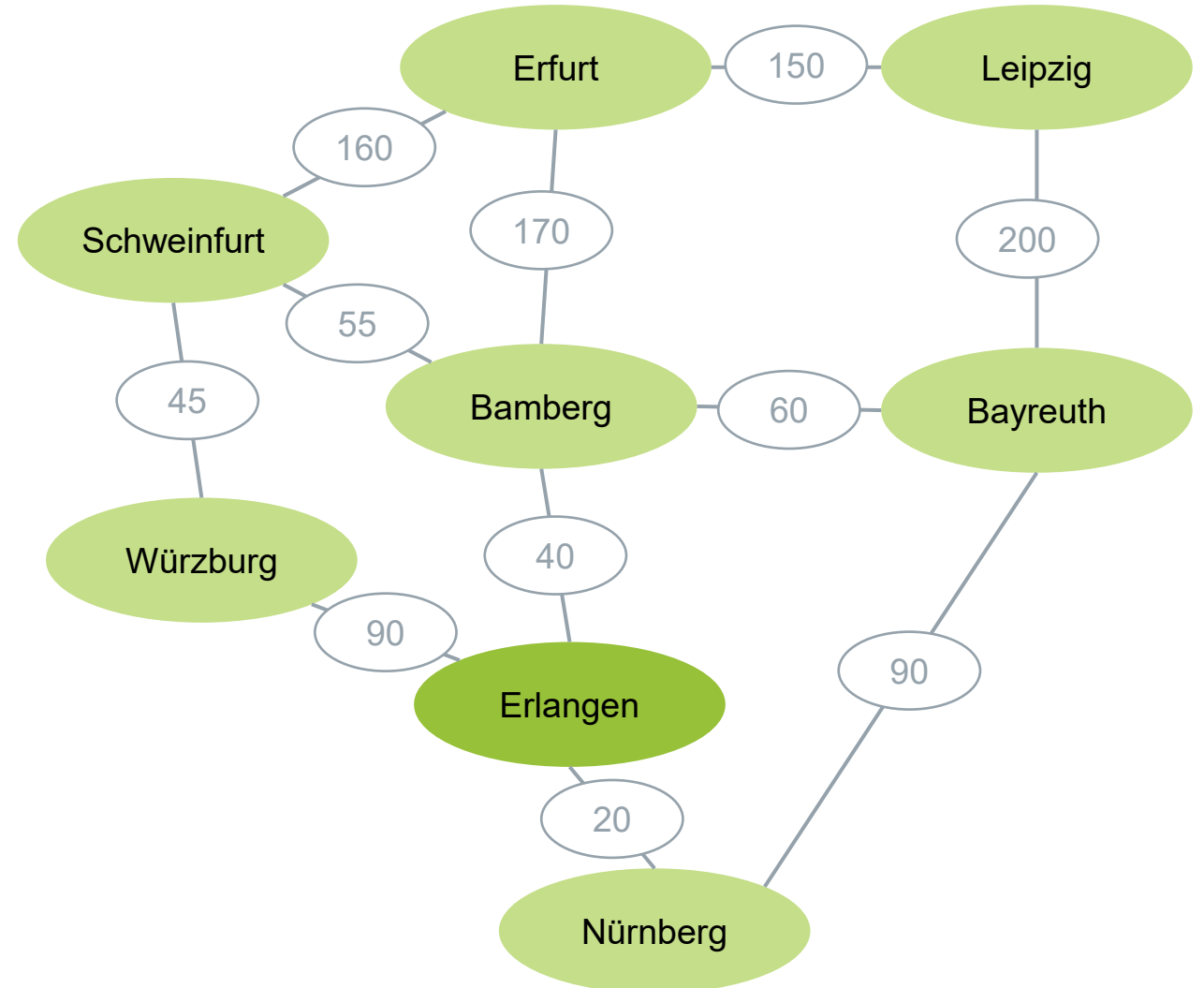


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Initialisiere Startknoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N							
ER							
WÜ							
BA							
BT							
EF							
L							
SW							

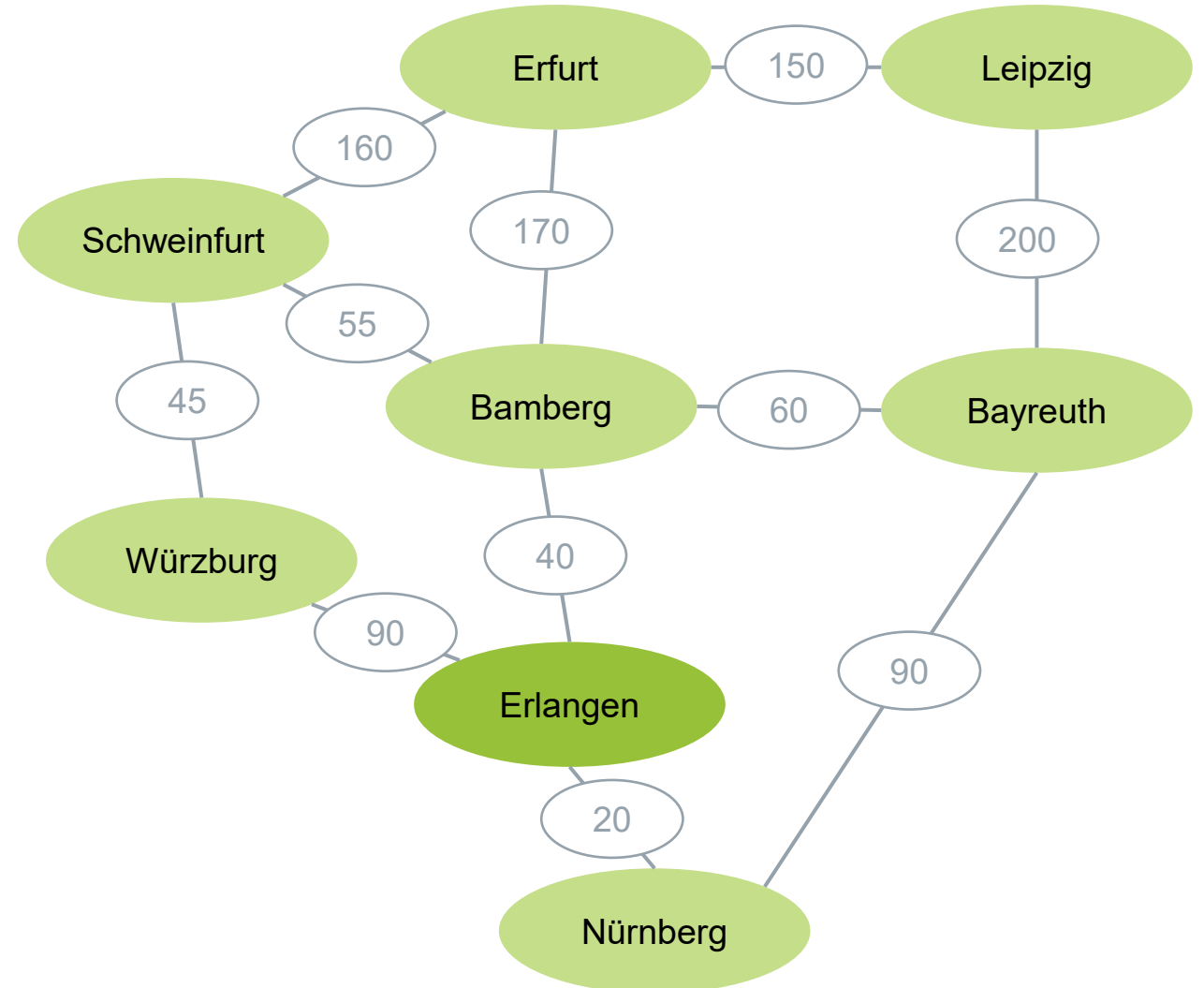


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Initialisiere Startknoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞						
ER	0						
WÜ	∞						
BA	∞						
BT	∞						
EF	∞						
L	∞						
SW	∞						

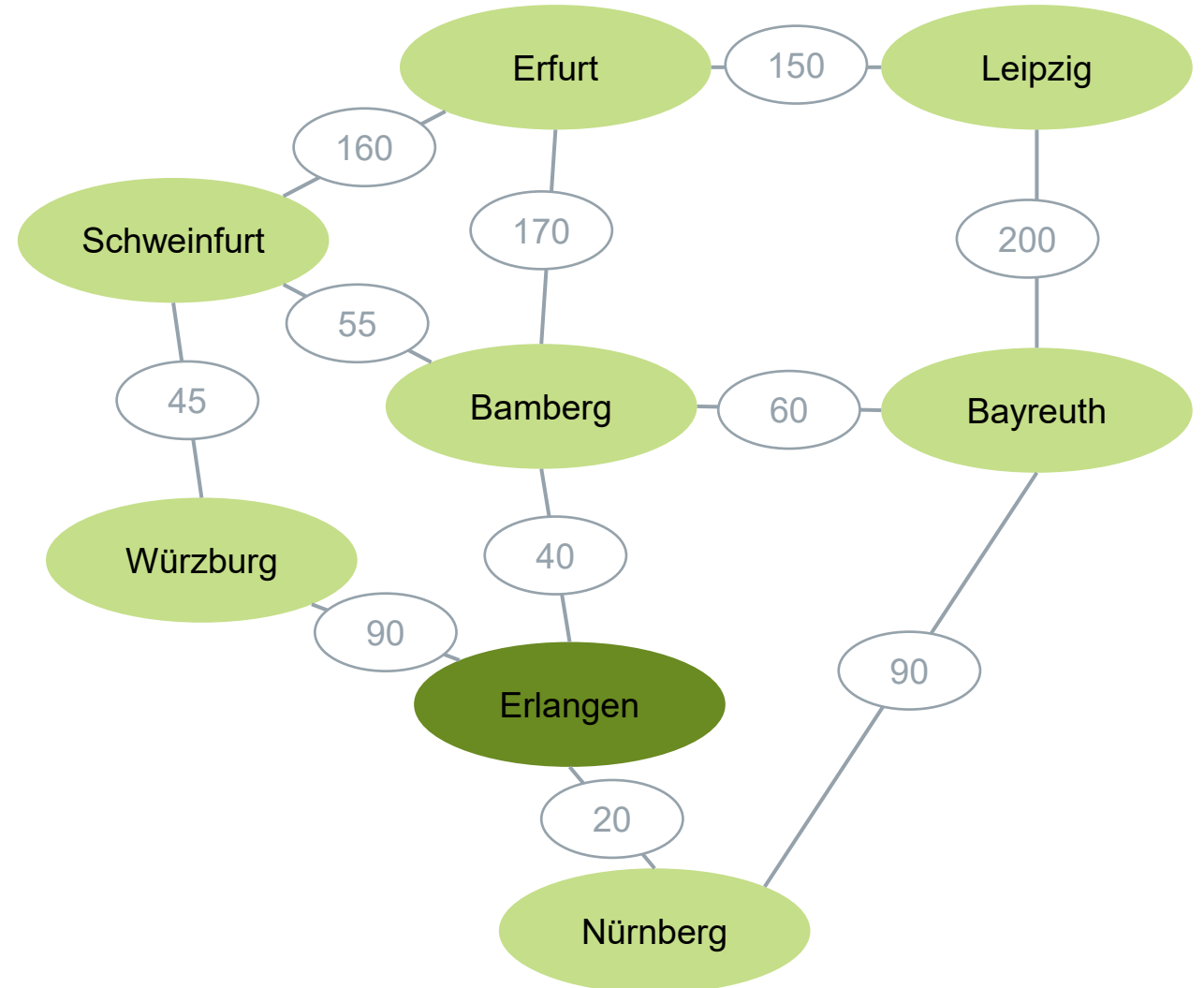


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Initialisiere Startknoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞						
ER	0						
WÜ	∞						
BA	∞						
BT	∞						
EF	∞						
L	∞						
SW	∞						

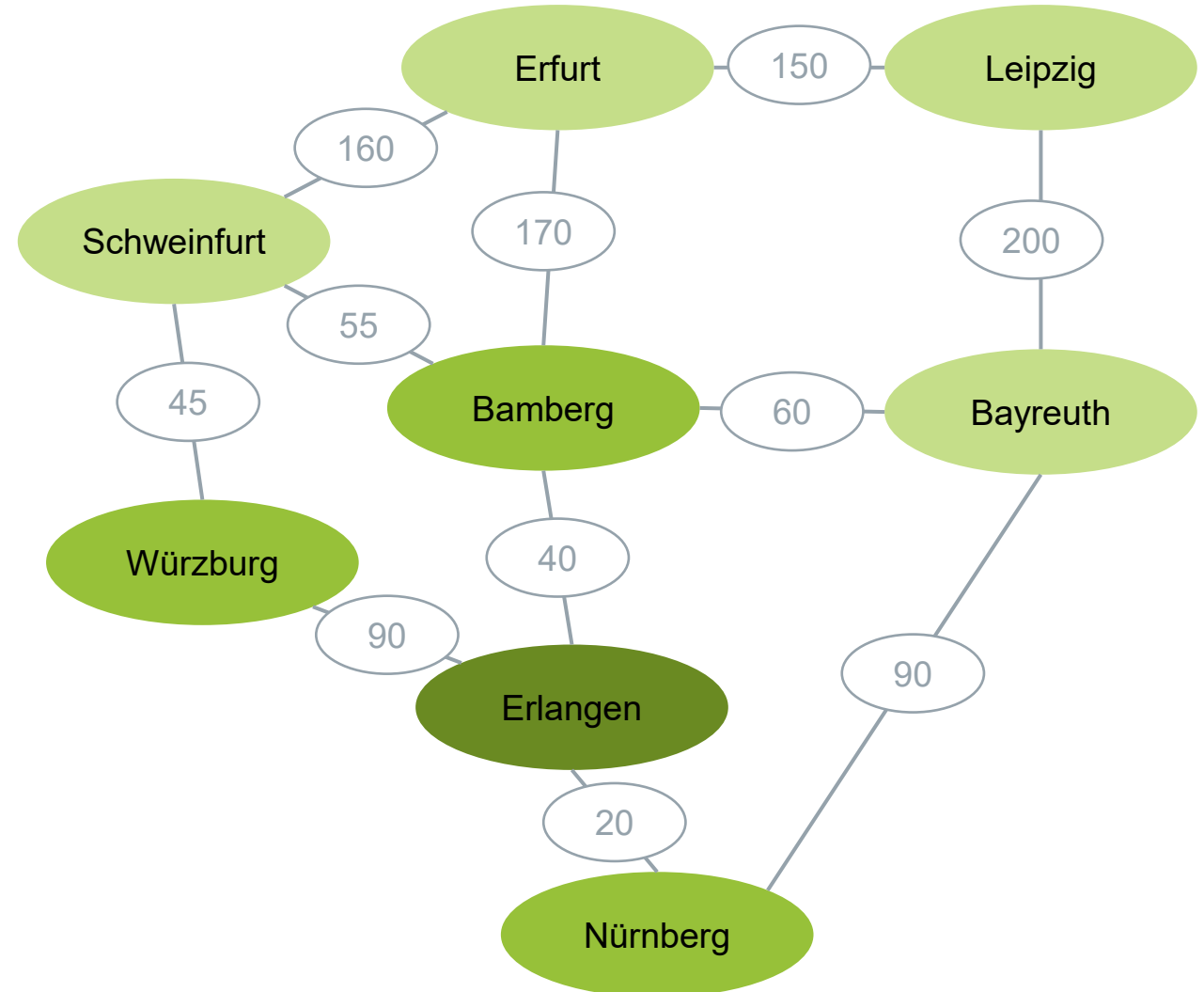


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Initialisiere Startknoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞	20_E					
ER	0	—					
WÜ	∞	90_E					
BA	∞	40_E					
BT	∞	∞					
EF	∞	∞					
L	∞	∞					
SW	∞	∞					

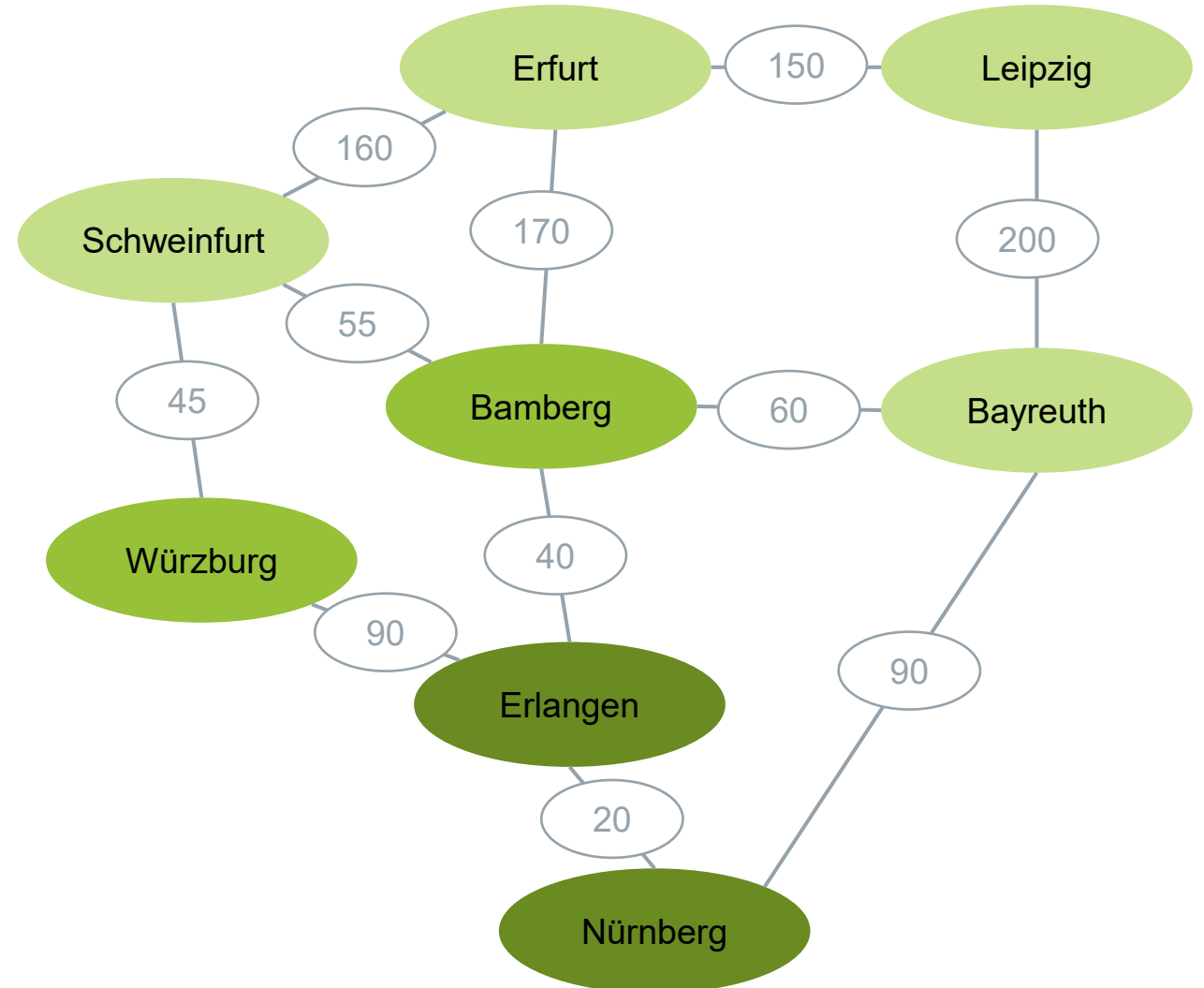


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Wähle günstigsten Knoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞	20_E					
ER	0	—					
WÜ	∞	90_E					
BA	∞	40_E					
BT	∞	∞					
EF	∞	∞					
L	∞	∞					
SW	∞	∞					

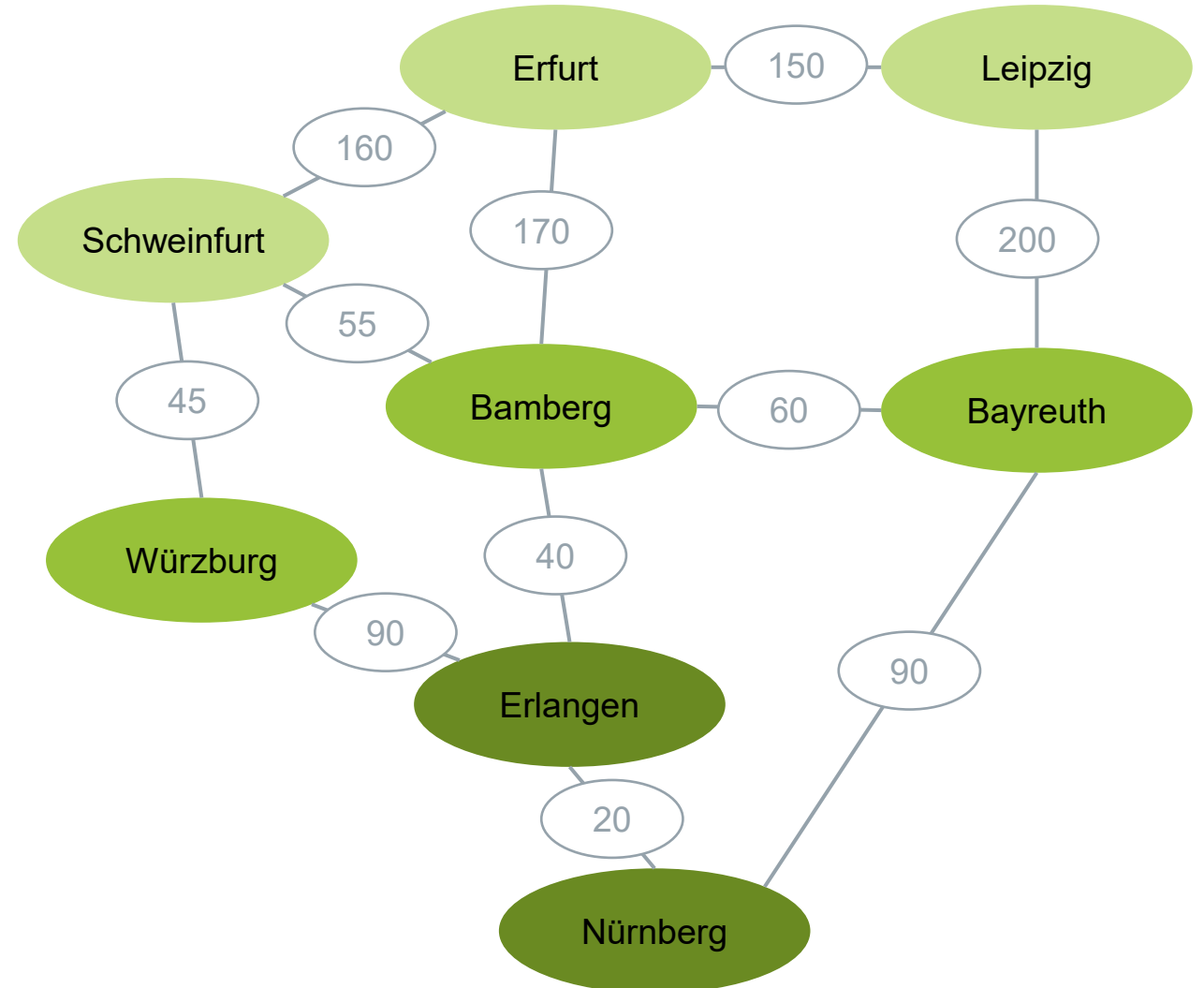


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Wähle günstigsten Knoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞	20 _E	—				
ER	0	—	—				
WÜ	∞	90 _E	90 _E				
BA	∞	40 _E	40 _E				
BT	∞	∞	110 _N				
EF	∞	∞	∞				
L	∞	∞	∞				
SW	∞	∞	∞				

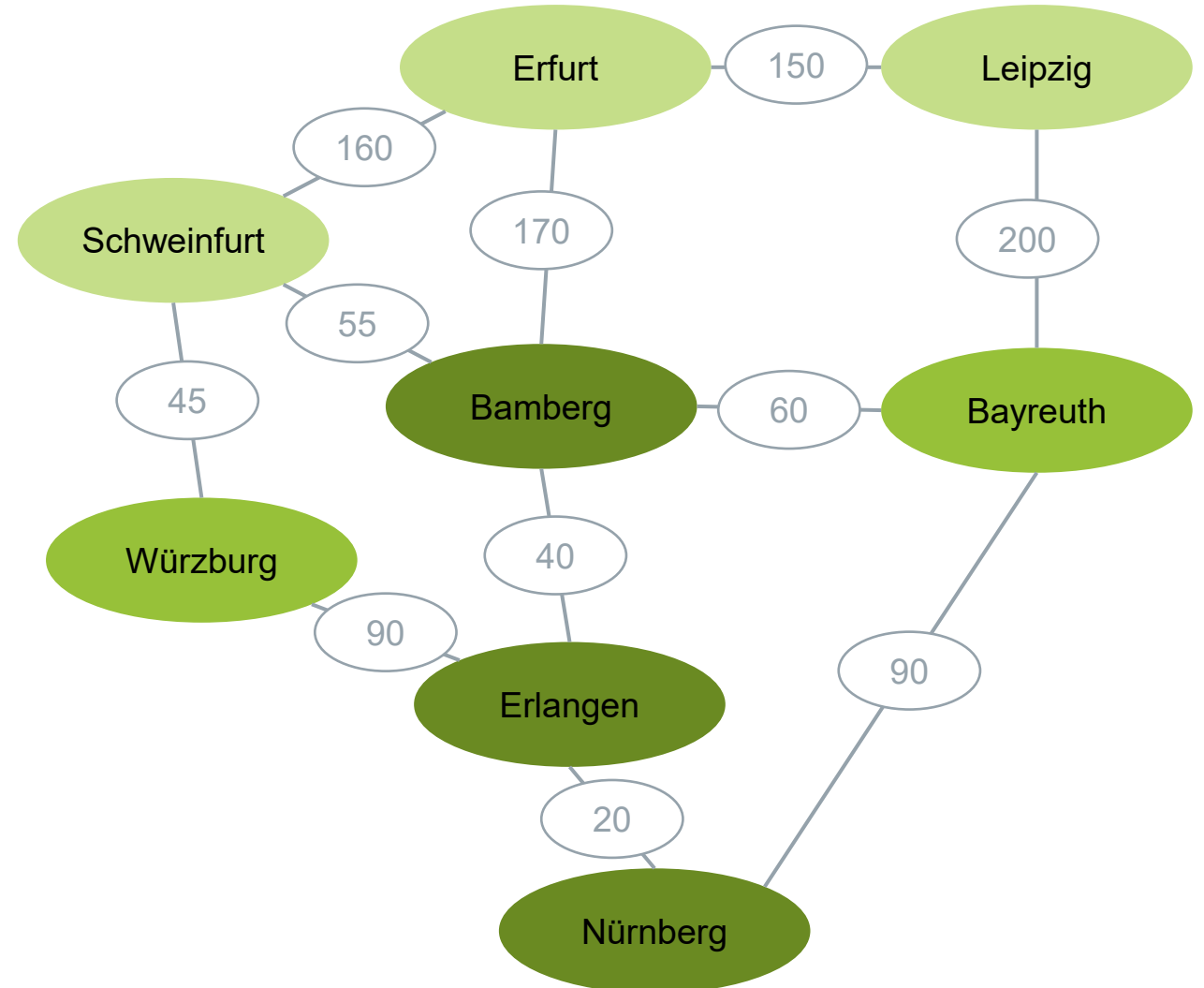


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Wähle günstigsten Knoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞	20 _E	—				
ER	0	—	—				
WÜ	∞	90 _E	90 _E				
BA	∞	40 _E	40 _E				
BT	∞	∞	110 _N				
EF	∞	∞	∞				
L	∞	∞	∞				
SW	∞	∞	∞				

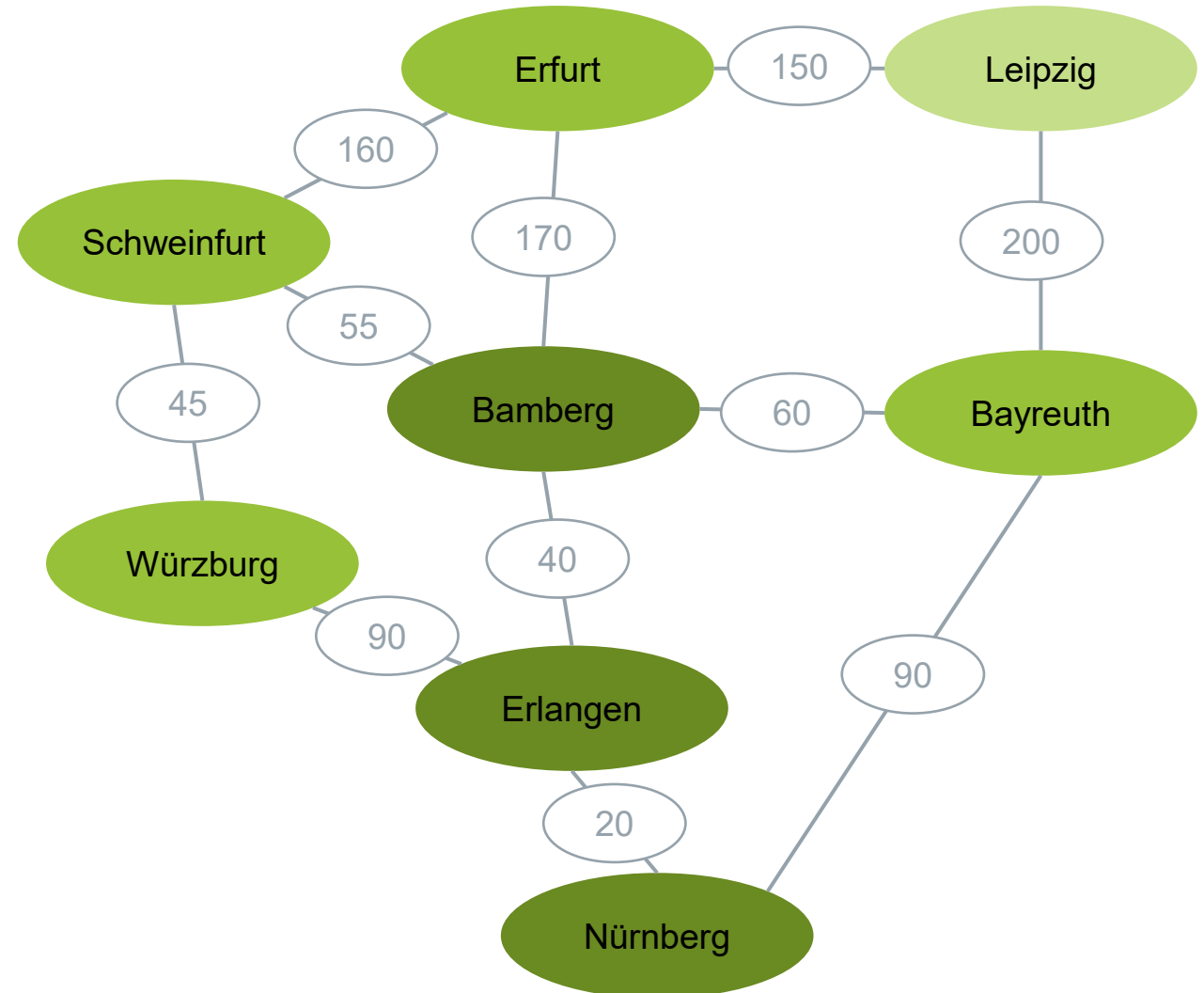


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Wähle günstigsten Knoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞	20_E	—	—			
ER	0	—	—	—			
WÜ	∞	90_E	90_E	90_E			
BA	∞	40_E	40_E	—			
BT	∞	∞	110_N	100_B			
EF	∞	∞	∞	210_B			
L	∞	∞	∞	∞			
SW	∞	∞	∞	95_B			

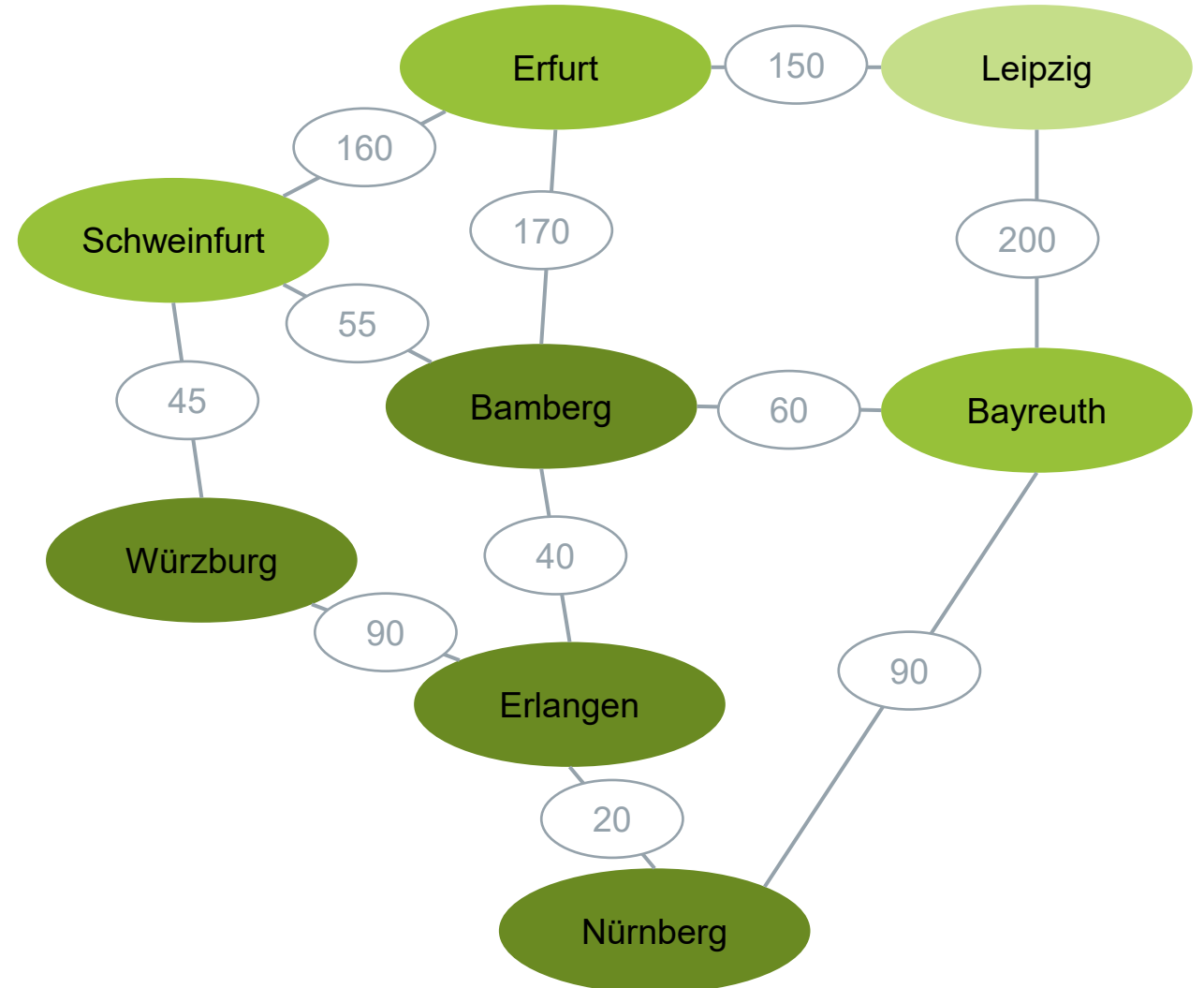


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Wähle günstigsten Knoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞	20_E	—	—			
ER	0	—	—	—			
WÜ	∞	90_E	90_E	90_E			
BA	∞	40_E	40_E	—			
BT	∞	∞	110_N	100_B			
EF	∞	∞	∞	210_B			
L	∞	∞	∞	∞			
SW	∞	∞	∞	95_B			

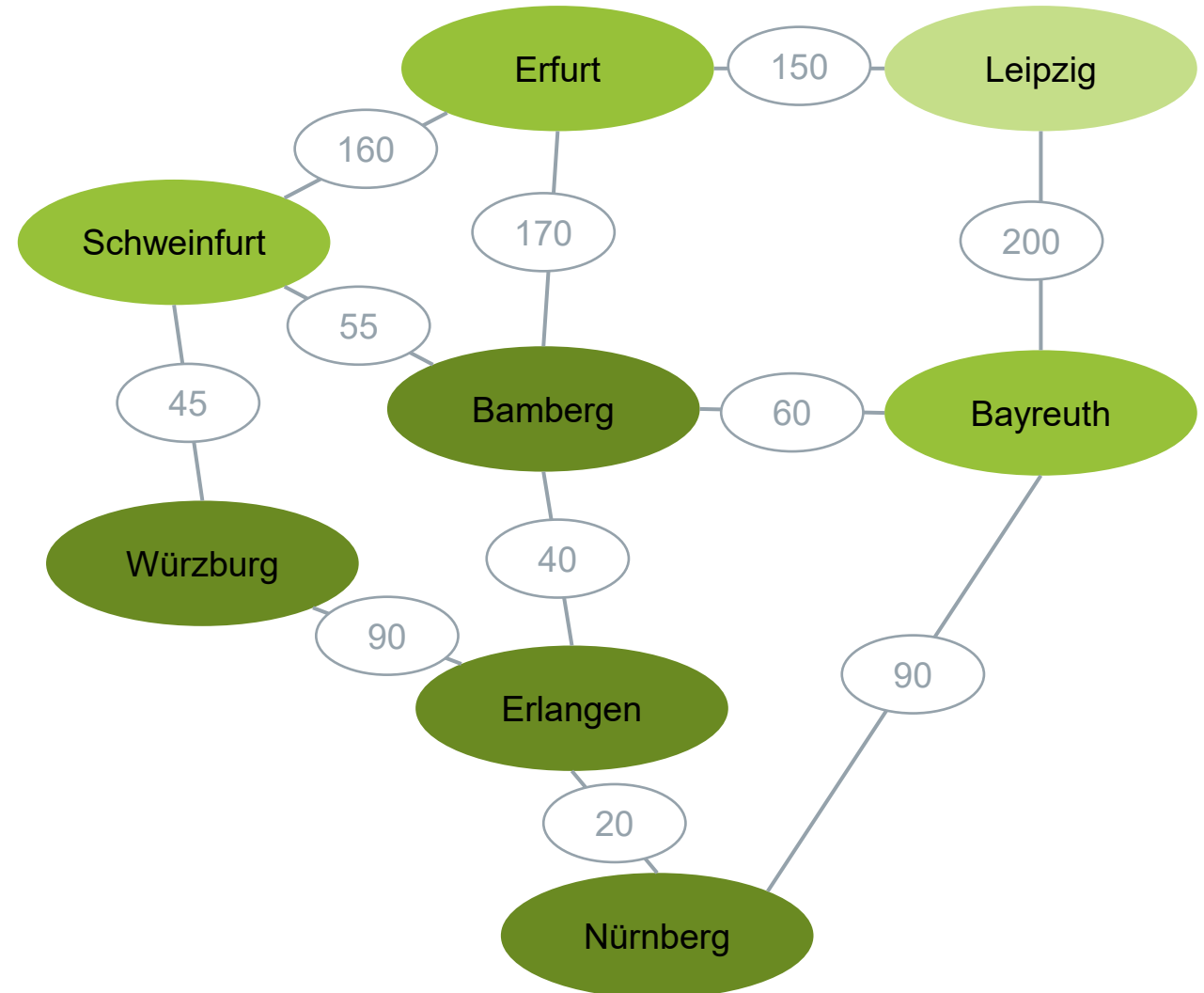


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Wähle günstigsten Knoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞	20 _E	—	—	—		
ER	0	—	—	—	—		
WÜ	∞	90 _E	90 _E	90 _E	—		
BA	∞	40 _E	40 _E	—	—		
BT	∞	∞	110 _N	100 _B	100 _B		
EF	∞	∞	∞	210 _B	210 _B		
L	∞	∞	∞	∞	∞		
SW	∞	∞	∞	95 _B	95 _B		

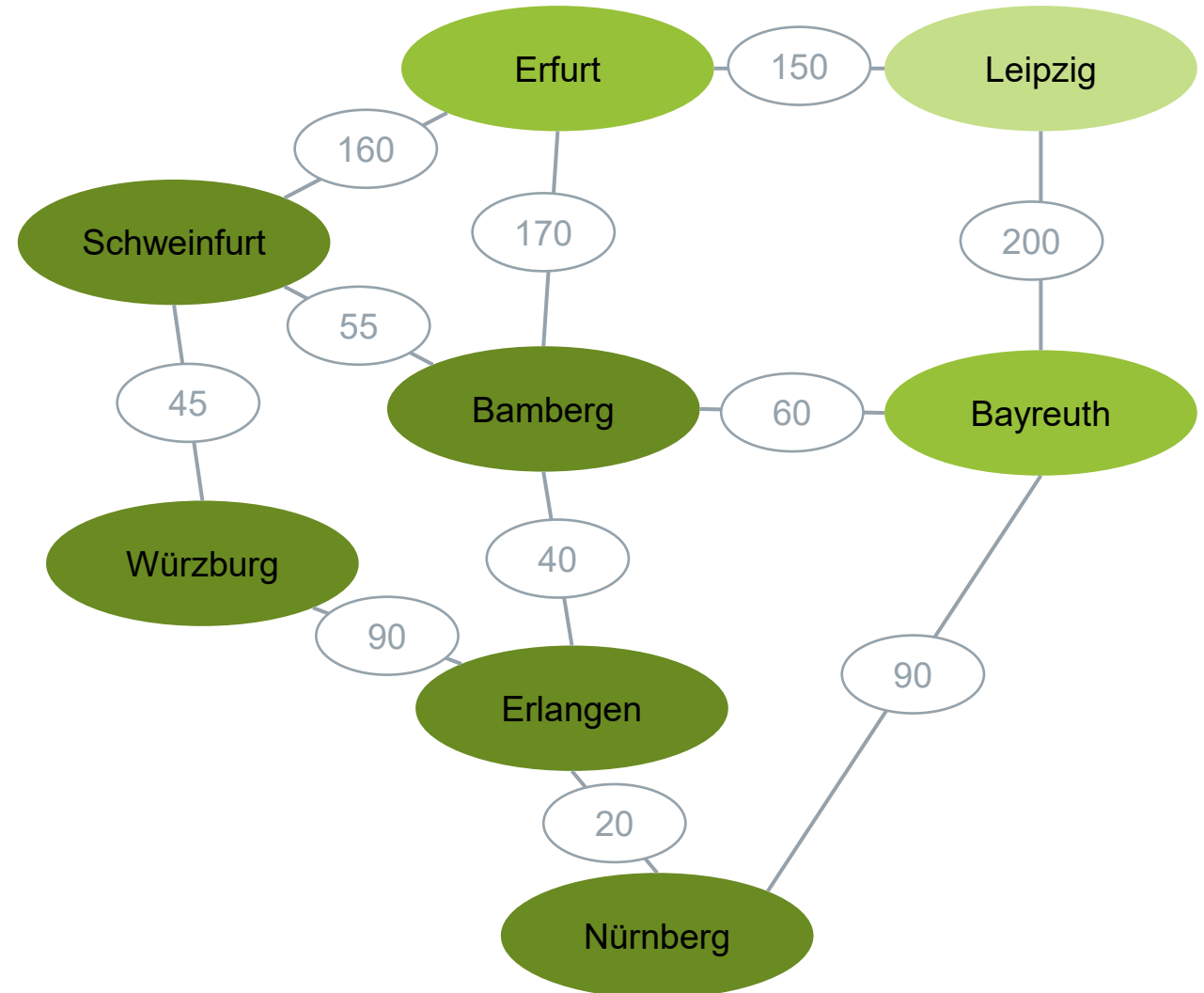


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Wähle günstigsten Knoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞	20 _E	—	—	—		
ER	0	—	—	—	—		
WÜ	∞	90 _E	90 _E	90 _E	—		
BA	∞	40 _E	40 _E	—	—		
BT	∞	∞	110 _N	100 _B	100 _B		
EF	∞	∞	∞	210 _B	210 _B		
L	∞	∞	∞	∞	∞		
SW	∞	∞	∞	95 _B	95 _B		

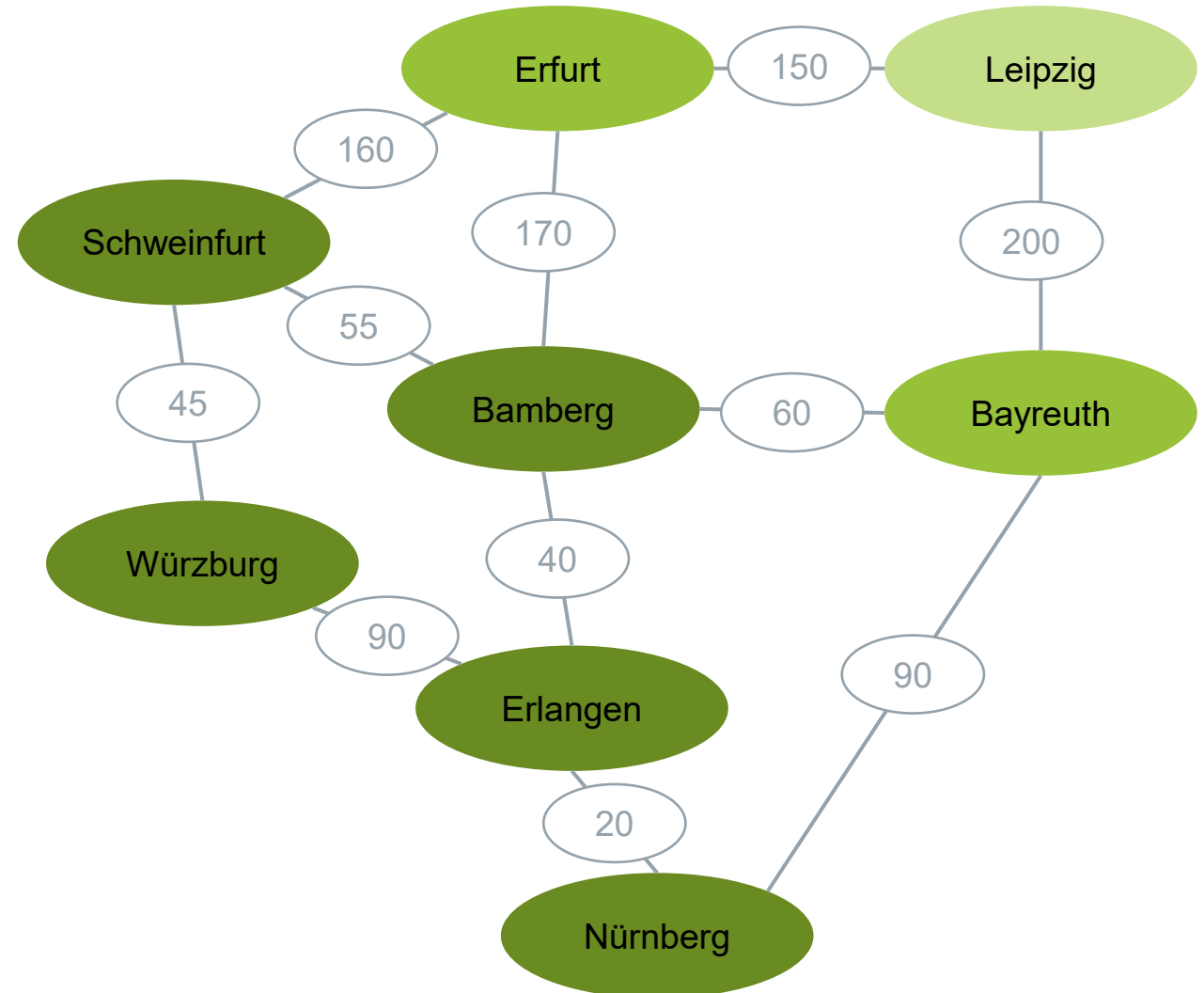


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Wähle günstigsten Knoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞	20 _E	—	—	—	—	
ER	0	—	—	—	—	—	
WÜ	∞	90 _E	90 _E	90 _E	—	—	
BA	∞	40 _E	40 _E	—	—	—	
BT	∞	∞	110 _N	100 _B	100 _B	100 _B	
EF	∞	∞	∞	210 _B	210 _B	210 _B	
L	∞	∞	∞	∞	∞	∞	
SW	∞	∞	∞	95 _B	95 _B	—	

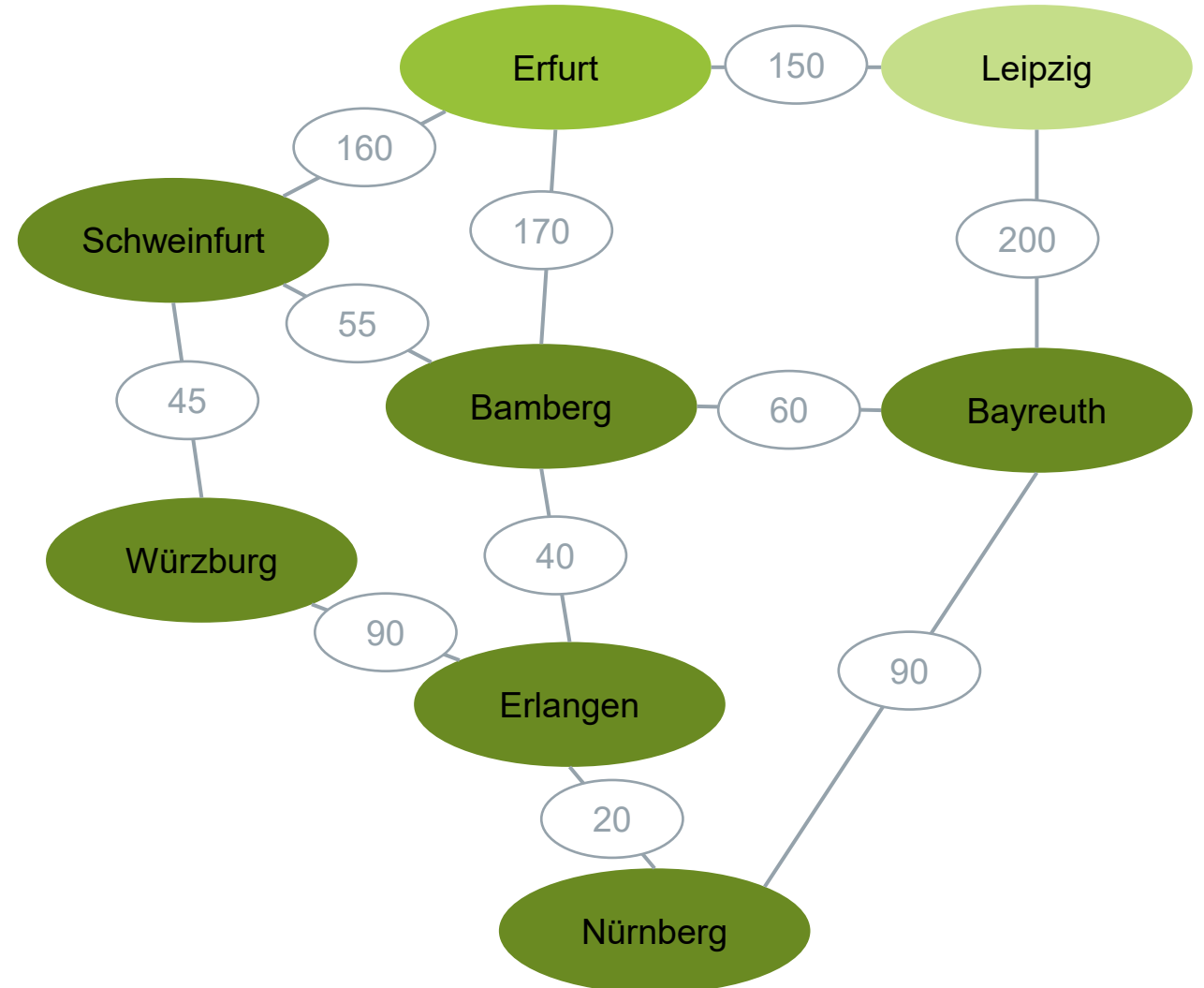


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Wähle günstigsten Knoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞	20 _E	—	—	—	—	
ER	0	—	—	—	—	—	
WÜ	∞	90 _E	90 _E	90 _E	—	—	
BA	∞	40 _E	40 _E	—	—	—	
BT	∞	∞	110 _N	100 _B	100 _B	100 _B	
EF	∞	∞	∞	210 _B	210 _B	210 _B	
L	∞	∞	∞	∞	∞	∞	
SW	∞	∞	∞	95 _B	95 _B	—	

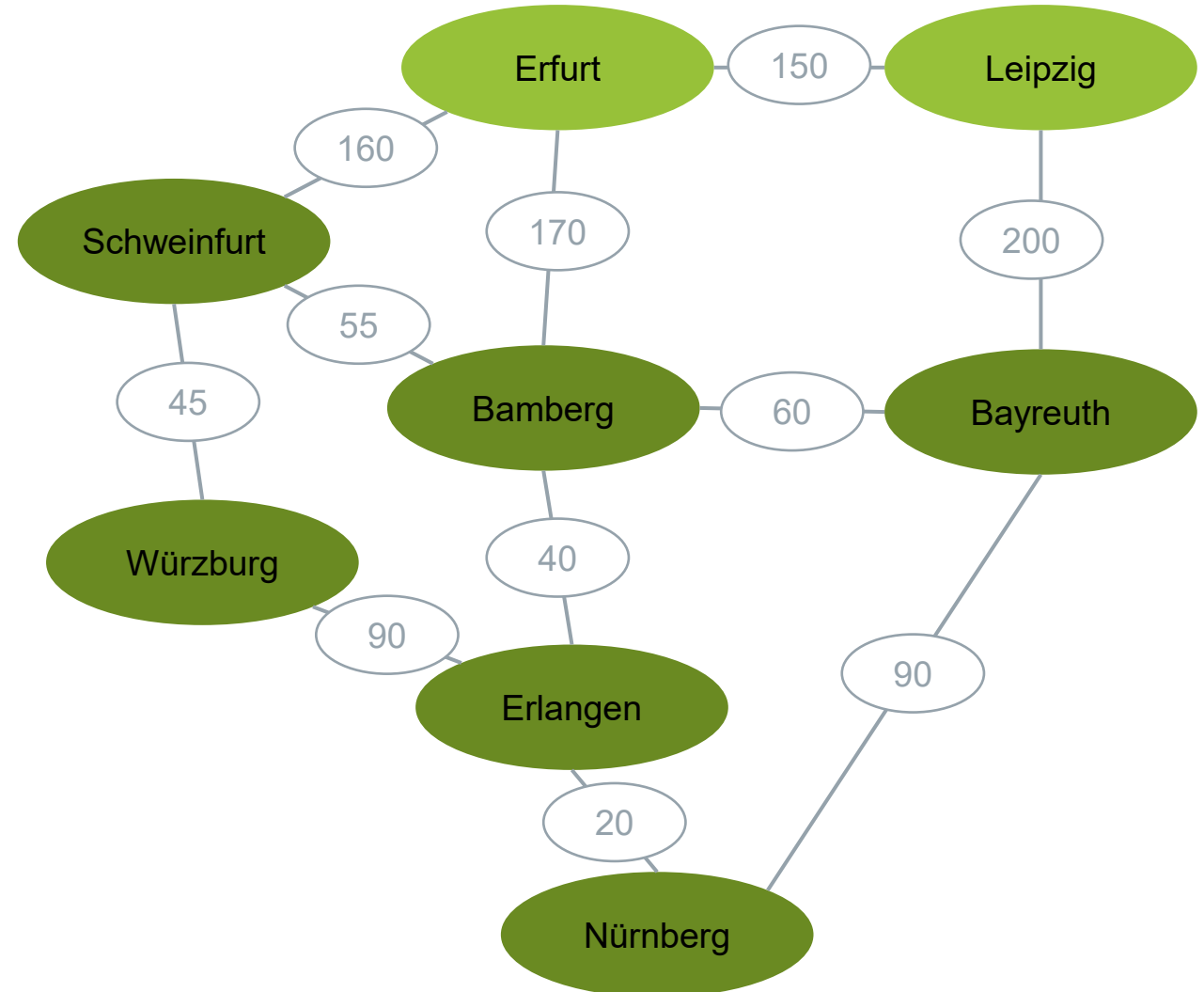


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Wähle günstigsten Knoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞	20_E	—	—	—	—	—
ER	0	—	—	—	—	—	—
WÜ	∞	90_E	90_E	90_E	—	—	—
BA	∞	40_E	40_E	—	—	—	—
BT	∞	∞	110_N	100_B	100_B	100_B	—
EF	∞	∞	∞	210_B	210_B	210_B	210_B
L	∞	∞	∞	∞	∞	∞	300_{BT}
SW	∞	∞	∞	95_B	95_B	—	—

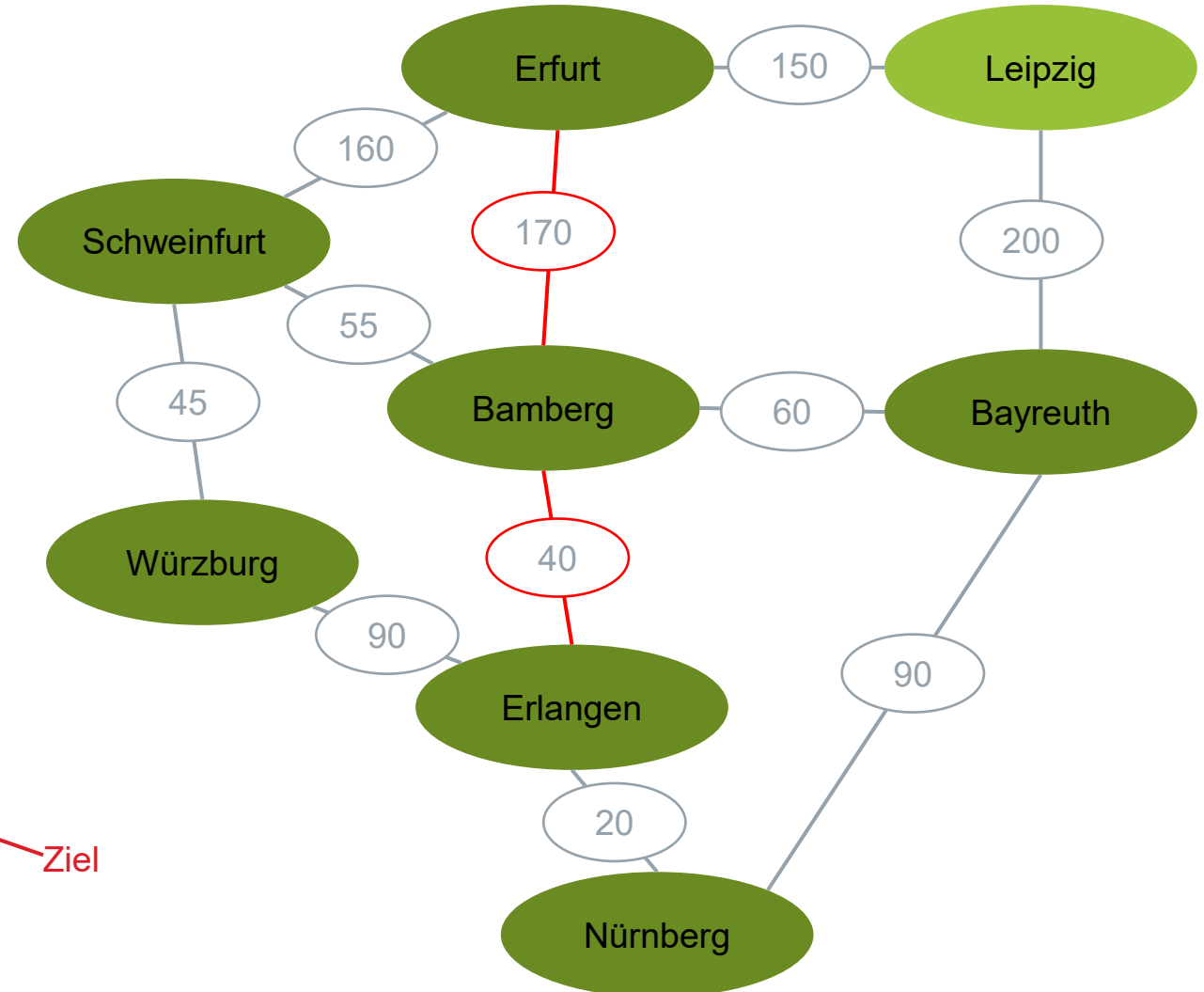


Übung: Der Dijkstra-Algorithmus löst das Problem des kürzesten Pfades und garantiert die optimale Lösung zu finden, wenn das Problem lösbar ist.

Vorgehen

- Wähle günstigsten Knoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4	5	6	7
N	∞	20_E	—	—	—	—	—
ER	0	—	—	—	—	—	—
WÜ	∞	90_E	90_E	90_E	—	—	—
BA	∞	40_E	40_E	—	—	—	—
BT	∞	∞	110_N	100_B	100_B	100_B	—
EF	∞	∞	∞	210_B	210_B	210_B	210_B
L	∞	∞	∞	∞	∞	∞	300_{BT}
SW	∞	∞	∞	95_B	95_B	—	—



Der A*-Algorithmus gehört zu den informierten Suchalgorithmen und optimiert das Expansionsverhalten durch die Verwendung einer Heuristik in Richtung des Zielknotens.

Ziel

- Finde kürzesten Pfad von Erlangen nach Erfurt

Grundidee

- Jedem Knoten wird eine Metrik $f(x)$ zugeordnet, die sich aus den Kosten $g(x)$ und einer Heuristik $h(x)$ berechnet:
– $f(x) = g(x) + h(x)$

Heuristik $h(x)$

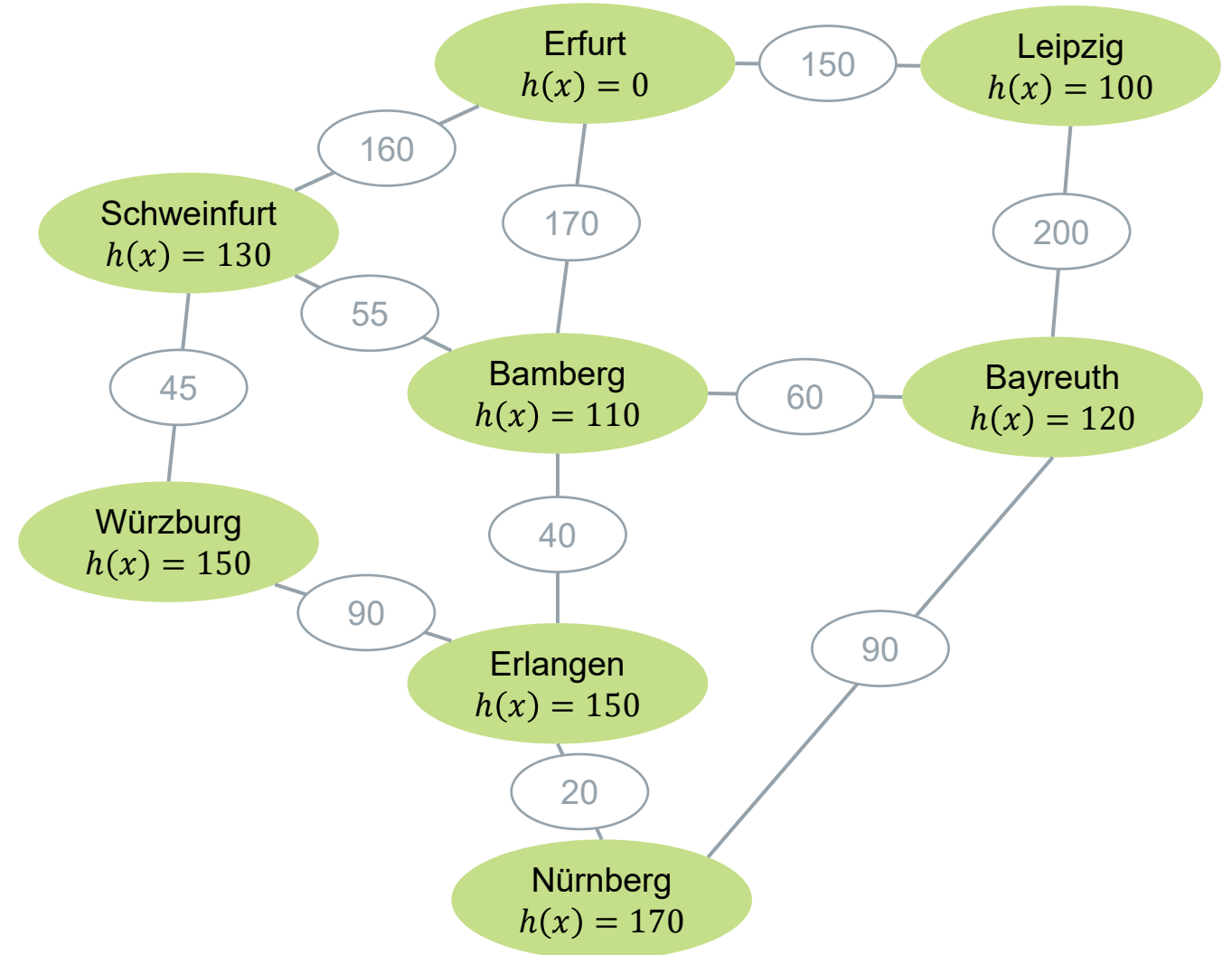
- Muss Optimistisch sein, die tatsächlichen Kosten immer unterschätzen
- Zum Beispiel Luftlinie

Vorgehen

- Aktualisiere die Knoten nach Metrik $f(x)$

Eigenschaften

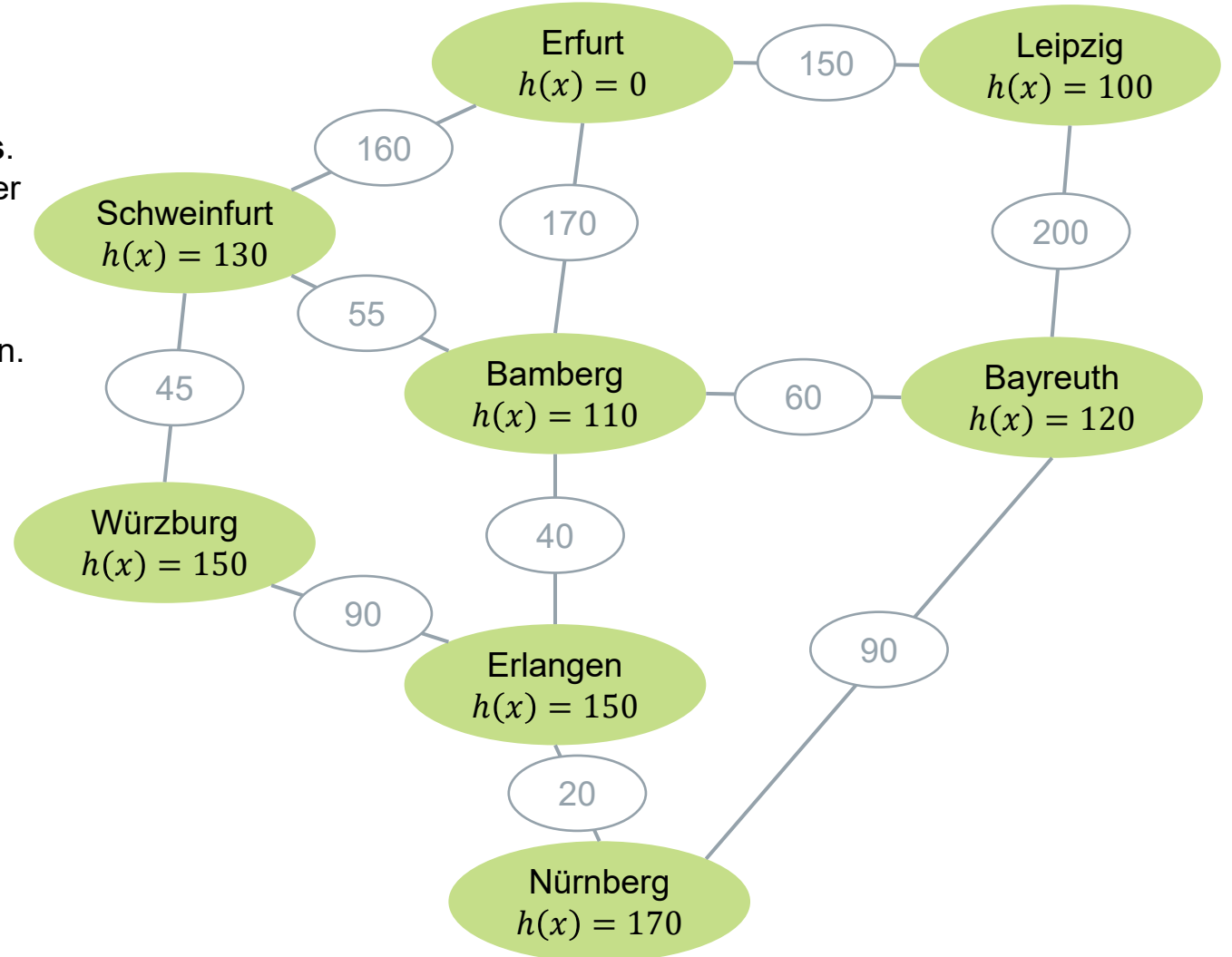
- Vollständig
- Optimal
- Informierte Suche



Der A*-Algorithmus gehört zu den informierten Suchalgorithmen und optimiert das Expansionsverhalten durch die Verwendung einer Heuristik in Richtung des Zielknotens.

Übungsaufgabe 4)

Berechnen Sie den kürzesten Pfad von Erlangen nach Erfurt unter Verwendung des **A*-Algorithmus**. Geben sie dabei ausführlich ihren Rechenweg in der gegebenen Tabelle mit den jeweiligen Kosten $g(x)$ und der Heuristik $h(x)$ an. Geben sie zu jeder Iteration den aktuell günstigsten Knoten an. Geben Sie den kürzesten Pfad und seine Kosten explizit an.

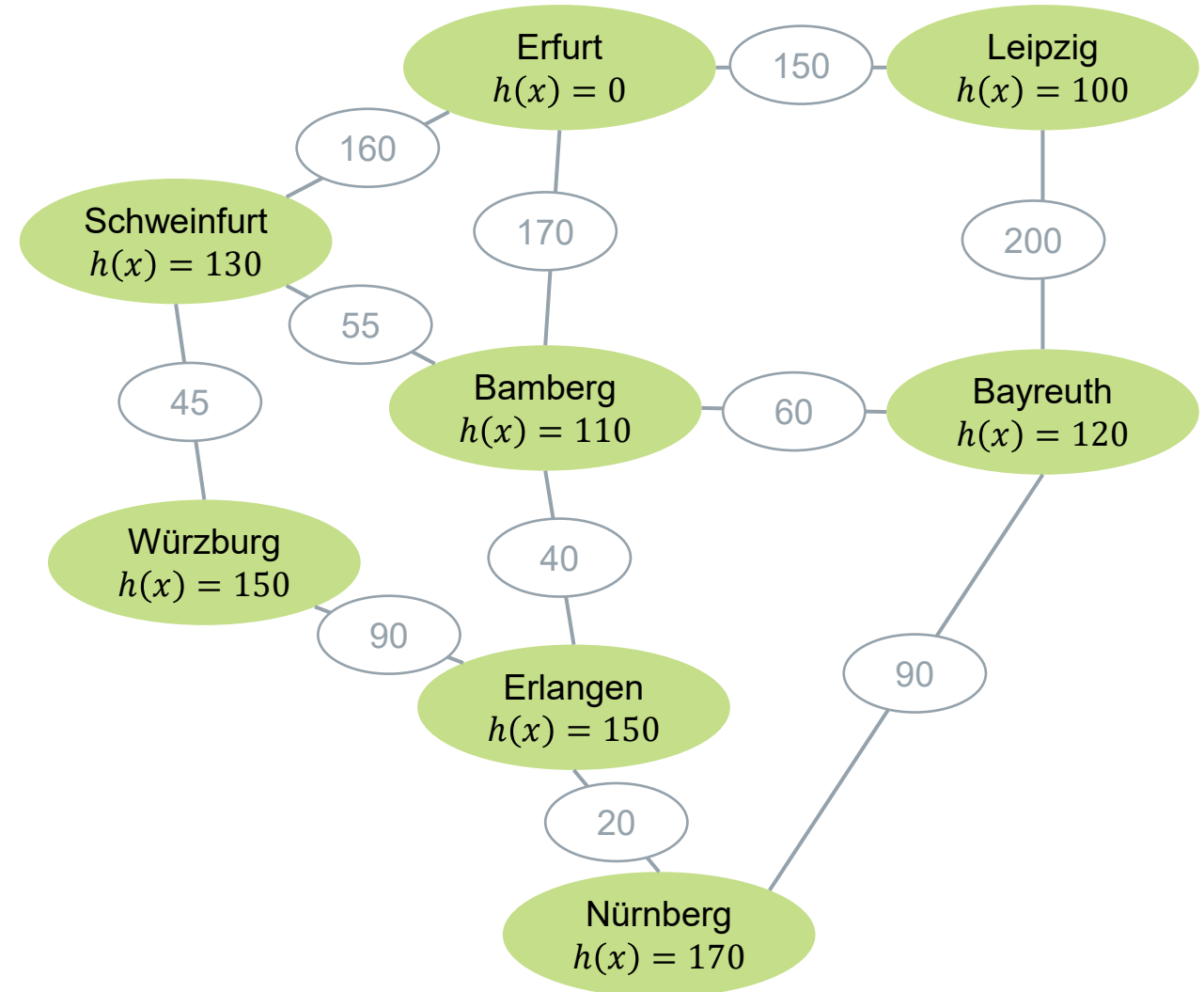


Übung: Der A*-Algorithmus gehört zu den informierten Suchalgorithmen und optimiert das Expansionsverhalten durch die Verwendung einer Heuristik in Richtung des Zielknotens.

Vorgehen

- Initialisiere Startknoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

lt.	1	2	3	4
N				
ER				
WÜ				
BA				
BT				
EF				
L				
SW				

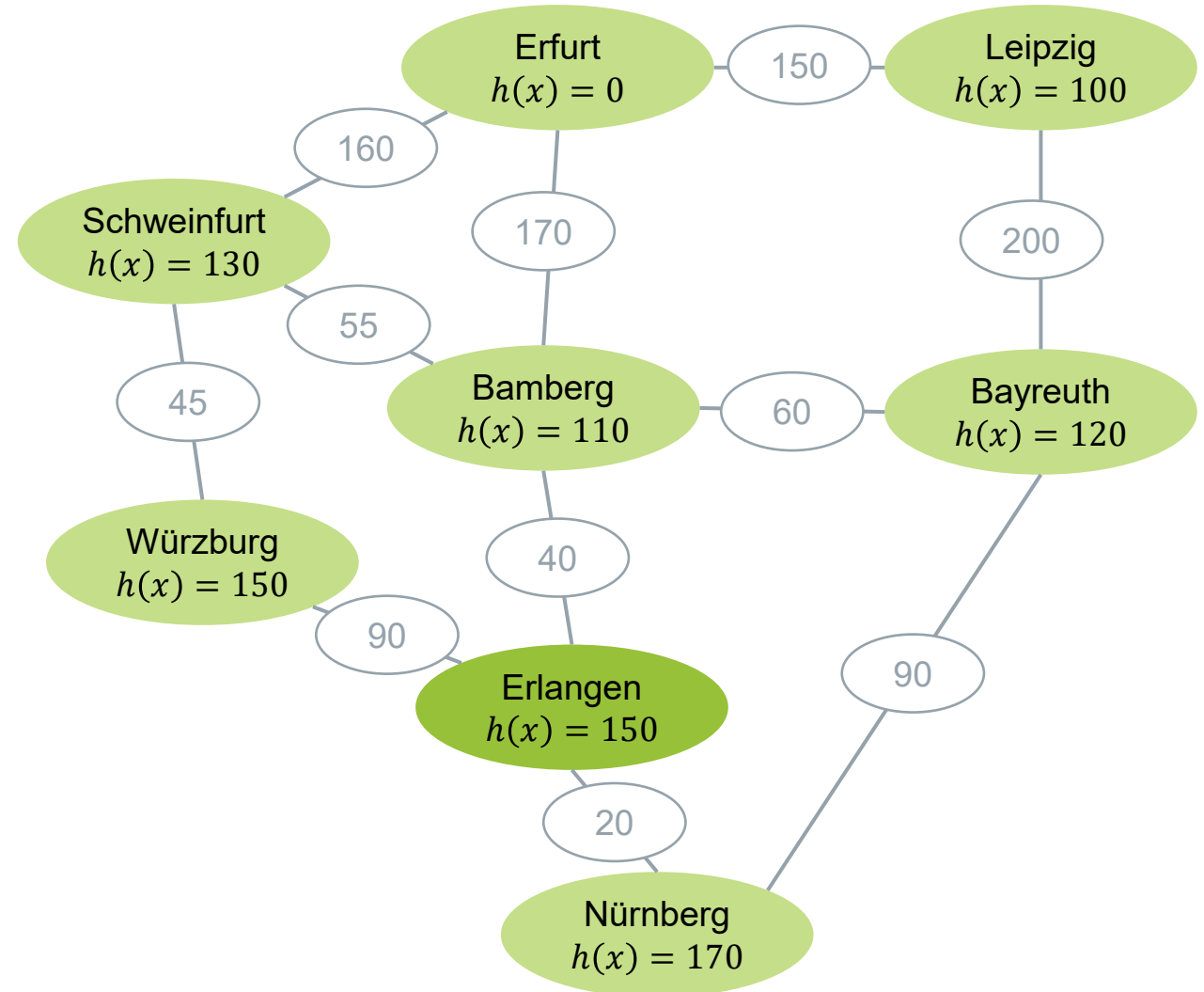


Übung: Der A*-Algorithmus gehört zu den informierten Suchalgorithmen und optimiert das Expansionsverhalten durch die Verwendung einer Heuristik in Richtung des Zielknotens.

Vorgehen

- Initialisiere Startknoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

lt.	1	2	3	4
N	∞			
ER	0			
WÜ	∞			
BA	∞			
BT	∞			
EF	∞			
L	∞			
SW	∞			

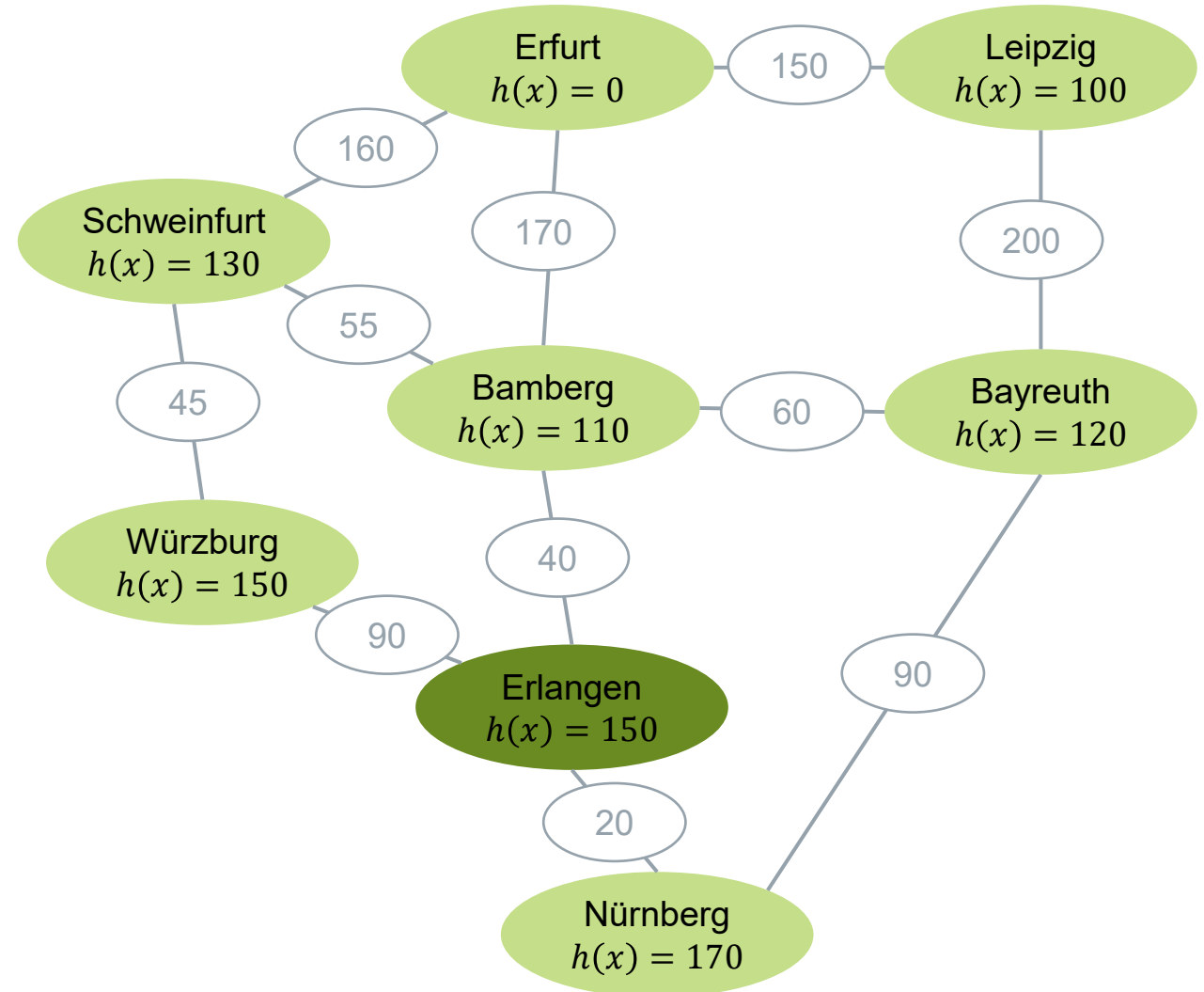


Übung: Der A*-Algorithmus gehört zu den informierten Suchalgorithmen und optimiert das Expansionsverhalten durch die Verwendung einer Heuristik in Richtung des Zielknotens.

Vorgehen

- Initialisiere Startknoten
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

lt.	1	2	3	4
N	∞			
ER	0			
WÜ	∞			
BA	∞			
BT	∞			
EF	∞			
L	∞			
SW	∞			

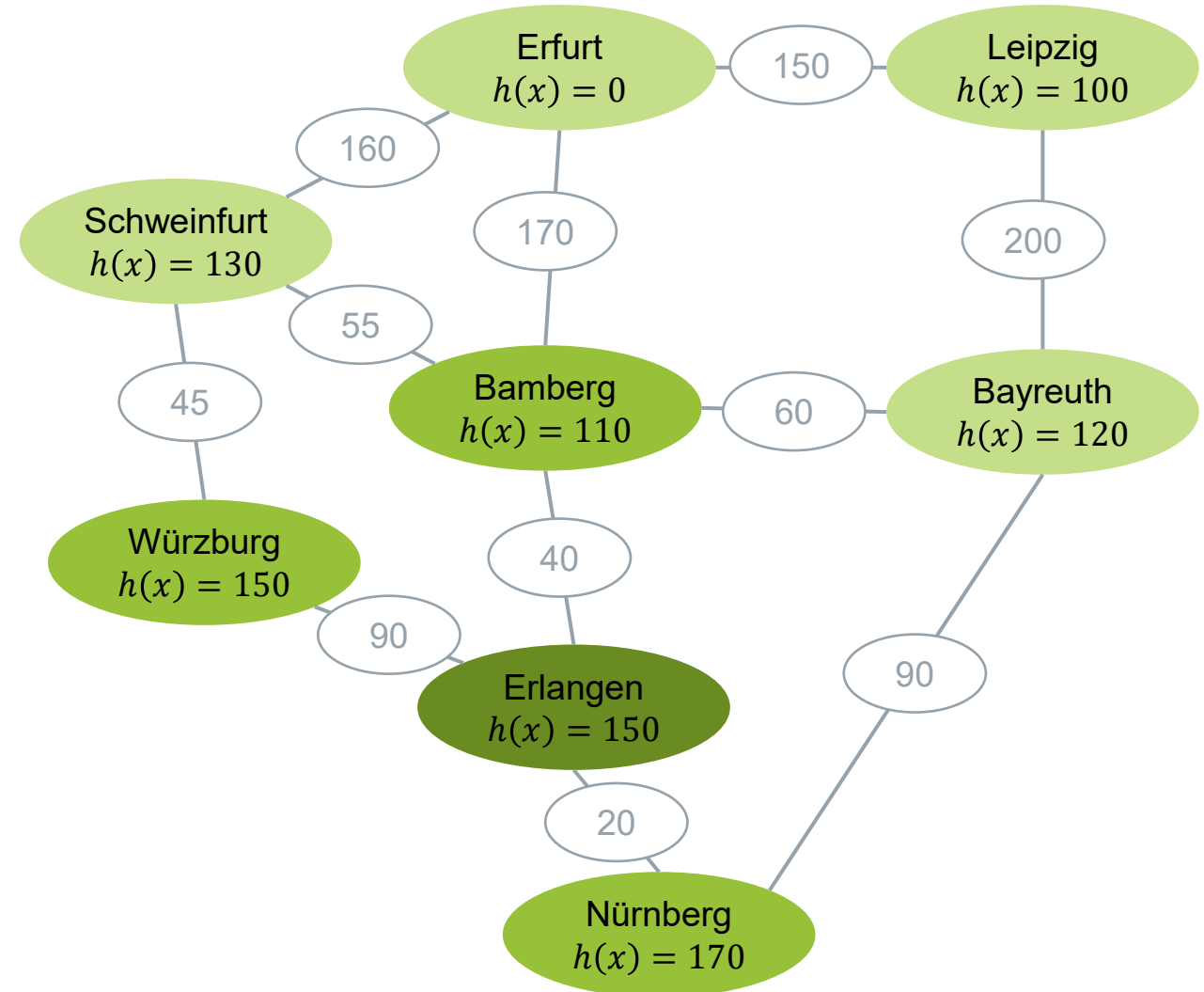


Übung: Der A*-Algorithmus gehört zu den informierten Suchalgorithmen und optimiert das Expansionsverhalten durch die Verwendung einer Heuristik in Richtung des Zielknotens.

Vorgehen

- Wähle günstigsten Knoten nach Metrik
 $f(x) = g(x) + h(x)$
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4
N	∞	$20 + 170_E$		
ER	0	—		
WÜ	∞	$90 + 150_E$		
BA	∞	$40 + 110_E$		
BT	∞	∞		
EF	∞	∞		
L	∞	∞		
SW	∞	∞		

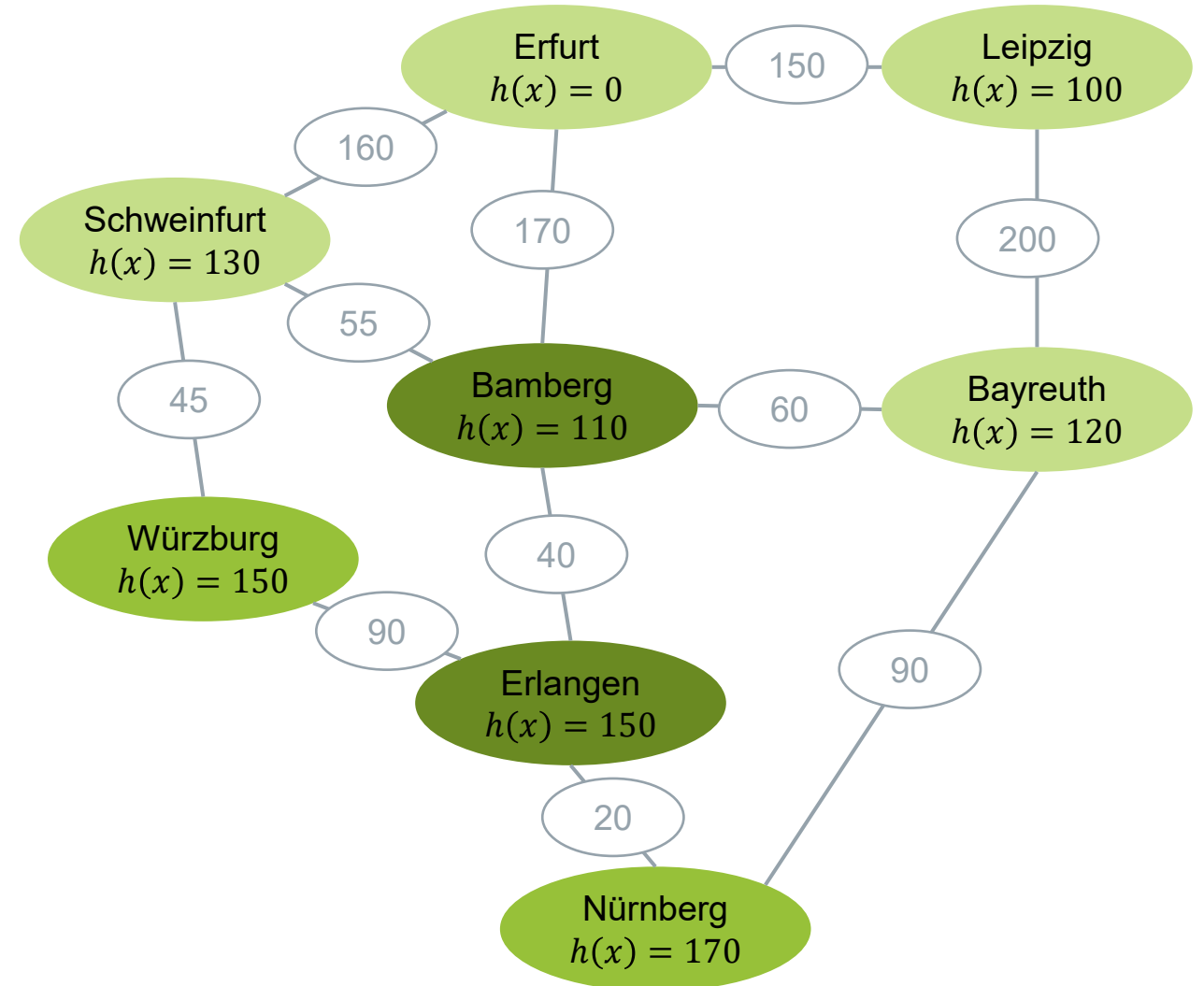


Übung: Der A*-Algorithmus gehört zu den informierten Suchalgorithmen und optimiert das Expansionsverhalten durch die Verwendung einer Heuristik in Richtung des Zielknotens.

Vorgehen

- Wähle günstigsten Knoten nach Metrik
 $f(x) = g(x) + h(x)$
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

It.	1	2	3	4
N	∞	$20 + 170_E$		
ER	0	—		
WÜ	∞	$90 + 150_E$		
BA	∞	$40 + 110_E$		
BT	∞	∞		
EF	∞	∞		
L	∞	∞		
SW	∞	∞		

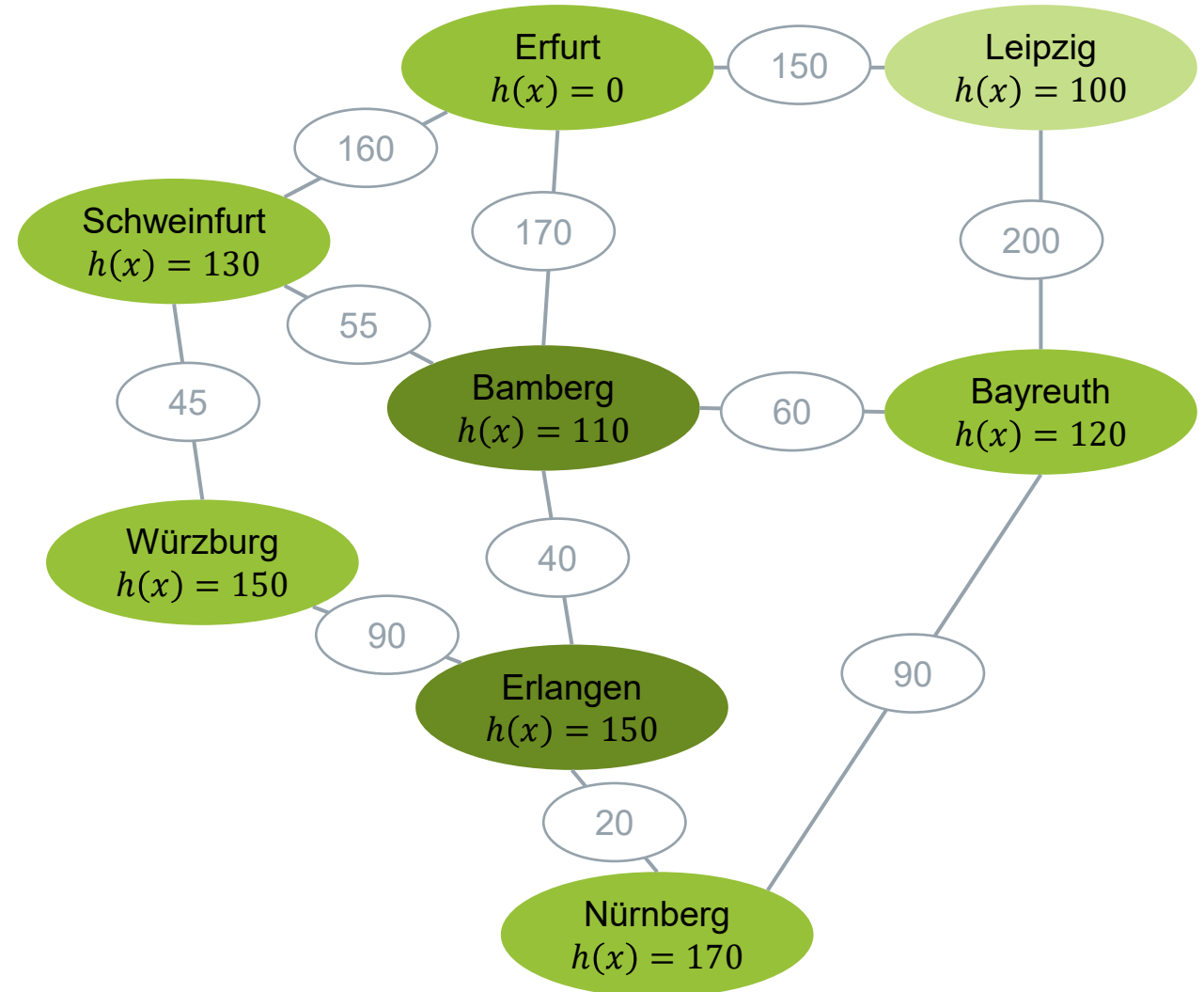


Übung: Der A*-Algorithmus gehört zu den informierten Suchalgorithmen und optimiert das Expansionsverhalten durch die Verwendung einer Heuristik in Richtung des Zielknotens.

Vorgehen

- Wähle günstigsten Knoten nach Metrik
 $f(x) = g(x) + h(x)$
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

lt.	1	2	3	4
N	∞	$20 + 170_E$	$20 + 170_E$	
ER	0	—	—	
WÜ	∞	$90 + 150_E$	$90 + 150_E$	
BA	∞	$40 + 110_E$	—	
BT	∞	∞	$100 + 120_B$	
EF	∞	∞	$210 + 0_B$	
L	∞	∞	∞	
SW	∞	∞	$95 + 130_E$	

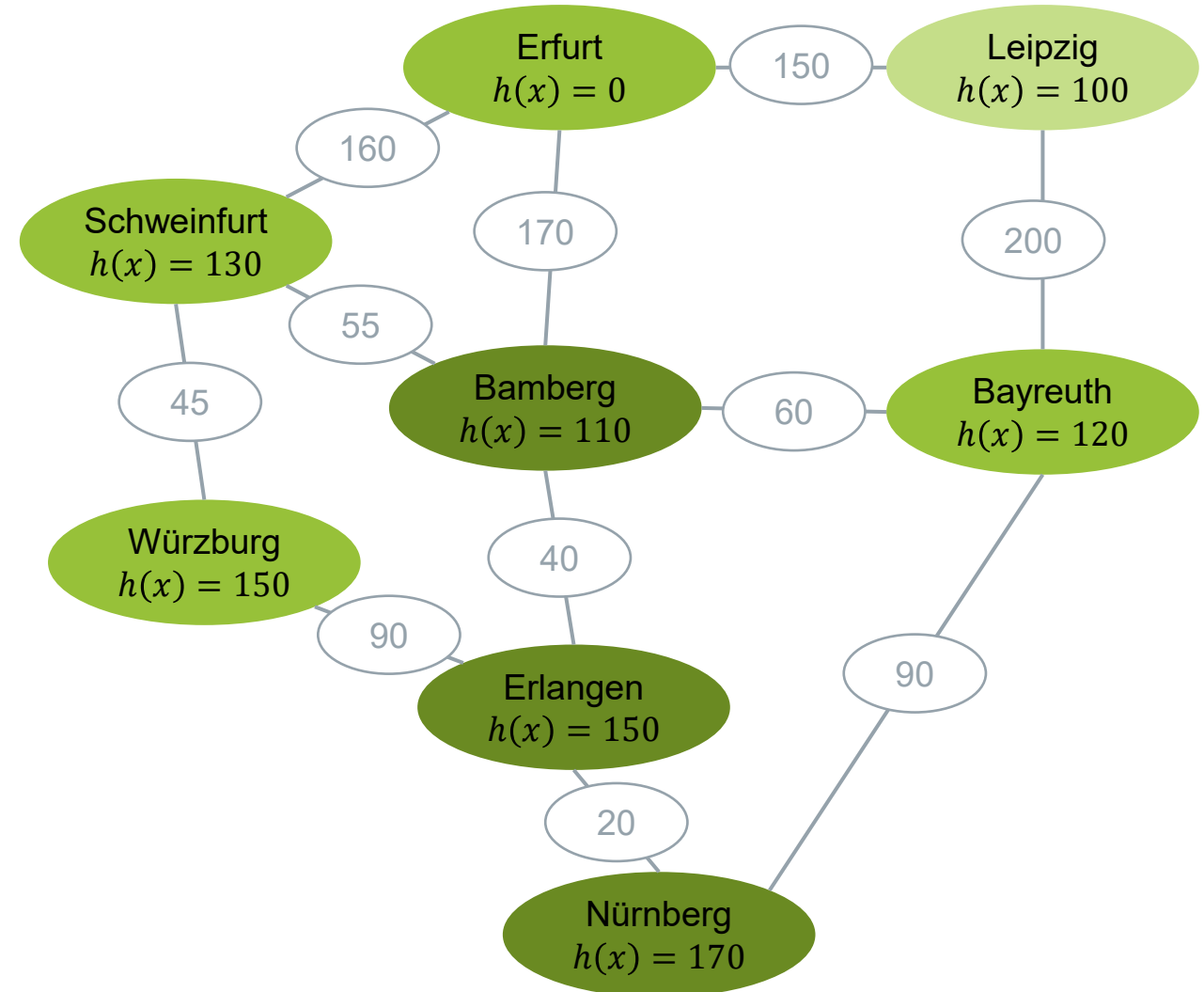


Übung: Der A*-Algorithmus gehört zu den informierten Suchalgorithmen und optimiert das Expansionsverhalten durch die Verwendung einer Heuristik in Richtung des Zielknotens.

Vorgehen

- Wähle günstigsten Knoten nach Metrik
 $f(x) = g(x) + h(x)$
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

lt.	1	2	3	4
N	∞	$20 + 170_E$	$20 + 170_E$	
ER	0	—	—	
WÜ	∞	$90 + 150_E$	$90 + 150_E$	
BA	∞	$40 + 110_E$	—	
BT	∞	∞	$100 + 120_B$	
EF	∞	∞	$210 + 0_B$	
L	∞	∞	∞	
SW	∞	∞	$95 + 130_E$	

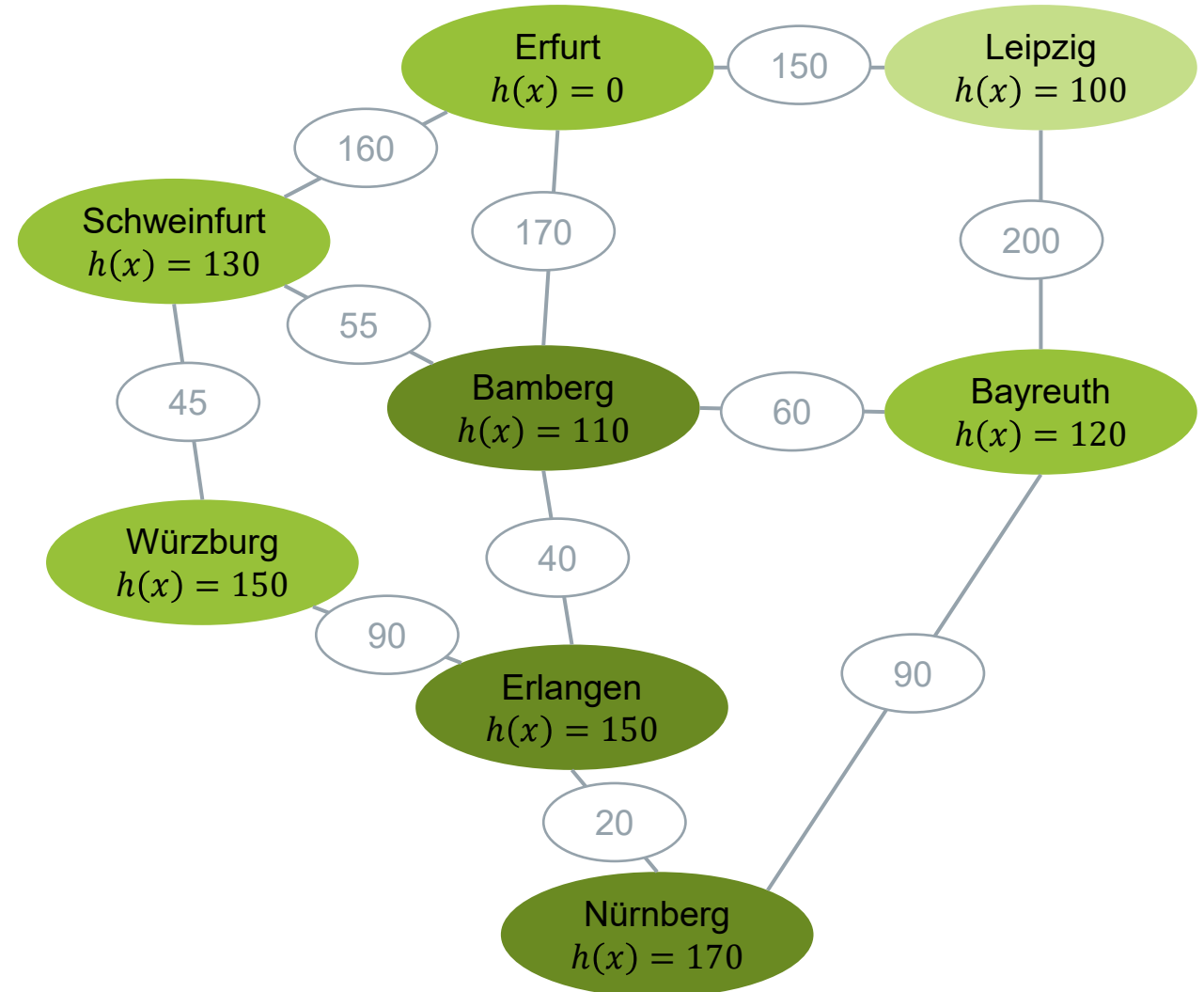


Übung: Der A*-Algorithmus gehört zu den informierten Suchalgorithmen und optimiert das Expansionsverhalten durch die Verwendung einer Heuristik in Richtung des Zielknotens.

Vorgehen

- Wähle günstigsten Knoten nach Metrik
 $f(x) = g(x) + h(x)$
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

lt.	1	2	3	4
N	∞	$20 + 170_E$	$20 + 170_E$	–
ER	0	–	–	–
WÜ	∞	$90 + 150_E$	$90 + 150_E$	$90 + 150_E$
BA	∞	$40 + 110_E$	–	–
BT	∞	∞	$100 + 120_B$	$100 + 120_B$
EF	∞	∞	$210 + 0_B$	$210 + 0_B$
L	∞	∞	∞	∞
SW	∞	∞	$95 + 130_E$	$95 + 130_E$



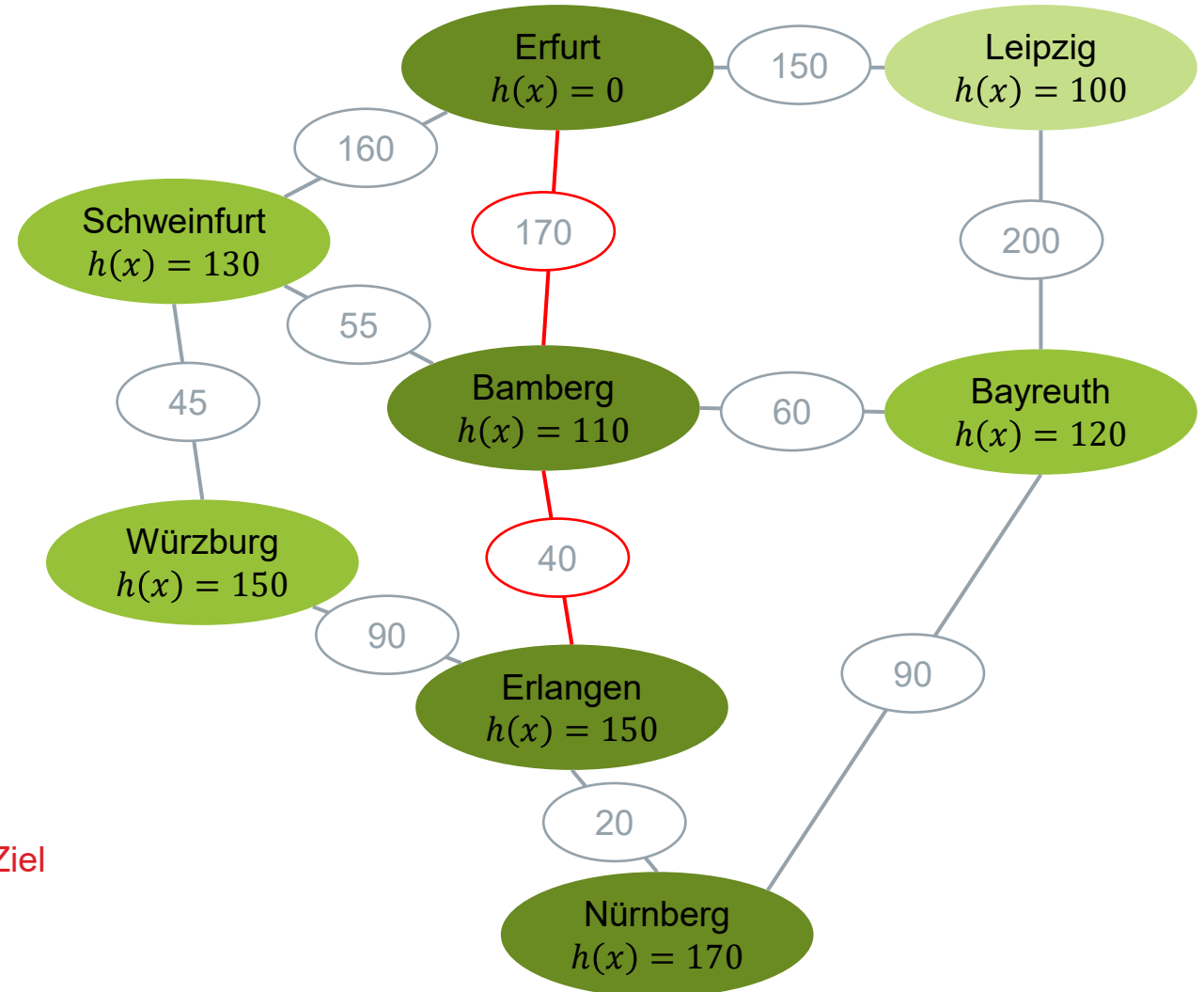
Übung: Der A*-Algorithmus gehört zu den informierten Suchalgorithmen und optimiert das Expansionsverhalten durch die Verwendung einer Heuristik in Richtung des Zielknotens.

Vorgehen

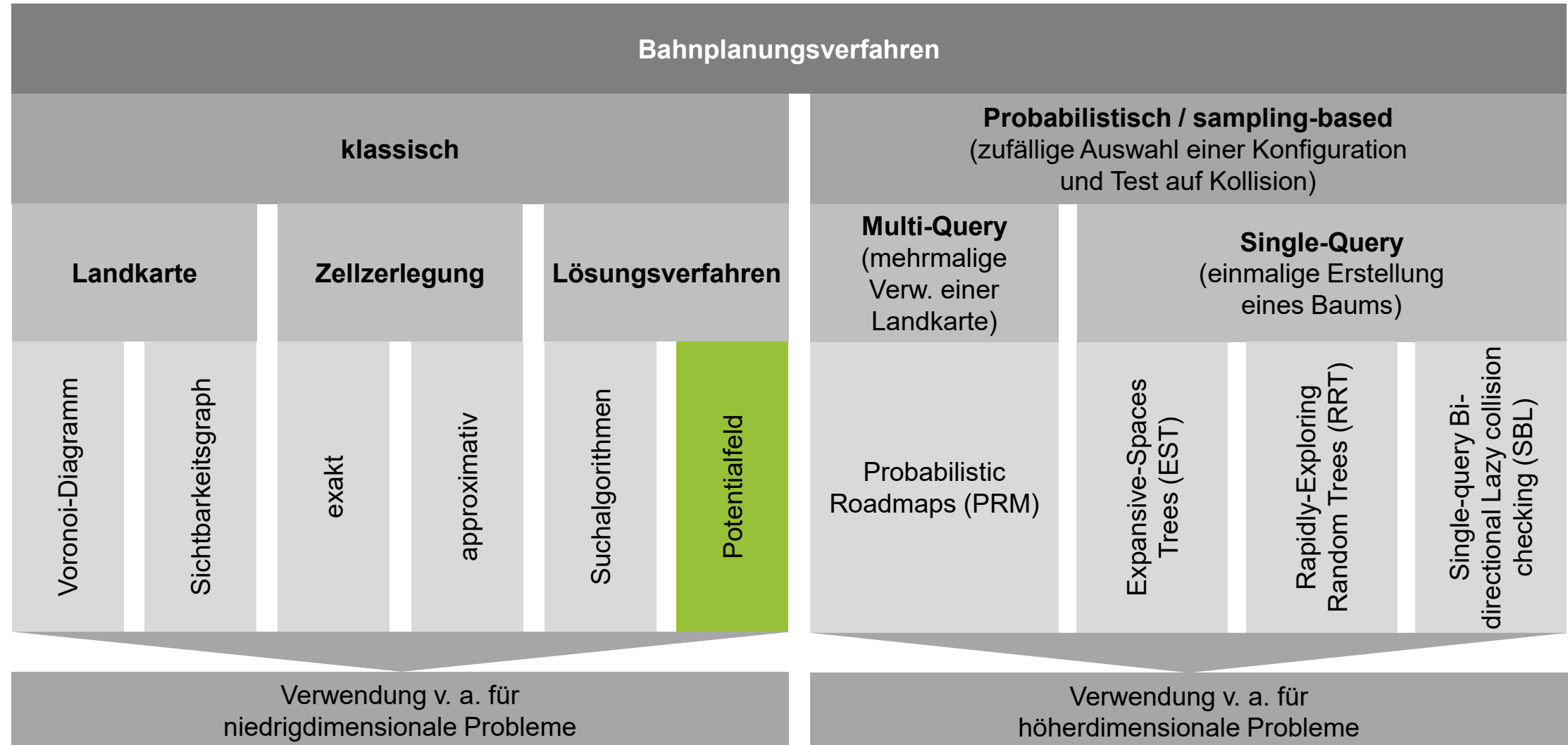
- Wähle günstigsten Knoten nach Metrik
 $f(x) = g(x) + h(x)$
- Expandiere unbekannte Nachbarknoten
- Aktualisiere und speichere Kosten
- Wiederholen bis Zielknoten erreicht ist

lt.	1	2	3	4
N	∞	$20 + 170_E$	$20 + 170_E$	–
ER	0	–	–	–
WÜ	∞	$90 + 150_E$	$90 + 150_E$	$90 + 150_E$
BA	∞	$40 + 110_E$	–	–
BT	∞	∞	$100 + 120_B$	$100 + 120_B$
EF	∞	∞	$210 + 0_B$	$210 + 0_B$
L	∞	∞	∞	∞
SW	∞	∞	$95 + 130_E$	$95 + 130_E$

Ziel



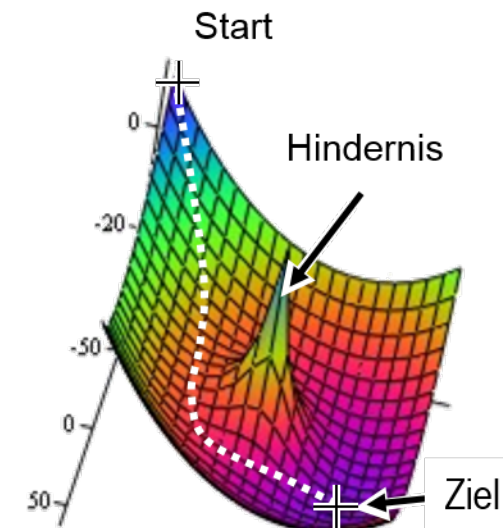
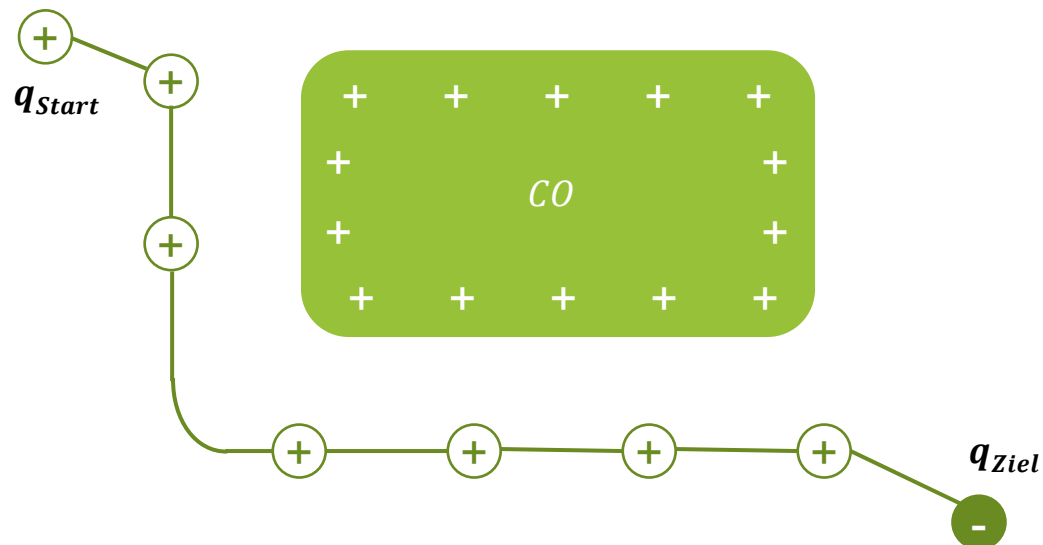
Bahnplanungsverfahren für Roboter gliedern sich in klassische und probabilistische (bzw. sampling-based) Verfahren.



Bei der Potentialfeldmethode wird der Roboter über ein virtuelles Potentialgefälle von der Start- zur Zielkonfiguration gelenkt.

Potentialfeldmethode

- Der Roboter bewegt sich unter dem Einfluss von Kräften, welches ein Potentialfeld auf ihn ausübt
- Roboter und Hindernisse → virtuelles **positives Potential**
- Zielkonfiguration → virtuelles **negatives Potential**
- Vgl. geladenes Teilchen im elektrischen Feld
- Einsatz als globales oder lokales Verfahren (es wird nur eine begrenzte Umgebung der aktuellen Konfiguration betrachtet, womit aufwändige Vorberechnungen entfallen)



Die Potentialfunktion $U(q)$ weist jeder Konfiguration einen skalaren Wert zu, der sich aus dem Potential der Zielkonfiguration und der Hindernisse ergibt.

- Ein **Potentialfeld** U ist eine Skalarfunktion über dem Freiraum

$$U: C_{free} \rightarrow \mathbb{R}$$

- Über die Potentialfunktion $U(q)$ wird jeder Konfiguration ein skalarer Funktionswert zugewiesen, der als Energie angesehen werden kann

- Die **Kraft** in einem Punkt q des Potentialfeldes ist der **negative Gradient** in diesem Punkt

$$F(q) = -\nabla U(q)$$

- Durch den Gradienten der Potentialfunktion wird ein Vektorfeld definiert

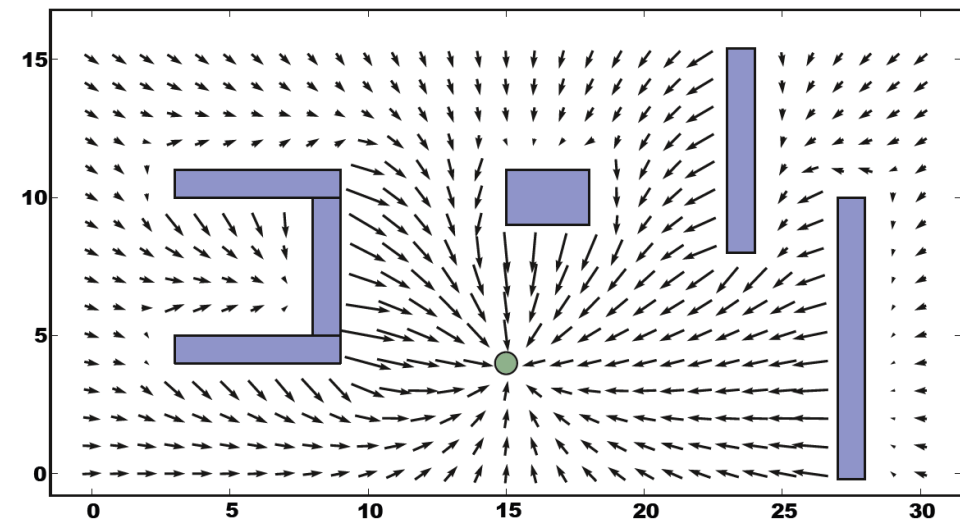
Gradient von U : $grad U = \nabla U(q) = \begin{bmatrix} \frac{\partial U}{\partial q_1}(q) \\ \vdots \\ \frac{\partial U}{\partial q_m}(q) \end{bmatrix}$

mit

U : Potentialfunktion

q : Konfiguration

∇ : Nabla-Operator $\nabla = \left(\frac{\partial}{\partial q_1}, \frac{\partial}{\partial q_2}, \dots, \frac{\partial}{\partial q_m} \right)$



Bildquelle: Mobile Roboter (J. Hertzberg)

Das von Hindernissen ausgehende Potential bewirkt, dass der Roboter von Hindernissen abgestoßen wird.

Abstoßendes Potential

- Das **abstoßende Potential** $U_{ab}(q)$ von Hindernissen wird bei Annäherung an das Hindernis größer, um eine Kollision zu vermeiden.
- In großem Abstand zu Hindernissen ($> \rho_0$) soll der Roboter nicht beeinflusst werden.

$$U_{ab}(q) = \begin{cases} \frac{1}{2} \eta \left(\frac{1}{\rho(q, q_{obs})} - \frac{1}{\rho_0} \right)^2, & \text{falls } \rho(q, q_{obs}) \leq \rho_0 \\ 0, & \text{falls } \rho(q, q_{obs}) > \rho_0 \end{cases}$$

mit η : Skalierungsparameter, ρ_0 : kritische Distanz, $\rho(q, q_{obs}) = \|q - q_{obs}\|$: minimale Distanz zwischen Roboter und Hindernis

- Der negative **Gradient** des **abstoßenden Potentials** zeigt von den Hindernissen weg.

$$F_{ab} = -\nabla U_{ab}(q) = \begin{cases} \eta \left(\frac{1}{\rho(q, q_{obs})} - \frac{1}{\rho_0} \right) \cdot \frac{1}{\rho(q, q_{obs})^2} \cdot \frac{q - q_{obs}}{\rho(q, q_{obs})}, & \text{falls } \rho(q, q_{obs}) \leq \rho_0 \\ 0, & \text{falls } \rho(q, q_{obs}) > \rho_0 \end{cases}$$

Das von der Zielkonfiguration ausgehende Potential bewirkt, dass der Roboter angezogen wird.

Anziehendes Potential

- Das **anziehende Potential** $U_{an}(q)$ wird mit zunehmender Entfernung zur Zielkonfiguration größer.
- **Lineare Funktion** der Distanz zum Ziel

$$U_{an} = k \cdot \|q - q_{Ziel}\|$$

$$F_{an} = -\nabla U_{an}(q) = -k \cdot \frac{q - q_{Ziel}}{\|q - q_{Ziel}\|}$$

- **Quadratische Funktion** der Distanz zum Ziel

$$U_{an}(q) = \frac{1}{2} k \|q - q_{Ziel}\|^2$$

$$F_{an} = -\nabla U_{an}(q) = -k \cdot (q - q_{Ziel})$$

mit k : frei wählbarem Skalierungsparameter; $\|q - q_{Ziel}\|$: Distanz zwischen q und q_{Ziel} ,

z. B. für $m = 2$: $\|q - q_{Ziel}\| = \sqrt{(q_1 - q_{Ziel_1})^2 + (q_2 - q_{Ziel_2})^2}$



Oft wird die Kombination aus linearer und quadratischer Funktion verwendet

- Lineare Funktion → in großer Entfernung zum Ziel
- Quadratische Funktion → nahe am Ziel

Das Potentialfeld wird als Summe aus anziehendem Potential und abstoßendem Potential berechnet.

- Die Summe der einwirkenden Kräfte bestimmt die Richtung der Bewegung
- Für das Potentialfeld gilt:

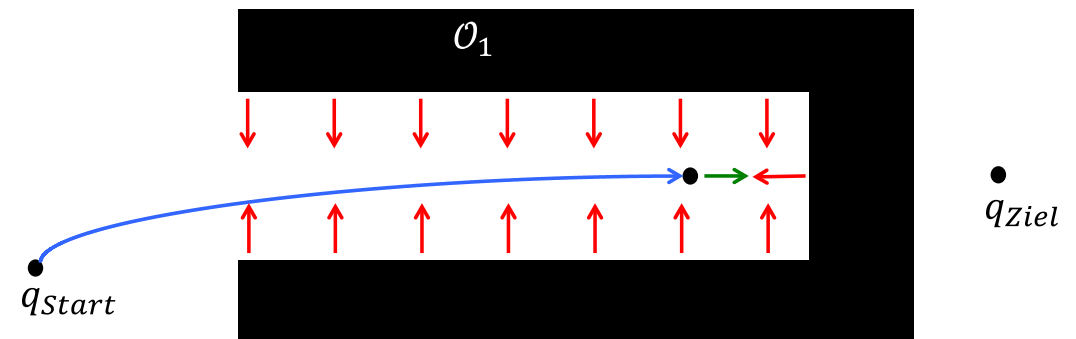
$$U(q) = U_{an}(q) + \sum U_{ab}(q)$$

- Für das Kräftefeld gilt:

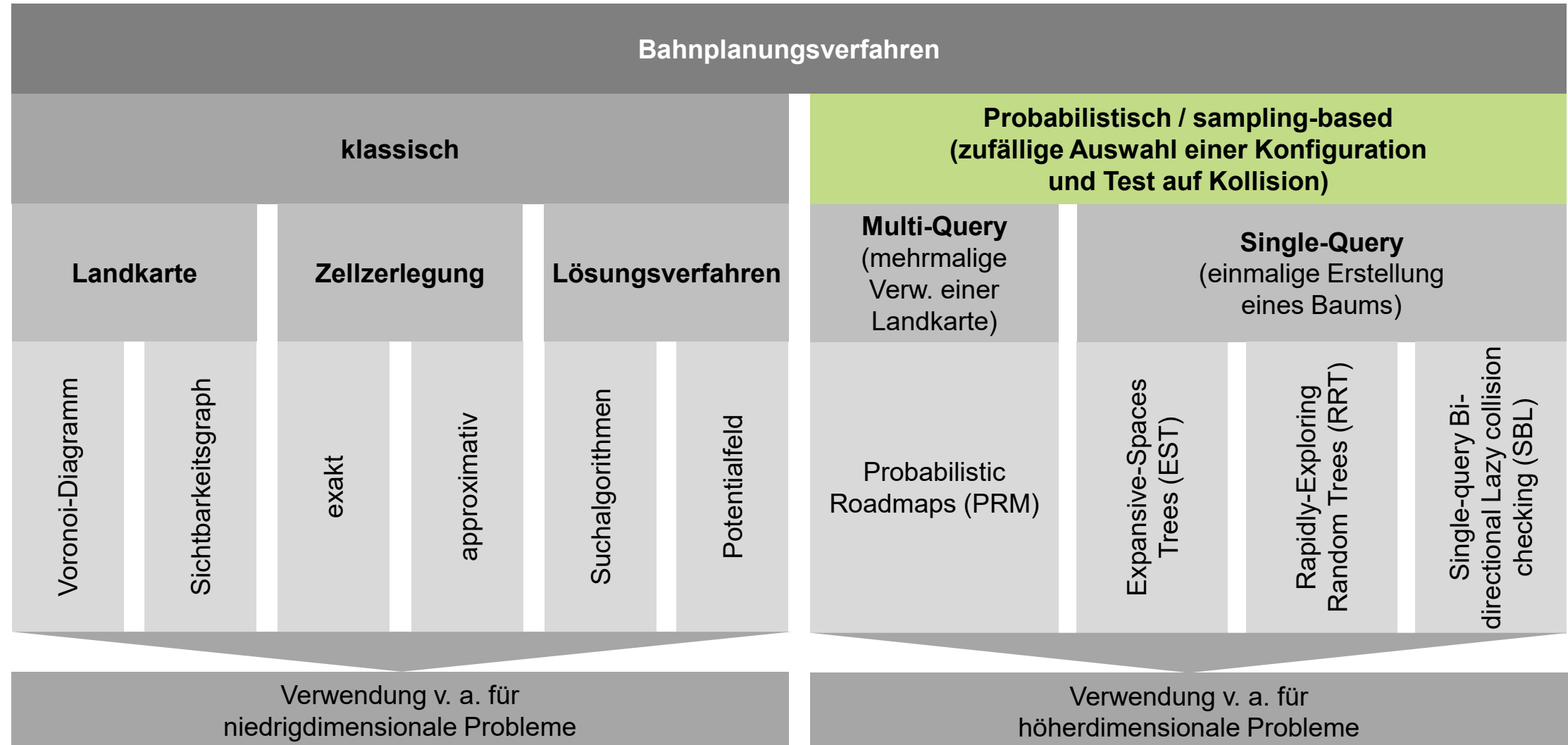
$$F(q) = F_{an}(q) + \sum F_{ab}(q)$$

Lokale Minima

- Durch die Summation von U_{an} und U_{ab} kann U lokale Minima besitzen.
- Dann entspricht das abstoßende dem anziehenden Potential, aber der Roboter hat die Zielkonfiguration noch nicht erreicht.
- Es existieren Ansätze wie bspw. den „Randomized Path Planner“, um mittels Zufallsbewegungen zu einer Konfiguration zu gelangen, die eine weitere Bewegung zur Zielkonfiguration zulässt.



Bahnplanungsverfahren für Roboter gliedern sich in klassische und probabilistische (bzw. sampling-based) Verfahren.



Probabilistische Planer eignen sich gut für höherdimensionale Bahnplanungsprobleme.

Probabilistische Bahnplaner

- Approximation des freien Raums, explizite Repräsentation wird vermieden
→ Effizienter: Eine Lösung kann oft schnell und Ressourcen schonend gefunden werden
- Auswahl einzelner Konfigurationen als **Stichproben**

Unterscheidung der sampling-based Verfahren hinsichtlich

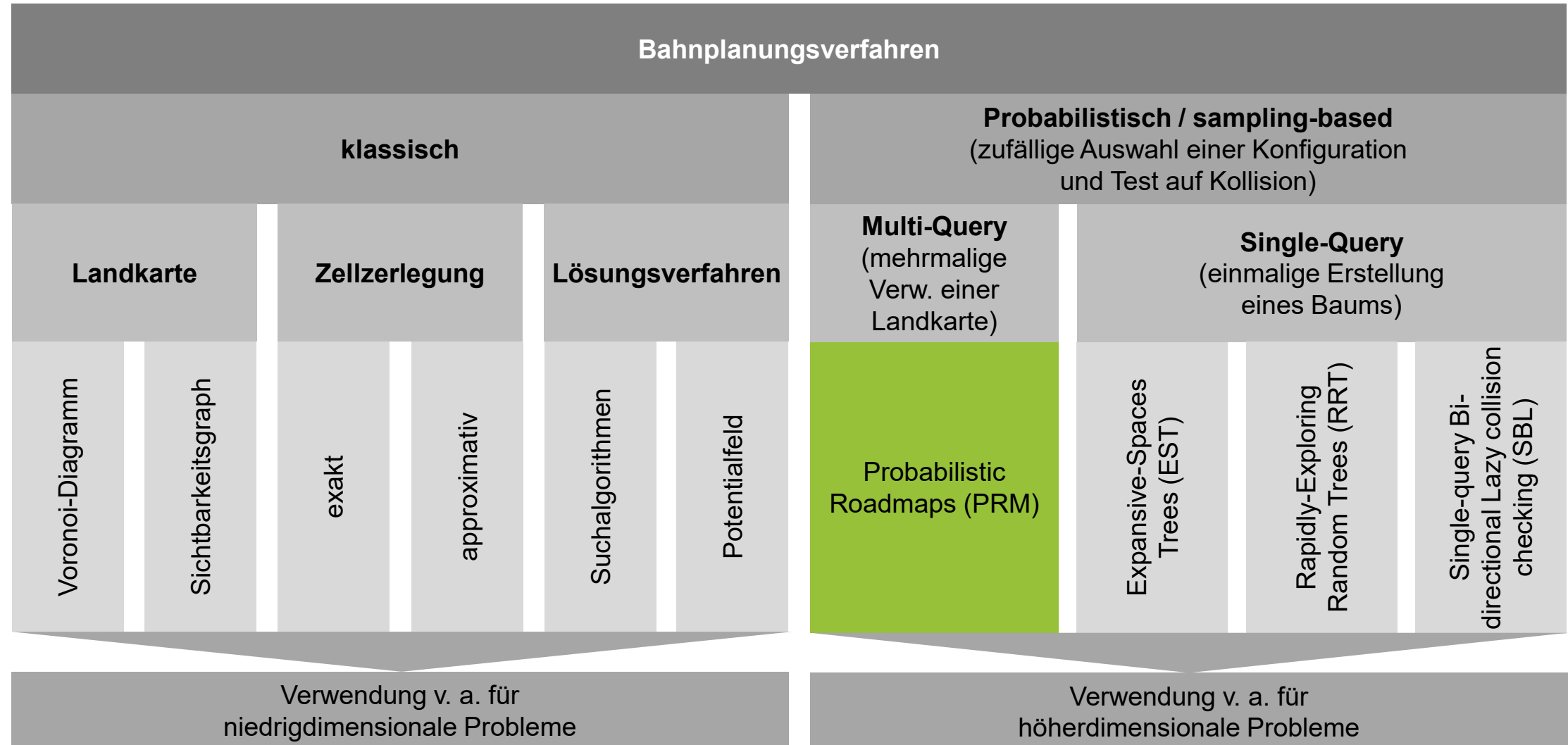
- Auswahl der Stichprobe
- Verbindungsstrategie ausgewählter Konfigurationen zu gültigen Pfaden
- Anfragen bzw. Verwendung der Landkarten/Graphen

Multi-Query: Wiederverwendung einer im Voraus erstellten Landkarte für neue Anfragen

Single-Query: Erstellung eines neuen Suchbaums im Konfigurationsraum für jede Anfrage

- **Probabilistisch-vollständig** bei einer zufallsbasierten Generierung der Stichproben:
Wahrscheinlichkeit einen gültigen Pfad zu finden (falls dieser existiert) nimmt mit zunehmender Anzahl an Stichproben zu und geht im Unendlichen gegen eins

Bahnplanungsverfahren für Roboter gliedern sich in klassische und probabilistische (bzw. sampling-based) Verfahren.



Bei der Methode der probabilistischen Landkarten (probabilistic roadmap (PRM)) wird eine Landkarte aus zufällig ausgewählten, kollisionsfreien Konfigurationen gebildet.

- **Multi-Query:** Mehrere Anfragen in einer gleichbleibenden Umgebung

Schritt 1: Vorverarbeitung

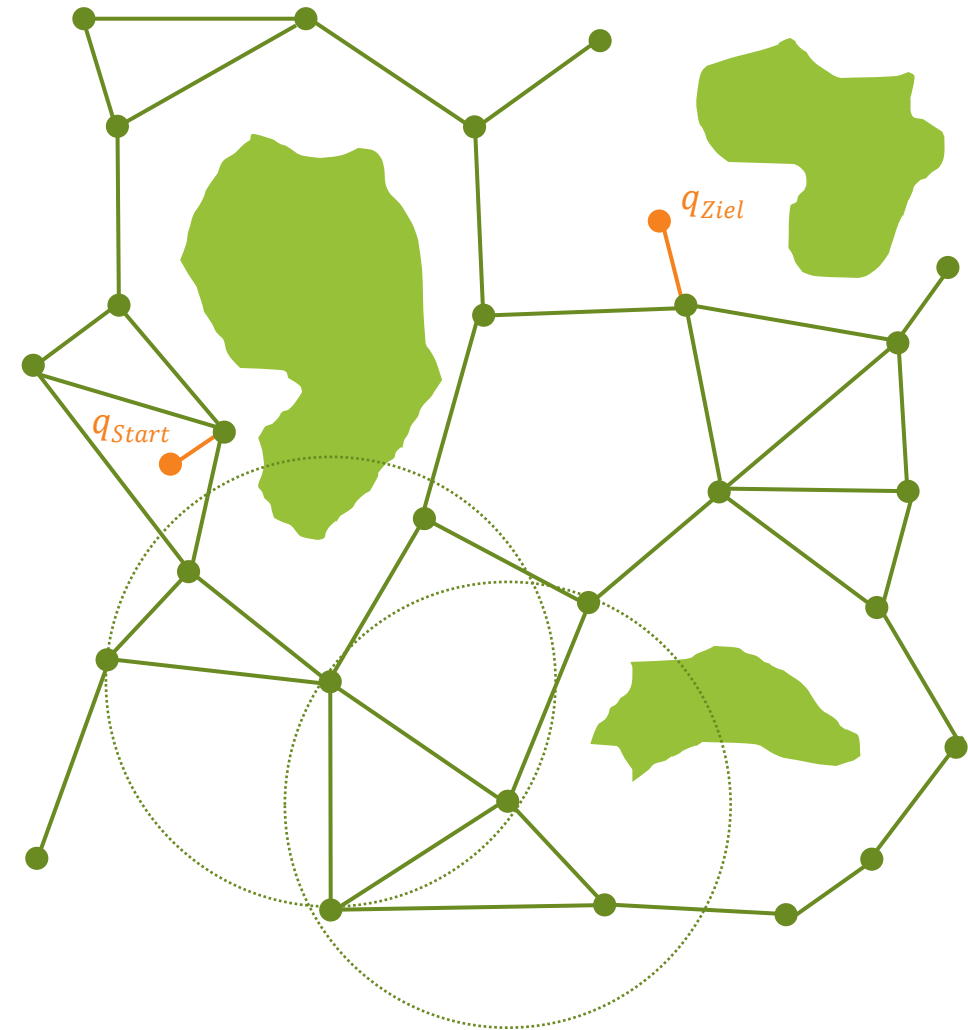
- Zufällige Erzeugung von kollisionsfreien Stichproben (Sampling)
→ N zufällige Konfigurationen in C_{free}
- Stichproben werden über kollisionsfreie Pfade miteinander verbunden d. h.
die k nächsten Nachbarn oder
alle Nachbarn, die innerhalb einer maximalen Distanz liegen

→ Lokale Planung

- Speichern der Landkarte in Form eines Graphen, dessen Knoten die Konfigurationen und dessen Kanten die durch einen lokalen Planer generierten Pfade darstellen

Schritt 2: Anfrage

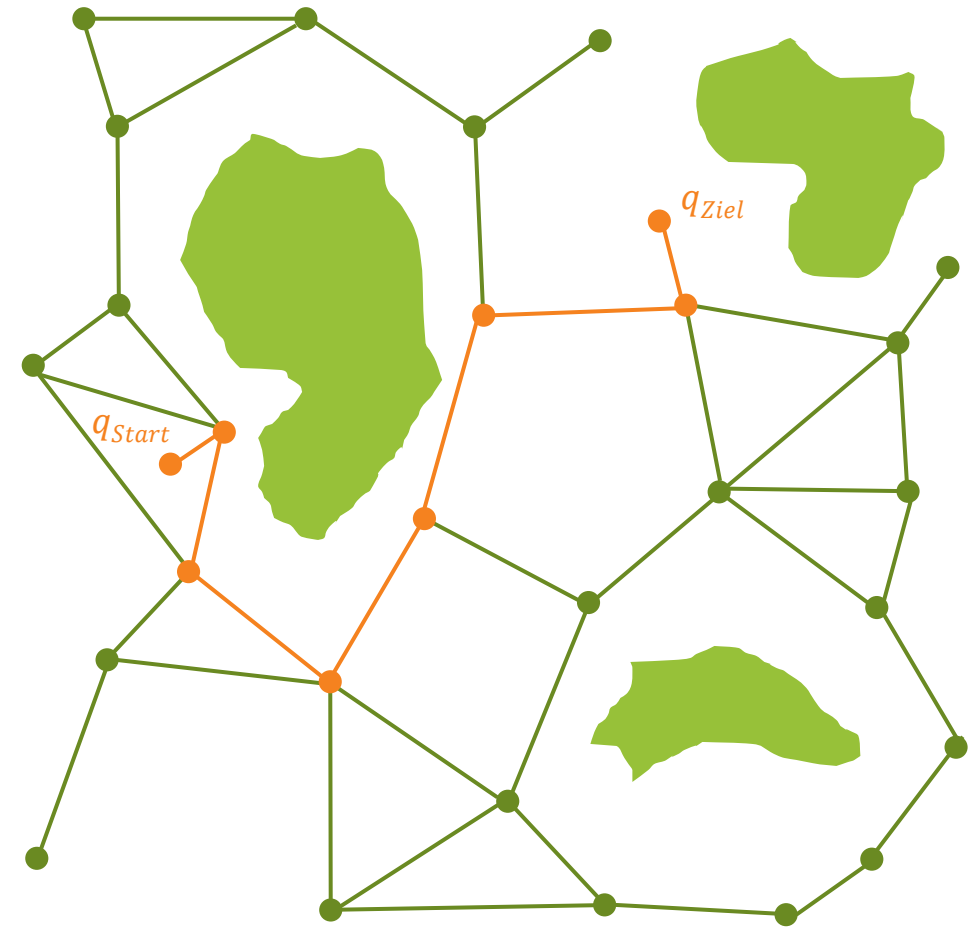
- Verbinde q_{start} und q_{ziel} mit dem Wegenetz
- Suche einen Weg im Graphen (z. B. mit A*)



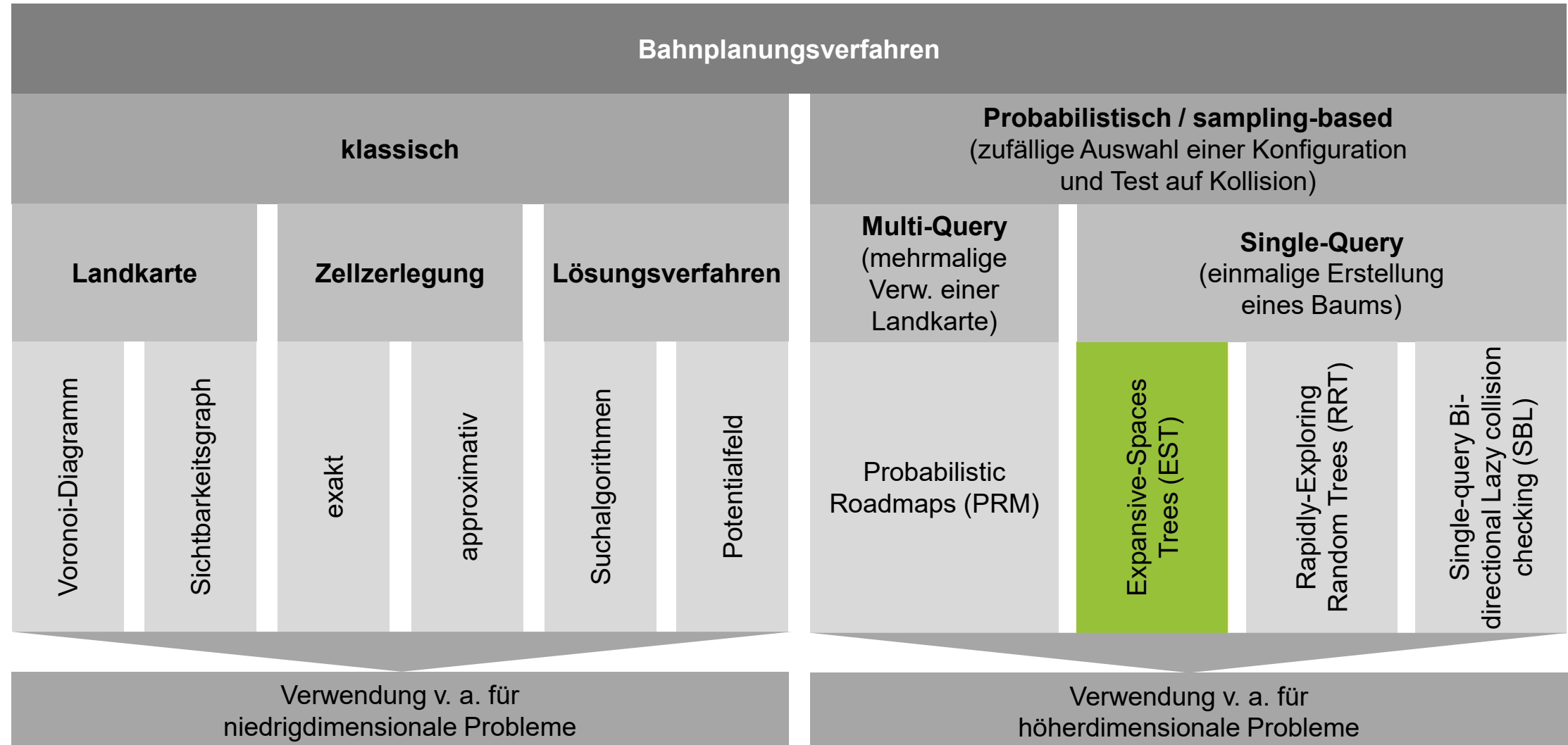
Bei der Probabilistic Roadmap wird die Landkarte einmal erstellt und für mehrere Anfragen verwendet.

Eigenschaften der Probabilistic Roadmap

- Einmalige Konstruktion des Graphen
 - Mehrere Anfragen können effizient beantwortet werden (multit-query)
- Randomisierter Ansatz zur Konstruktion (probabilistisch)
 - Exponentieller Anstieg der Laufzeit mit der Dimension des Konfigurationsraums wird vermieden
- Verfahren hängt stark vom verwendeten Sampling ab
- Problem: Schmale Passagen zwischen Hindernissen
 - Lösungsansatz: Sampling in der Nähe von Hindernissen erhöhen



Bahnplanungsverfahren für Roboter gliedern sich in klassische und probabilistische (bzw. sampling-based) Verfahren.



Die Single-Query-Planer EST und RRT erstellen meist zwei Bäume, ausgehend von der Start- und der Zielkonfiguration, die miteinander verbunden werden.

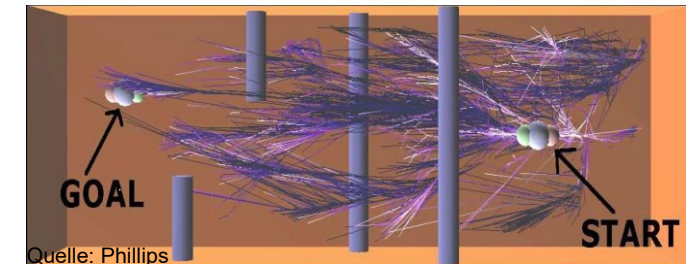
Single-Query-Planer

- Algorithmen zur Einmalanfrage (keine Vorverarbeitung nötig, keine Probleme mit sich verändernder Umgebung)
- Effizient für hochdimensionale Probleme

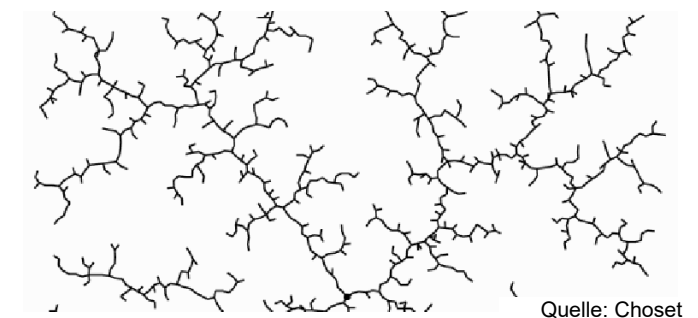
EST und RRT

- Oft werden bei EST und RRT zwei Bäume erstellt, die ihre Wurzeln in der Start- bzw. der Zielkonfiguration haben, in die Richtung des jeweils anderen wachsen und über einen lokalen Planer zusammengeführt werden (**bidirektionale Planer**)
- Es ist ebenfalls möglich, nur einen Baum von der Startkonfiguration ausgehend zu generieren, der in Richtung der Zielkonfiguration wächst.
- Neue Konfigurationen werden in den Randbereichen der bestehenden Bäume generiert und diesen nur hinzugefügt, wenn sie über einen lokalen Planer mit einer bestehenden Konfiguration verbunden werden können.
- Im Gegensatz zur PRM-Methode wird nur der für eine Planungsaufgabe notwendige Bereich des freien Raums abgedeckt. Konfigurationen, die in den Baum aufgenommen werden, hängen von der Start-, der Ziel- und vorher generierten Konfigurationen ab.

Beispielhafter EST



Ausschnitt eines beispielhaften RRT



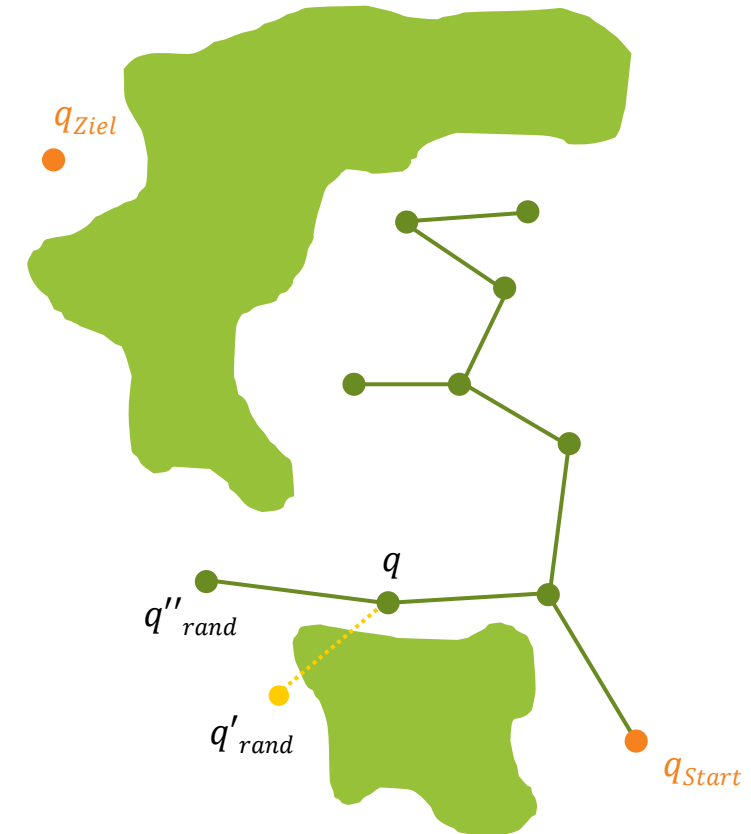
Beim EST (Expansive-Spaces Trees)-Verfahren werden nur Konfigurationen in den Baum aufgenommen, wenn sie eine gültige Verbindung besitzen.

Ablauf

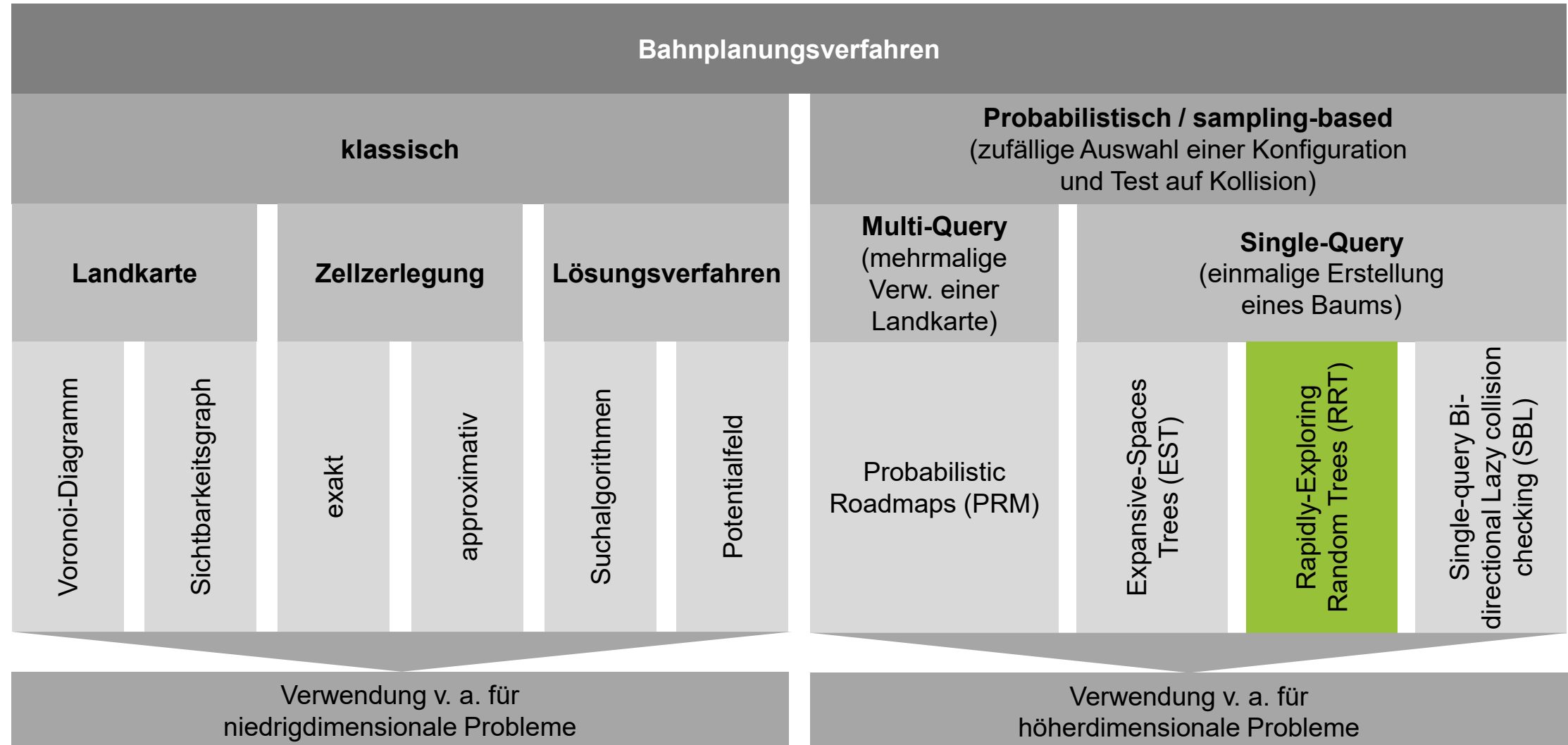
- Auswahl einer Konfiguration q aus dem bestehenden Baum, von der aus der Baum wachsen soll
- Wahrscheinlichkeitsdichtefunktion (Konfigurationen mit weniger Nachbarkonfigurationen werden bevorzugt)
(zu Beginn wird q_{Start} bzw. q_{Ziel} verwendet)
- Generierung einer zufälligen Stichprobenkonfiguration q_{rand} in der Umgebung von q mit Hilfe einer Gleichverteilung
- Verbinden von q und q_{rand} mit lokalem Planer, bei Erfolg wird q_{rand} als neuer Knoten und die Kante (q, q_{rand}) dem bisherigen Baum hinzugefügt
- Wiederholung des Prozesses bis eine vorgegebene Anzahl an Konfigurationen dem Baum hinzugefügt wurden

Bidirektionale ESTs

- Der lokale Planer versucht eine neue Konfiguration q_{rand} des einen Baums mit einer definierten Anzahl nächster Konfigurationen des zweiten Baums zu verbinden



Bahnplanungsverfahren für Roboter gliedern sich in klassische und probabilistische (bzw. sampling-based) Verfahren.



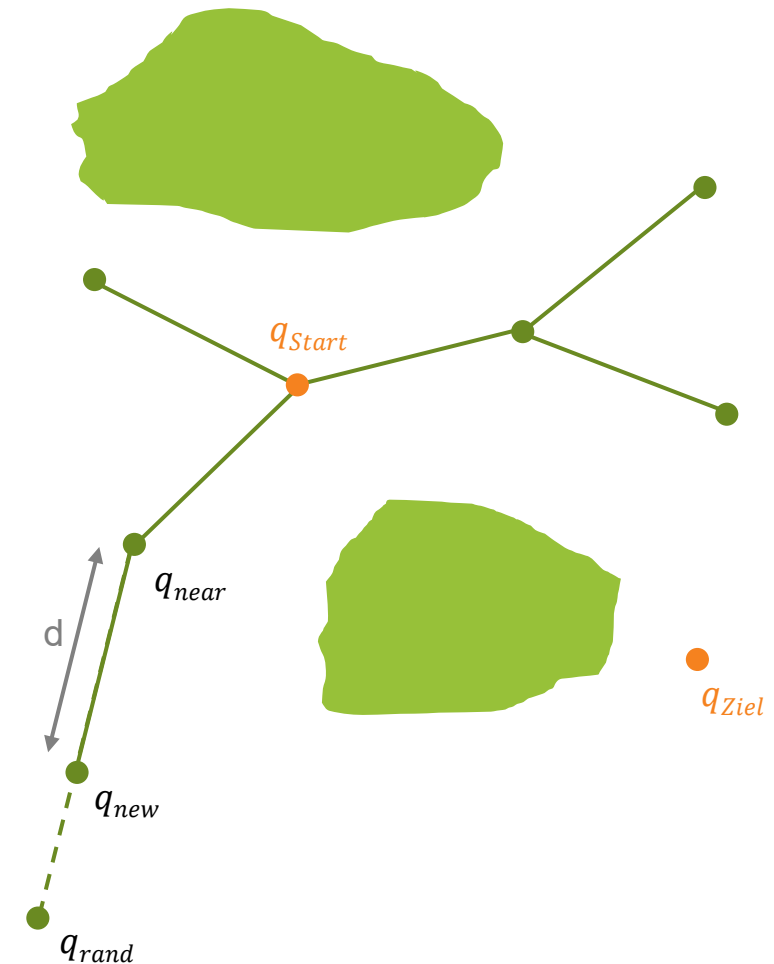
Bei der Methode der Rapidly-Exploring Random Trees (RRT) wird, anders als bei ESTs, die Anzahl benachbarter Konfigurationen nicht bestimmt.

Ablauf

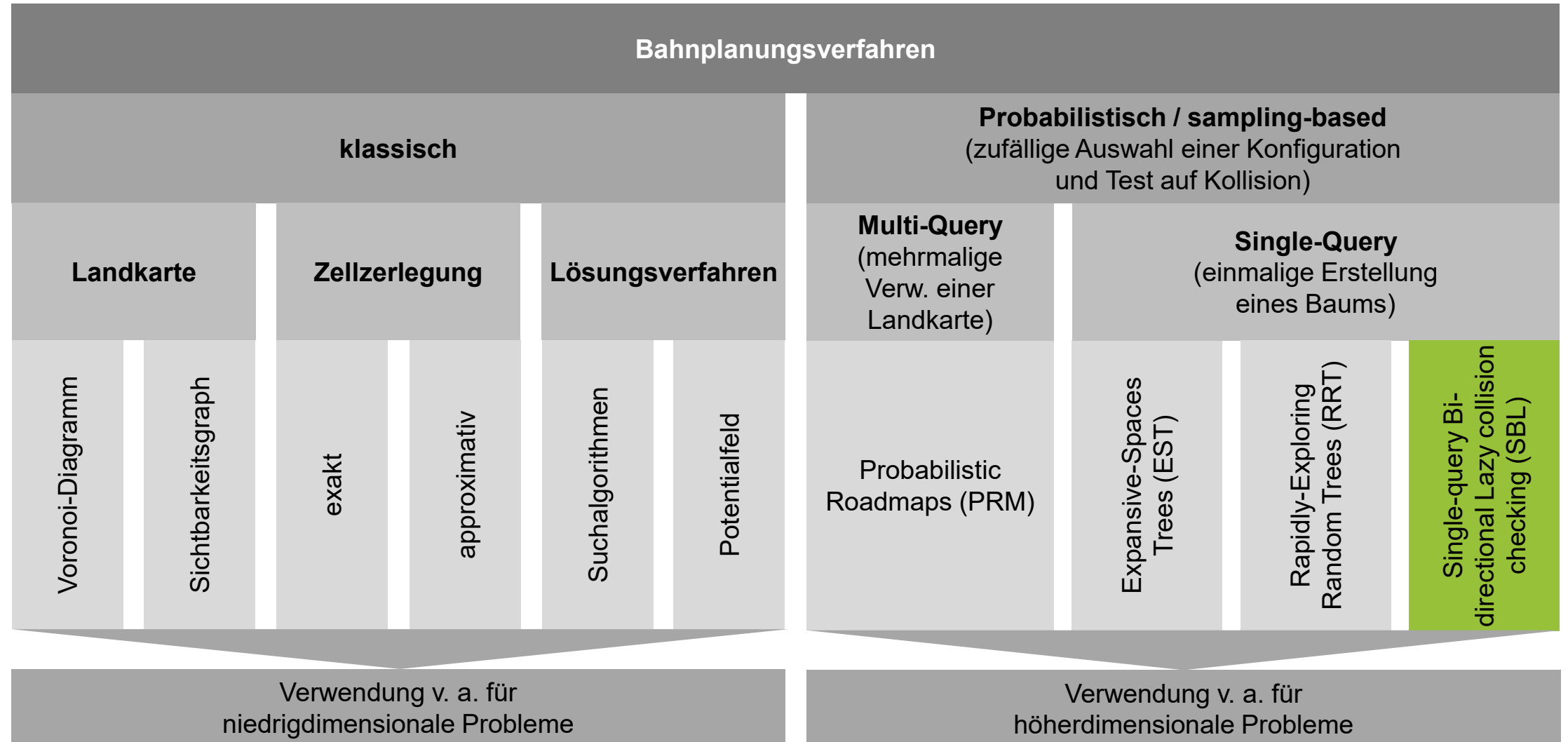
- Zufällige Auswahl einer Konfiguration q_{rand} im Konfigurationsraum
- Ermittlung des nächsten Nachbarn q_{near} im bestehenden Baum
- Einfügen einer neuen Konfiguration q_{new} auf der Verbindungsline zwischen q_{near} und q_{rand} mit vorgegebenem Abstand zu q_{near} (Schrittweite d)
- Prüfung auf Kollision: Falls die neue Konfiguration q_{new} kollisionsfrei ist, wird sie mit der zugehörigen neuen Kante dem Baum hinzugefügt
- Wiederholung des Prozesses bis eine vorgegebene Anzahl an Versuchen, neue Konfigurationen zu erstellen, durchgeführt wurde

Bidirektionale RRTs

- Es wird versucht, eine neue Konfiguration q_{new} des einen Baums mit der am nächsten liegenden Konfiguration des zweiten Baums zu verbinden



Bahnplanungsverfahren für Roboter gliedern sich in klassische und probabilistische (bzw. sampling-based) Verfahren.

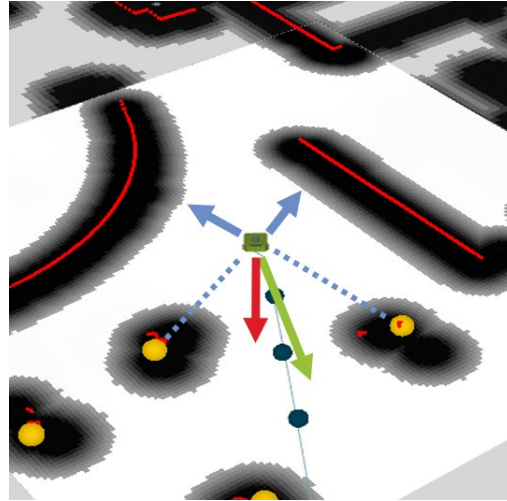


Der SBL (Single-query Bi-directional Lazy collision-checking)-Planer ist ein bidirektionales EST-Verfahren mit einer „faulen“ Kollisionsprüfung.

Eigenschaften des SBL-Planers

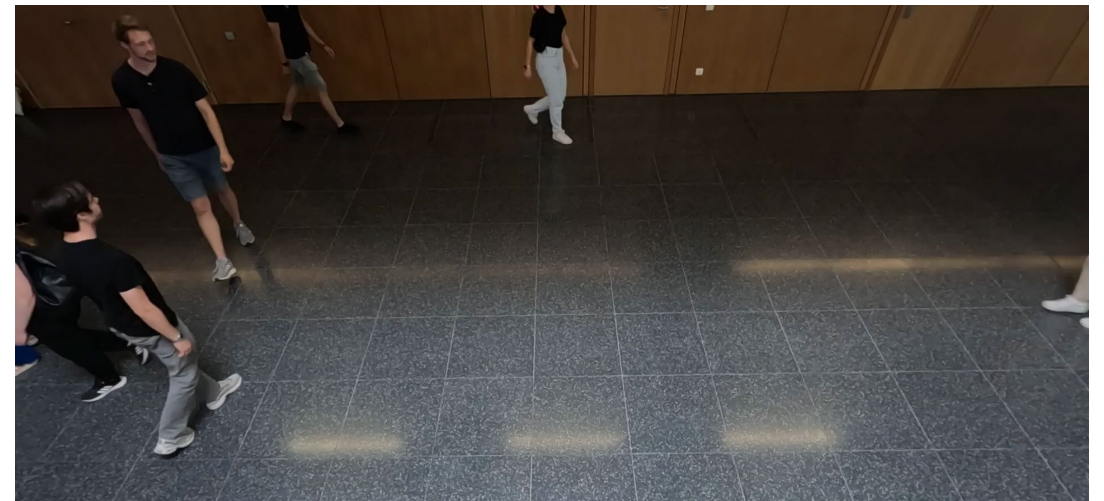
- Der SBL-Planer erstellt **zwei Bäume**, die in q_{Start} bzw. in q_{Ziel} ihre Wurzeln haben.
- Dadurch, dass **Kanten und Knoten** des Suchbaumes **erst auf Kollision** geprüft werden, falls sie Teil eines existierenden Pfads von q_{Start} nach q_{Ziel} sind, kann erheblich Zeit eingespart werden (lazy collision checking)
- Im Gegensatz zum ursprünglichen EST-Verfahren wird zur **Auswahl einer zu expandierenden Konfiguration** anstelle einer Wahrscheinlichkeitsdichtefunktion ein **vereinfachtes Gitterverfahren** eingesetzt. Dabei wird über den Konfigurationsraum ein grobes Gitter gelegt und zufällig eine Konfiguration aus einer nicht leeren Zelle ausgewählt. Zellen mit weniger Konfigurationen im bestehenden Baum werden bevorzugt.

Der vorgeschlagene Ansatz DRL-SFM zur lokalen Pfadplanung für autonome Rollstühle in belebten Umgebungen eröffnet weitere Forschungspotenziale.



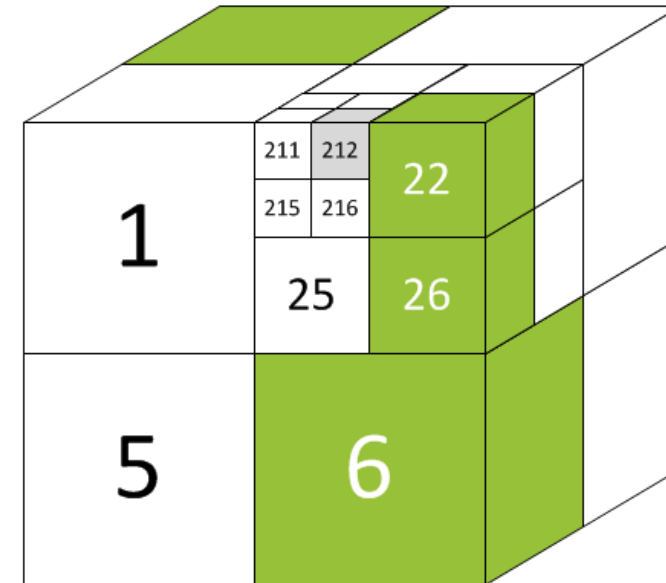
DRL-SFM

- Belohnungsfunktion → SFM
- Beobachtung → HuMap
- Training und Evaluation in der Simulation
- Höhere **Erfolgsraten** ↑
- Seltenerer **Überschreitung von intimen Distanzen** ↓
- Verwendung des Ansatzes mit dem realen Rollstuhlsystem

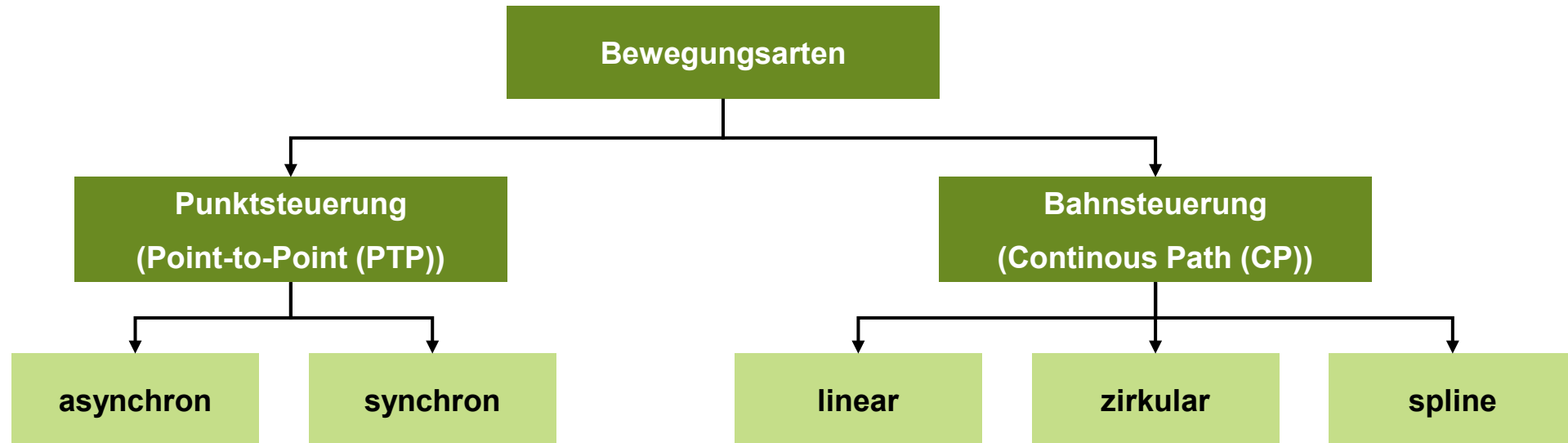


Die Bahnplanung ist ein Teilbereich der Roboterbewegungsplanung; Steuerung und Regelung ermöglichen die Ausführung von Bewegungen.

- Aufgaben der Bewegungsplanung
- Bahnplanungsproblem und Begrifflichkeiten der Bahnplanung
- Bahnplanungsverfahren
- **Bewegungssteuerung**
- Grundlagen der Regelungstechnik



Die Steuerungsarten von Roboterbewegungen beschreiben die Art des Verfahrens zwischen zwei Zielstellungen, d. h. zwei Endeffektorposen des Roboters.



- Einfachste Art des Verfahrens zwischen zwei Punkten bzw. Stellungen
- Bewegung auf einer für den Anwender nicht vorhersehbaren Bahn ohne definierte Orientierung des Endeffektors
- Anwendungen: Punktschweißen, Handhabungsaufgaben

- Ziel: Exakte Einhaltung einer vorgegebenen Bahn
- Bewegung des TCPs z.B. auf einer Linearbahn oder Zirkularbahn (Kreisbogen)
- Anwendungen: Bahnschweißen, Laserschneiden, Lackieren ect.

Die Punkt-zu-Punkt-Steuerung (PTP) beschreibt die einfachste Art des Verfahrens zwischen zwei Punkten bzw. Stellungen.

Punktsteuerung (PTP)

- Einfach, aber unvorhersehbare Bewegungen
- Berechnung von Zwischenwerten in Achskoordinaten
- Achskoordinaten ändern sich nur geringfügig bzw. wachsen und fallen stetig, keine schnellen und großen Achsbewegungen

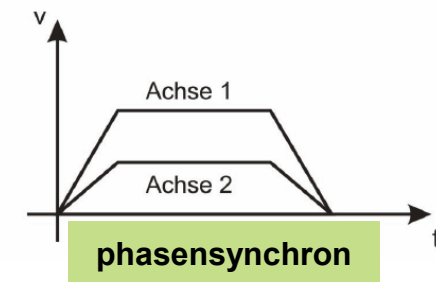
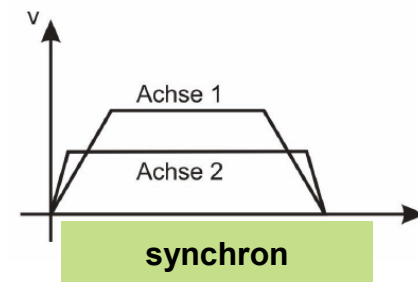
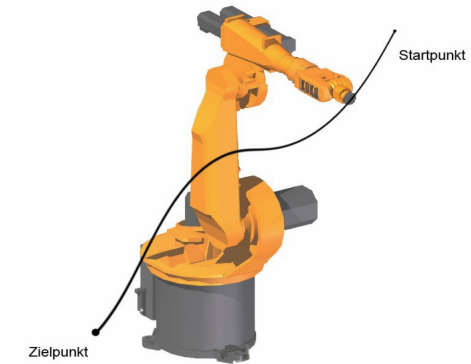
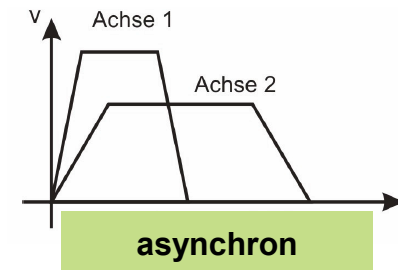
Asynchrones PTP

- Jede Achse verfährt zur Zielstellung vollständig unabhängig von den anderen Achsen
- Achsen kommen im Allgemeinen nicht zum gleichen Zeitpunkt zum Stillstand, die langsamste Achse bestimmt die Gesamtdauer

Synchrones PTP

- Alle Achsen erreichen ihre Zielstellung zum gleichen Zeitpunkt
- Leitachse (Achse mit der größten Bahndauer) gibt die Länge der Bewegung d. h. die Gesamtdauer vor
- Vollsynchron/phasensynchron: Nicht nur die Dauer aller Gelenkbewegungen ist gleich, sondern auch die Zeitspannen zum Beschleunigen und Abbremsen

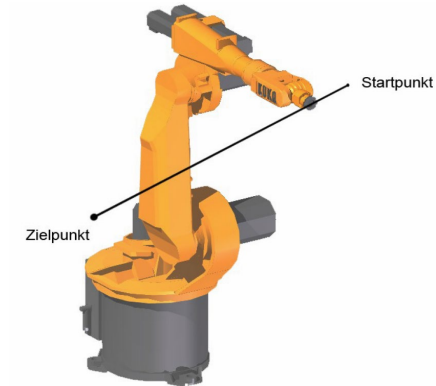
Punktsteuerung (Point-to-Point - PTP)



Bei der Bahnsteuerung (CP) werden vorgegebene Bahnen möglichst exakt abgefahren.

Bahnsteuerung

- Definierte Bahn, aber aufwändige Berechnung
- Koordiniertes Verfahren der Achsen, um eine gewünschte Raumkurve zu erzeugen
- Berechnung von Zwischenstellungen in kartesischen Koordinaten und Rücktransformation zur Berechnung der Gelenkwerte
- Ggf. größere Änderungen von Achswerten notwendig
- Zwischenwerte der Bahn können außerhalb des Arbeitsbereiches des Roboters liegen, ggf. Bahn nicht ausführbar



Linearbahn

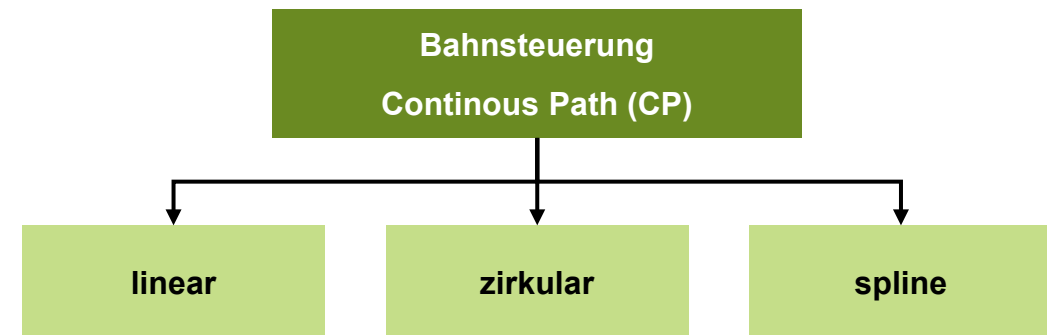
- Bewegung auf einer Geraden

Zirkularbahn

- Bewegung auf einem Kreisbogen, Angabe von Startpunkt, Hilfspunkt und Zielpunkt

Spline-Bahn

- Beliebige Kurven möglich



Als Bewegungsprofile werden das Rampenprofil sowie das Sinuidenprofil unterschieden.

Interpolation: Berechnung von Bahnzwischenpunkten auf Basis von Geschwindigkeits- und Beschleunigungsvorgaben

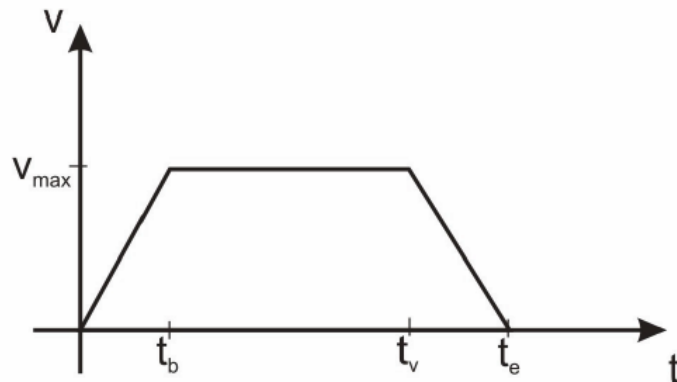
Rampenprofil

- Konstante Beschleunigung auf maximale Geschwindigkeit, (dann konstante Geschwindigkeit,) dann konstante negative Beschleunigung
- Sprunghafte Änderung der Beschleunigung und Kraft, hohe mechanische Belastung
- Dreiecksförmiges Profil bei zeitoptimaler Bahn

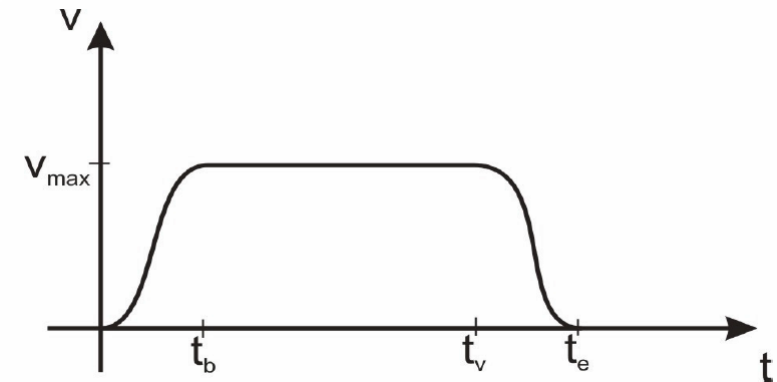
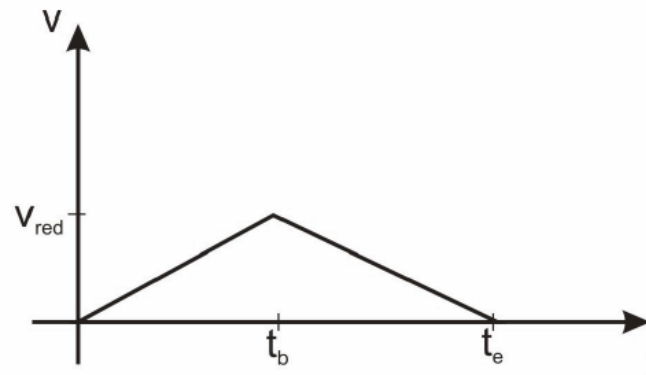
Sinuidenprofil

- Stetige Beschleunigung gemäß einer Zeitfunktion
- Veränderung der Kraft stetig, Bewegung ruckfrei

Trapezförmig

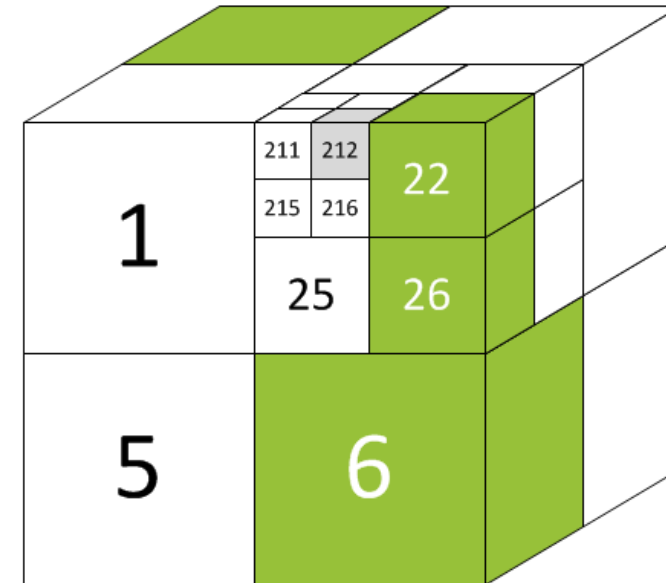


Dreiecksförmig



Die Bahnplanung ist ein Teilbereich der Roboterbewegungsplanung; Steuerung und Regelung ermöglichen die Ausführung von Bewegungen.

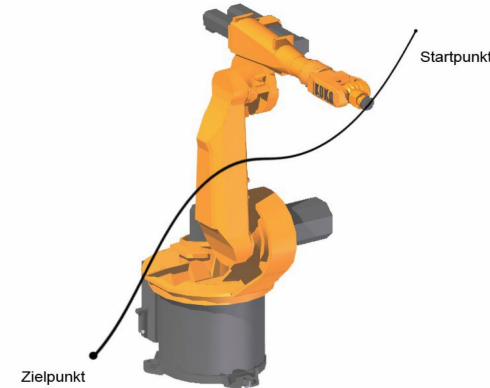
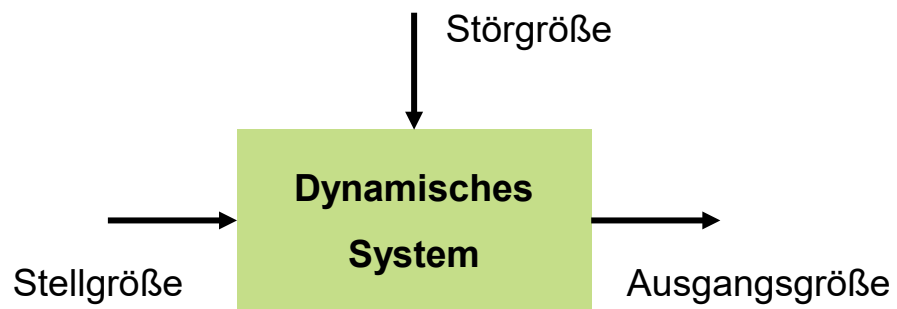
- Aufgaben der Bewegungsplanung
- Bahnplanungsproblem und Begrifflichkeiten der Bahnplanung
- Bahnplanungsverfahren
- Bewegungssteuerung
- **Grundlagen der Regelungstechnik**



Die Regelungstechnik beinhaltet die selbsttätige, gezielte Beeinflussung dynamischer Prozesse während des Prozessablaufs.

Regelungstechnik

- Selbsttätige, gezielte Beeinflussung dynamischer Prozesse während des Prozessablaufs
- Unvollständige Systemkenntnis, insbesondere bei Einwirkung von Störgrößen
- Methoden der Regelungstechnik sind allgemeingültig



Anwendungsbeispiel

Bewegen des Endeffektors eines Roboters möglichst exakt auf einer bestimmten Trajektorie unter verschiedenen Lasten

Der **Ausgangsgröße** soll mittels der **Stellgröße** ein **Sollverhalten** aufgeprägt werden gegen den Einfluss einer **Störgröße**, die nur unvollständig bekannt ist.

Die Darstellung eines Regelungssystems erfolgt im Wirkungsplan.



Regelung

- Messeinrichtung
- Regeleinrichtung
- Stelleinrichtung

Darstellung im Wirkungsplan

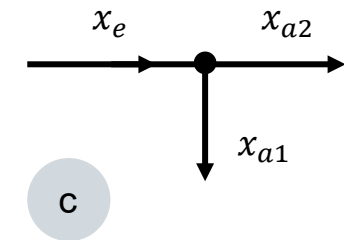
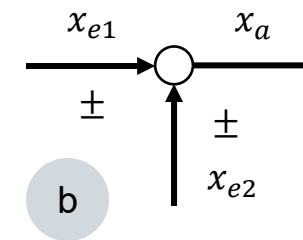
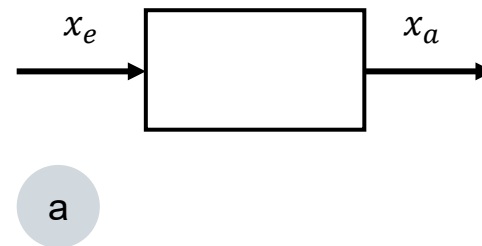
- Blocksymbol (a)
- Additionsstelle (b)

$$x_a = \pm x_{e1} \pm x_{e2}$$

- Verzweigungsstelle (c)

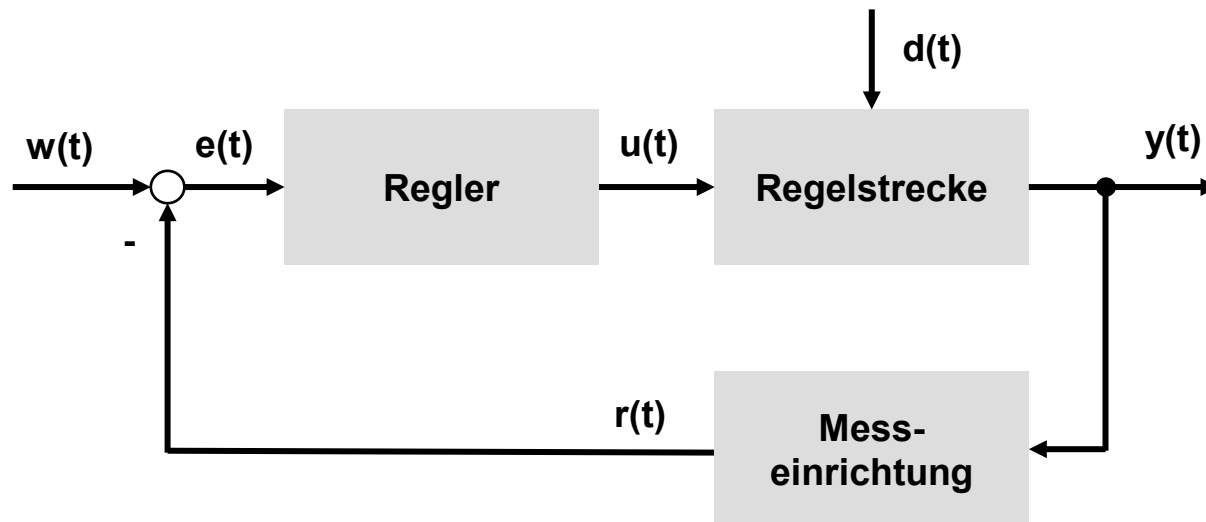
$$x_{a1} = x_{a2} = x_e$$

- Wirkungslinien mit Wirkungsrichtung



Die Regelung zeichnet sich durch einen geschlossenen Wirkungskreis aus.

Vereinfachter Wirkungsplan eines Regelkreises



w : Führungsgröße
 u : Stellgröße
 y : Regelgröße
 r : Rückführgröße
 d : Störgröße
 e : Regeldifferenz/
 Regelabweichung

Regeleinrichtung

- Wirkt der Abweichung vom Soll-Wert kontinuierlich entgegen
- Negative Rückkopplung

Die Wirkungsweise der Regelung durch negative Rückkopplung wird anhand eines Beispiels gezeigt.

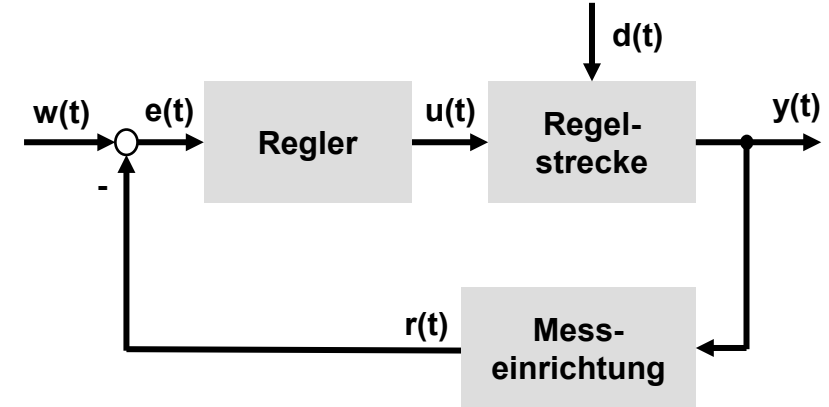
Sollwert von y : y_s

Messeinrichtung: $r = K_j \cdot y \quad K_j > 0$

Wahl der Führungsgröße: $w = K_j \cdot y_s$

Dann: $e = w - r$
 $= K_j \cdot y_s - K_j \cdot y = K_j \cdot (y_s - y)$

Es sei zunächst: $y = y_s \Rightarrow e = 0$
 (Regelung in Ruhe)



d wird größer $\rightarrow$ y werde abgesenkt $\rightarrow$

r wird abgesenkt $\rightarrow$ e wird angehoben $\rightarrow$

u wird angehoben $\rightarrow$ y wird angehoben mit der Tendenz,
den Sollwert wieder anzunehmen

$\rightarrow$ Die Störgröße wird ausgeglet.

w: Führungsgröße
 u: Stellgröße
 y: Regelgröße
 r: Rückführgröße
 d: Störgröße
 e: Regeldifferenz/
 Regelabweichung

Bei einer Regelung können alle Störeinflüsse indirekt erfasst und ausgeglichen werden.

Steuerung

„Die **Steuerung** ist ein Vorgang in einem System, bei dem eine oder mehrere variable Größen als **Eingangsgrößen** andere variable Größen als **Ausgangsgrößen** aufgrund der dem System eigenen Gesetzmäßigkeit beeinflussen.“
(DIN IEC 60050-351)

Regelung

„Die **Regelung** ist ein Vorgang, bei dem fortlaufend eine variable Größe, die **Regelgröße**, erfasst (gemessen), mit einer anderen variablen Größe, der **Führungsgröße**, verglichen und im Sinne einer Angleichung an die Führungsgröße **beeinflusst** wird.“
(DIN IEC 60050-351)

Vorteile der Steuerung

- geringerer automationstechnischer Aufwand
- stabil

Nachteile der Steuerung

- Einhaltung vorgegebener Soll-Werte nicht möglich
- Nicht alle Störungen können erfasst werden

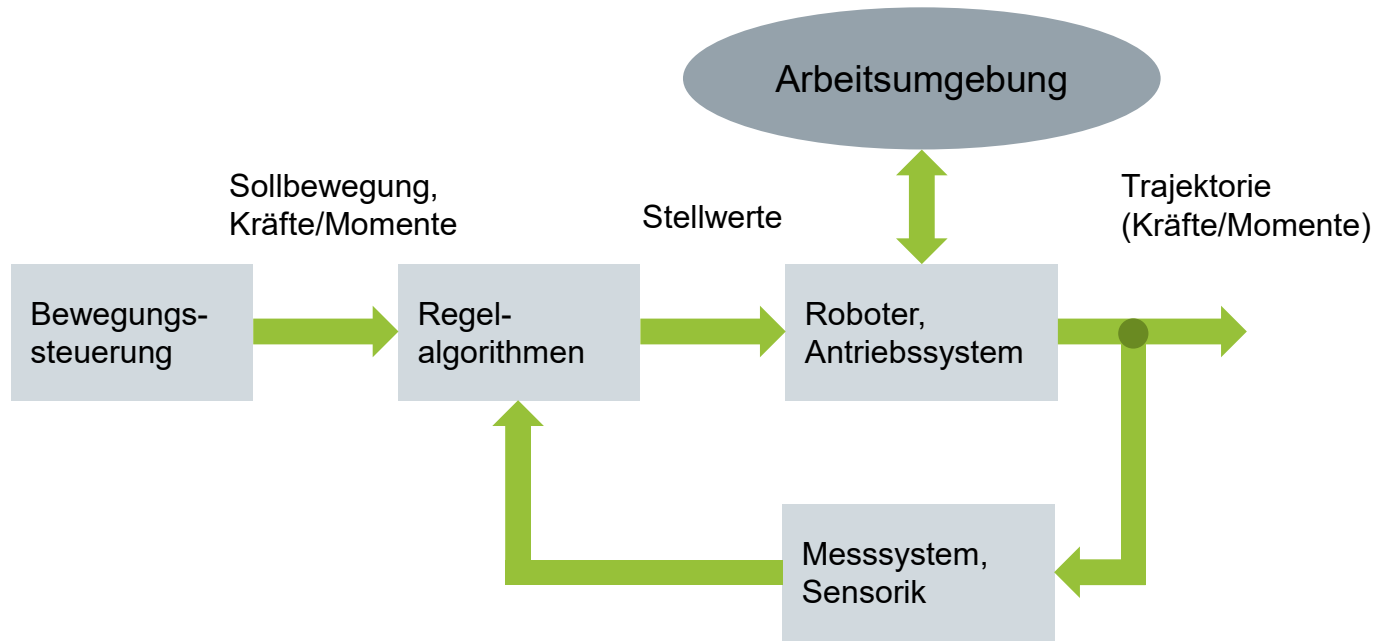
Vorteile der Regelung

- Einhaltung eines vorgegebenen Wertes einer gemessenen Größe so genau wie nötig
- Indirekte Erfassung aller Störeinflüsse

Nachteile der Regelung

- höherer gerätetechnischer Aufwand
- Regelung kann erst dann reagieren, wenn eine Abweichung vorhanden ist

Bei der Regelung von Robotern wird zwischen interner und externer Regelung unterschieden, weiterhin erfolgt eine Einteilung in Kraftregelung und Lageregelung.



Externe Regelung

- Sensoren liefern Informationen über die Umgebungssituation
- Angepasste Bewegungssollwerte werden ermittelt
- Koordinatentransformation und Übermittlung an interne Regelung

Interne Regelung

- Gelenkregelung
- Interne Sensoren liefern Informationen über die Bewegung der Achsen

Regelung von Industrierobotern

Lageregelung

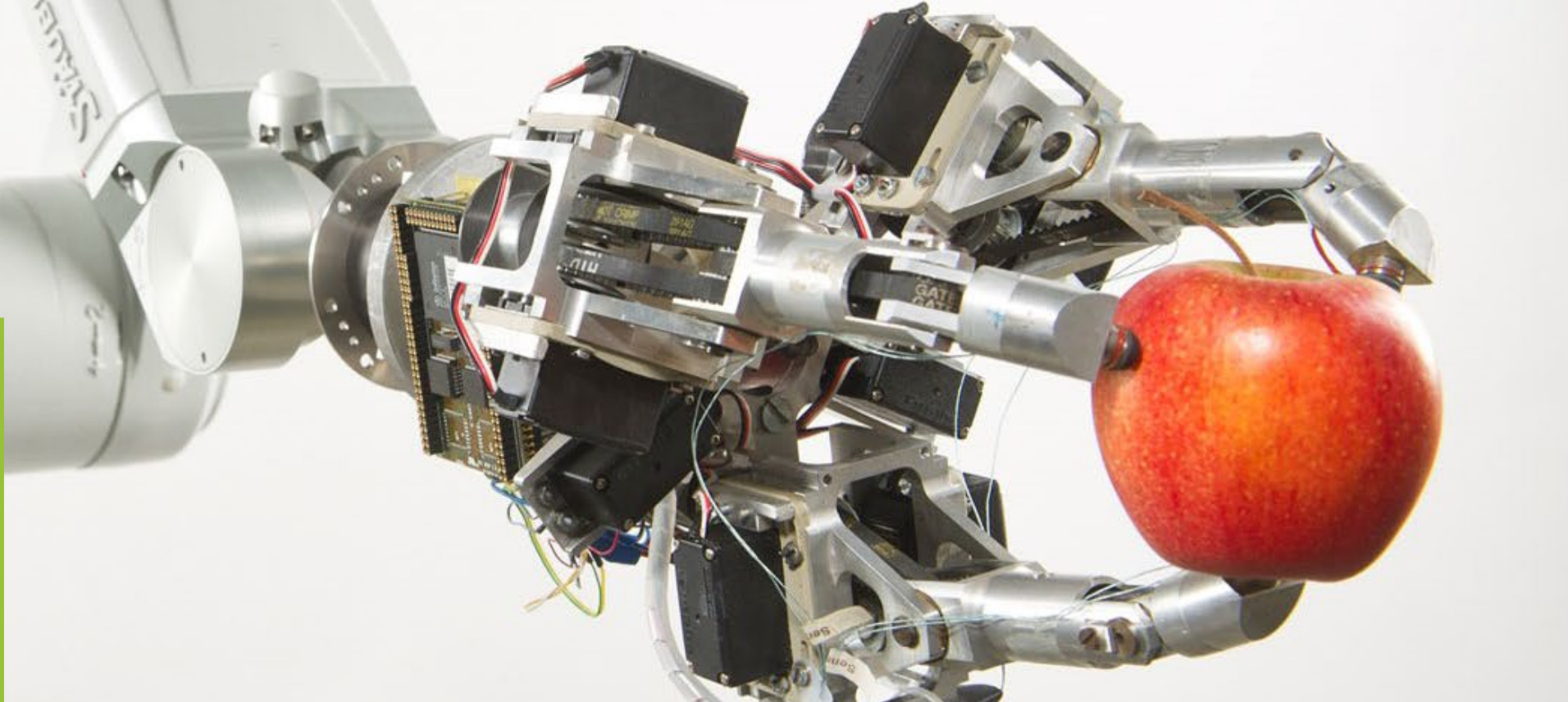
- Definiertes Bewegungsverhalten soll unabhängig von Bearbeitungskräften/Momenten und aufgenommenen Lasten ausgeführt werden

Kraftregelung

- Definierte Kräfte/Drehmomente sollen auf die Arbeitsumgebung ausgeübt werden

Hybride Regelung

- Mischformen zwischen Lage- und Kraftregelungen, aufgabenspezifischer Wechsel zwischen den Regelungen



Prof. Dr.-Ing. Jörg Franke

**Lehrstuhl für Fertigungsautomatisierung
und Produktionssystematik**

Friedrich-Alexander-Universität Erlangen-Nürnberg



**Friedrich-Alexander-Universität
Technische Fakultät**

DANKE